Darshana Opticals (Pvt) Ltd.

Software Design Specification (SDS)

By

E.K.A.K. AMALKA

SE/2021/008

SANDARU MIHIRAN

SE/2021/010

T.H. IMASHA

SE/2021/053

D.S.U. WEERAKKODY

SE/2021/018

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Version History

Version No.	Date	Author(s)	Summary of Changes
1.0	19.04.2026	Sachini Weerakkody, Hiruni Imasha, Sandaru Mihiran, Kavindu Amalka	Initial SDS draft.
1.1 (Review Candidate)	25.10.2026	Hiruni Imasha	Revision initiated under DDP-088 to align the SDS with the implemented OSMS architecture, accepted ADR-001–015, updated database design, user roles, business workflows, security controls, and deployment architecture. Affected diagrams and known implementation limitations are being reviewed and updated.

Table of Contents

Chapter 1: Architectural Design 1
1.1 System Context Diagram 1
1.2 High-level Architecture 2
1.3 Cloud-Native Architecture 7
1.4 Justification for Architecture Choice 9
1.5 Scalability & Maintainability Considerations 11
Chapter 2: Database Design 14
2.1 ER Diagram 14
2.2 Normalization Explanation 16
Chapter 3: Class Diagram 26
Chapter 4: Sequence Diagrams 29
4.1 User Visit and Homepage 29
4.2 Authentication (Register and Login) 31
4.3 Product Discovery (Search, Filter, Detail) 34
4.4 Cart Management 37
4.5 Checkout and Order Confirmation 40
4.6 Appointment Booking 44
4.7 Chatbot Interaction 48
4.8 Staff Order Processing and Tracking 53
4.9 Inventory Management and Reorder Alerts 56
4.10 Doctor Session Management 59
4.11 Prescription Recording and History 62
Chapter 5: UI/UX Design 65
5.1 Screen Mock-ups for Key User Interfaces 65
5.2 User flow / Navigation Diagrams 87
5.3 Accessibility Considerations 102
Chapter 6: Security Design 111
6.1 Authentication Mechanism 111
6.2 Authorization Levels 114
6.3 Data Protection Approach 125
6.4 Input Validation and Sanitization Strategy 132

6.5 Encryption Standards for Data at Rest and in Transit 136
Chapter 7: Deployment Design 140
7.1 Hosting Plan 140
7.2 Hardware Requirements 146
7.3 Network Diagram 149
7.4 Environment Specification 155
7.5 CI/CD Pipeline Overview 162
Chapter 8: References 170

List of Tables

Table 1: Customer Table 18
Table 2: Staff Table 18
Table 3: Admin Table 18
Table 4: Branch Table 19
Table 5: Inventory Table 19
Table 6: Order Table 20
Table 7: Order Item Table 20
Table 8: Payment Table 21
Table 9: Appointment Table 21
Table 10: Chatbot Table 22
Table 11: Knowledge Area Table 22
Table 12: Updated Access Control Matrix Table 119
Table 13: Environment Comparison Table 159

Table of Figures

Figure 1: System Context Diagram 1
Figure 2: High-level Three-Tier Architecture of OSMS 3
Figure 3: Logical Entity Relationship Diagram of OSMS 14
Figure 4(A): Core Domain Model Class Diagram 26
Figure 4(B): Application Service Operations 27
Figure 5: User Visit and Home Page Sequence Diagram 29
Figure 6: Authentication Sequence Diagram 31
Figure 7: Product Discovery Sequence Diagram 34
Figure 8: Cart Management Sequence Diagram 37

Figure 9: Checkout and Order Confirmation Sequence Diagram 40
Figure 10: Appointment Booking Sequence Diagram 44
Figure 11: Chatbot Interaction Sequence Diagram 48
Figure 12: Staff Order Processing and Tracking Sequence Diagram 53
Figure 13: Inventory Management and Reorder Alerts Sequence Diagram 56
Figure 14: Doctor Session Management Sequence Diagram 59
Figure 15: Prescription Recording and History Sequence Diagram 62
Figure 16: Landing Page 67
Figure 17: Sign Up Page 68
Figure 18: Login Page 68
Figure 19: Profile Page 69
Figure 20: Men's Collection Page 70
Figure 21: Women's Collection Page 71
Figure 22: Kids' Collection Page 72
Figure 23: Sunglasses Collection Page 73
Figure 24: Product Information Page 74
Figure 25: Cart Drawer 75
Figure 26: Shipping Information Page 76
Figure 27: Payment Details Page 77
Figure 28: Order Review Page 78
Figure 29: Order Confirmation Page 79
Figure 30: Patient Details Page 80
Figure 31: Doctor Selection Page 81
Figure 32: Appointment Scheduling Page 82
Figure 33: Appointment Review Page 83
Figure 34: Appointment Confirmation Page 84
Figure 35: Chatbot Widget 85
Figure 36: Doctor Sessions Overview 86
Figure 37: Role-Based User Flow 87
Figure 38: Browse and Product Discovery Flow 88
Figure 39: Cart Management Flow 89
Figure 40: Checkout and Order Confirmation Flow 90

Figure 41: User Profile Management Flow 91
Figure 42: Authentication Flow 92
Figure 43: Appointment Booking Flow 93
Figure 44: Chatbot Interaction Flow 95
Figure 45: Staff Order Processing and Tracking Flow 96
Figure 46: Inventory Management and Reorder Alerts Flow 97
Figure 47: Doctor Session and Appointment Administration Flow 98
Figure 48: Clinical Prescription Management Flow 99
Figure 49: Admin Staff and Doctor Management Flow 100
Figure 50: Sales and Inventory Reporting Flow 101
Figure 51: Loyalty Points and History Flow 102
Figure 52: Network Diagram — OSMS Cloud Deployment Architecture 155

Chapter 1: Architectural Design

1.1 System Context Diagram

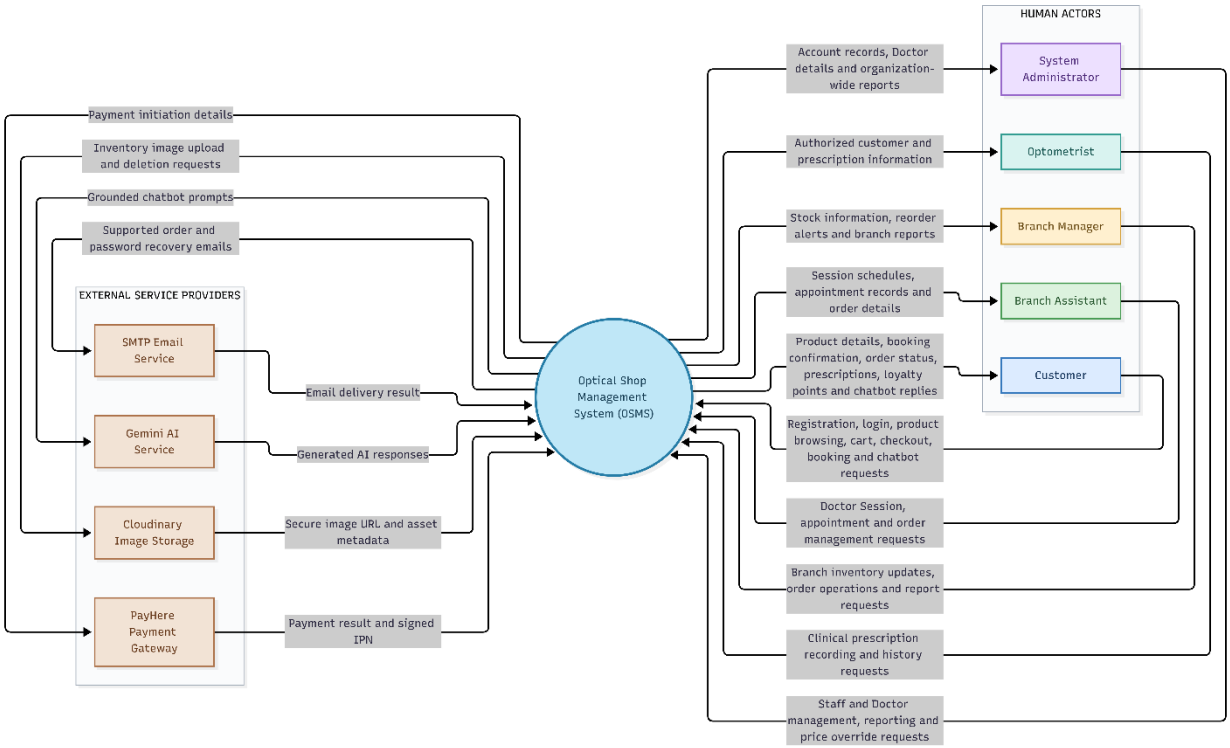


Figure 1: System Context Diagram

The System Context Diagram illustrates the interactions between the Optical Shop Management System (OSMS), its authorized users, and external service providers.

The OSMS serves as the central system supporting retail, clinical, and administrative operations. The diagram identifies five primary authenticated user roles: Customer, Branch Assistant, Branch Manager, Optometrist, and System Administrator.

- **Customer:** Interacts with the system to browse products, manage shopping carts, complete purchases, book eye examinations, view order and prescription histories, access loyalty information, and use the AI chatbot.
- **Branch Assistant:** Manages Doctor Sessions, coordinates customer appointments, and performs authorized order-processing activities.
- **Branch Manager:** Manages branch-specific inventory, monitors reorder alerts, processes authorized orders, and accesses branch-level business reports.
- **Optometrist:** Records clinical prescriptions and accesses authorized customer prescription histories.

- **System Administrator:** Manages staff accounts and Doctor records, accesses organization-wide reports, and performs authorized administrative operations and price overrides.

The system also communicates with the following external services:

- **PayHere:** Processes online payments and provides payment notifications for secure backend verification.
- **Cloudinary:** Provides hosted storage and management of inventory product images.
- **Gemini AI:** Generates chatbot responses using the approved knowledge-based assistance workflow.
- **SMTP Email Service:** Supports password recovery and approved order-related email notifications.

System Boundary: MongoDB Atlas, application-level authentication, authorization, and business logic are internal architectural components and are detailed in the following sections. Automated appointment reminders, third-party courier tracking, and direct medical-device integration are excluded from the current system scope.

1.2 High-level Architecture

The Optical Shop Management System (OSMS) follows a three-tier layered architecture based on the MERN stack (MongoDB, Express.js, React.js, and Node.js).

The architecture separates the user interface, application business logic, and data persistence into distinct layers. This separation supports modular development, maintainability, security, and controlled access to centralized business information.

The system integrates external services for payment processing, inventory image storage, AI chatbot assistance, and supported email notifications.

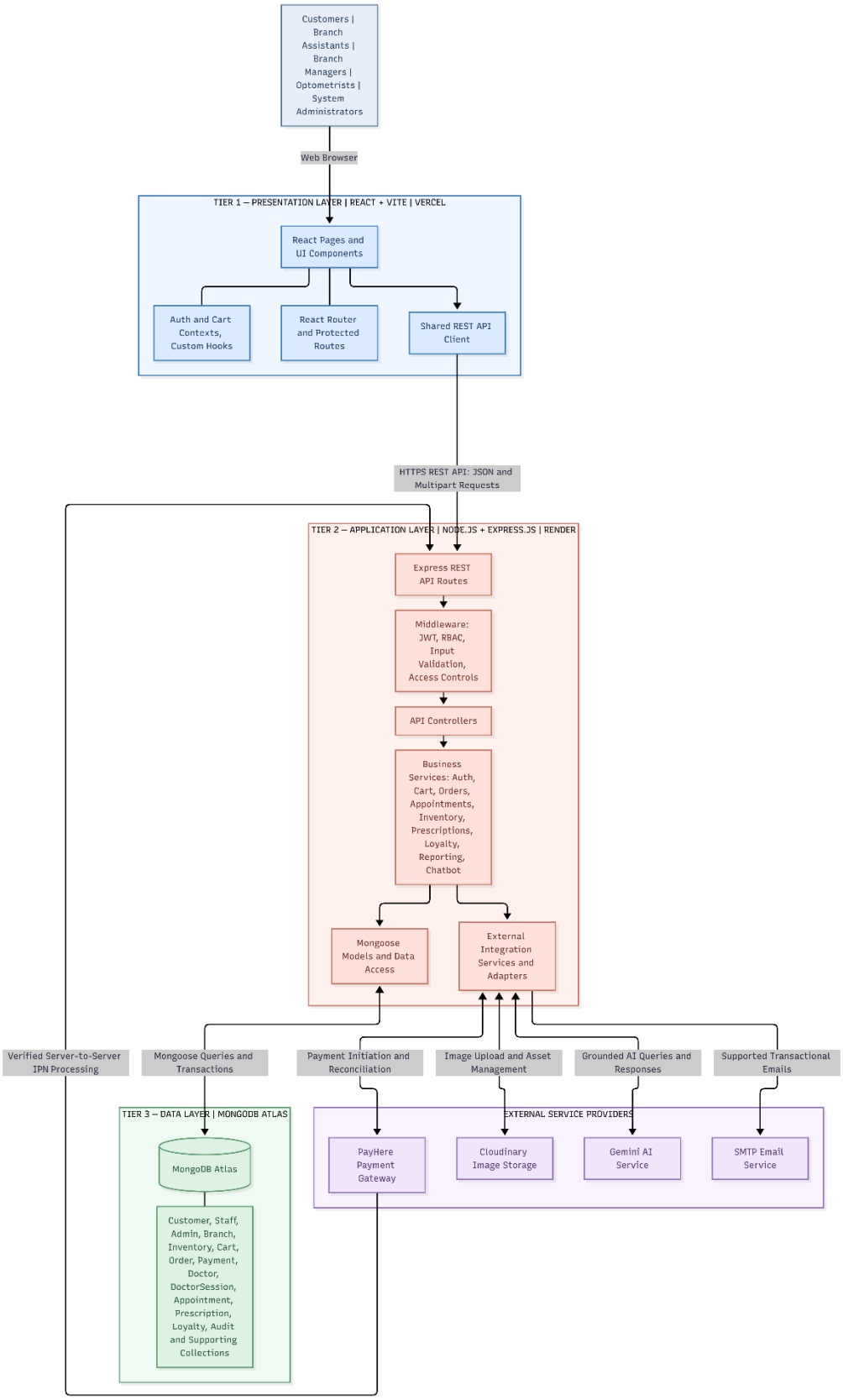


Figure 2: High-level Three-Tier Architecture of the Optical Shop Management System (OSMS)

1.2.1 Detailed Description of Layers

The OSMS architecture consists of the following three layers:

1. Presentation Layer (Frontend – React.js)

- **Role:** The Presentation Layer is responsible for the User Interface (UI) and User Experience (UX). It enables customers and authorized staff to interact with the OSMS through a web browser.
- **Technologies:** The frontend is developed using React.js with Vite and React Router to provide a component-based Single Page Application (SPA).
- **Functionality:**
    - Provides role-specific interfaces for Customers, Branch Assistants, Branch Managers, Optometrists, and System Administrators.
    - Handles client-side routing, form validation, and interface state management through React components, contexts, and custom hooks.
    - Supports product browsing and filtering (FR-003), shopping carts and checkout (FR-004), appointment booking (FR-007), and other authorized system functions.
    - Uses protected routes to control navigation based on authenticated user roles, while the backend remains responsible for enforcing authorization.
    - Communicates with the Application Layer through a shared API client using HTTPS-based RESTful requests.

2. Application Layer (Backend – Node.js & Express.js)

- **Role:** The Application Layer processes client requests, executes business rules, validates user permissions, manages transactions, and coordinates access to the database and external services.
- **Technologies:** The backend is developed using Node.js and Express.js, with Mongoose for database operations.

- **Functionality:**
    - **Authentication and Security:** Implements JWT-based authentication, Bcrypt password hashing, role-based access control (RBAC), request validation, and ownership-based authorization (FR-001).
    - **Business Logic Execution:** Processes customer accounts, shopping carts, checkout, order lifecycles, branch-specific inventory, reorder alerts, Doctor Sessions, appointment bookings, prescriptions, and loyalty point calculations.
    - **Order and Payment Processing:** Validates checkout requests, reserves inventory, creates orders, initiates PayHere payments, and processes verified payment notifications.
    - **Appointment Management:** Validates Doctor Session availability, booking lead time, and patient capacity before creating confirmed appointments.
    - **Reporting and Audit:** Provides authorized sales and inventory reports, report exports, and audit-event recording for relevant business operations.
    - **Integration Services:** Communicates with PayHere for payments, Cloudinary for inventory image management, Gemini AI for chatbot responses, and SMTP-based email services for supported notifications.
    - **Application Structure:** Organizes implementation responsibilities through routes, controllers, services, models, middleware, validation modules, and external provider adapters.

3. Data Layer (Persistence – MongoDB)

- **Role:** The Data Layer is responsible for the centralized storage, retrieval, validation, and management of business and clinical records.
- **Technologies:** The system uses MongoDB Atlas, a cloud-hosted document-oriented database, together with Mongoose schemas and data-access operations.
- **Functionality:**
    - **Document-Based Storage:** Maintains structured collections for customer accounts, staff, administrators, branches, inventory, carts, orders, payments,

appointments, Doctor Sessions, prescriptions, loyalty transactions, and supporting system records.

- **Data Relationships:** Uses MongoDB document references and embedded data structures to represent relationships between customers, orders, inventory items, appointments, Doctors, and clinical prescriptions.
- **Data Consistency:** Supports backend-controlled inventory updates, validation rules, indexes, and MongoDB transactions for operations requiring coordinated data changes.
- **Branch Data Management:** Maintains centralized inventory records associated with individual branches. Automatic inter-branch stock transfers are not included in the current implementation.
- **Security and Access Control:** Restricts database access to authorized backend operations and protects sensitive information through application-level authentication and authorization.
- **Audit and Retention:** Maintains append-only audit-event records for critical operations, with a minimum seven-year retention policy defined under ADR-012. Long-term archival and retention guarantees require independent operational verification.
- **Temporary Data Management:** Supports expiry of temporary chatbot usage records through a MongoDB Time To Live (TTL) index, subject to deployment verification.

1.3 Cloud-Native Architecture

The Optical Shop Management System (OSMS) adopts a cloud-hosted three-tier architecture using managed services to support centralized data access, system availability, security, and maintainability.

The implemented deployment architecture uses Vercel for frontend hosting, Render.com for backend hosting, and MongoDB Atlas for centralized database management. External services, including Cloudinary, PayHere, Gemini AI, and SMTP-based email services, support specific application functions.

The original AWS-based deployment approach was revised through the approved architectural decision process to accommodate project requirements and deployment constraints.

Core Cloud Infrastructure Components

- **Networking & Security (Vercel, Render and Application Security):**
    - **HTTPS Communication:** Vercel and Render provide HTTPS endpoints for secure communication between the frontend and backend.
    - **Application Security:** The Node.js/Express.js backend applies configured Cross-Origin Resource Sharing (CORS) restrictions, security middleware, JWT-based authentication, and role-based access control to protect application resources.
- **Compute Tier (Vercel & Render.com):**
    - **Vercel:** Hosts and delivers the React.js frontend developed using Vite, allowing customers and authorized staff to access the OSMS through web browsers.
    - **Render.com:** Hosts the containerized Node.js/Express.js backend using the existing Docker-based application configuration. The backend processes REST API requests, executes business rules, and manages database and external service interactions.

- **Storage Tier (MongoDB Atlas & Cloudinary):**
    - **MongoDB Atlas:** Provides centralized document-oriented database storage for customer accounts, inventory, orders, payments, appointments, clinical prescriptions, loyalty transactions, and other operational records. Mongoose schemas and MongoDB transactions support data validation and consistency.
    - **Cloudinary:** Provides hosted image storage and management for inventory product images. Authorized image uploads and modifications are handled through the backend according to ADR-008.
- **Content Delivery & Application Hosting (Vercel & Render.com):**
    - **Vercel Content Delivery:** Delivers frontend application assets, including JavaScript, CSS, and images, through its managed hosting infrastructure.
    - **Backend API Hosting:** Render.com serves the Express.js REST API. Frontend requests are directed to the configured backend HTTPS endpoint. Application performance and availability depend on the selected hosting resources and service conditions.
- **Communication & External Services:**
    - **Email Notifications:** Nodemailer integrates with a configured SMTP service to support password recovery and approved order-related email notifications.
    - **Payment Processing:** PayHere provides external online payment processing. The backend validates payment notifications before updating payment and order records.
    - **AI Chatbot:** Gemini AI is accessed through the backend chatbot service to generate responses using information retrieved from the system's internal knowledge records.
    - **Appointment Communication:** Appointment-related communication is handled manually by authorized branch staff. Automated SMS and email appointment reminders are excluded from the current implementation under ADR-009.

Architectural Deviation: The original SDS proposed AWS services, including ECS/Fargate, S3, CloudFront, ALB, and SES/SNS. The implemented hosting approach uses Vercel, Render.com, MongoDB Atlas, and Cloudinary, with relevant changes documented under ADR-008, ADR-015, and the updated DDP-083 deployment decision. The original AWS design remains preserved in SDS v1.0 as the historical architectural baseline.

The selected cloud-hosted architecture supports centralized operations without requiring dedicated application servers at individual shop branches. However, numerical performance, availability, scalability, and security compliance targets remain subject to verification against the approved non-functional requirements.

1.4 Justification for Architecture Choice

1.4.1 Architecture

The selection of a 3-Tier Layered Architecture for the Optical Shop Management System (OSMS) is based on the following technical and operational requirements identified in the SRS:

- **Separation of Concerns (SoC):** By separating the Presentation, Application, and Data layers, the architecture minimizes dependencies between user interface components, business logic, and data persistence. This improves modularity, maintainability, and the ability to update individual application components.
- **Scalability for Growth:** The system is required to support a 50% increase in concurrent users and transactions without performance degradation (NFR-005). The layered architecture allows the frontend, backend, and database resources to be managed independently according to application demand. Actual scaling capabilities depend on the configured cloud hosting services and available resources.
- **Centralized Data Consistency:** The centralized Application and Data layers support consistent management of inventory records, orders, and appointments. Backend validation and database transactions help prevent conflicting updates during concurrent operations. Branch-specific inventory is maintained centrally, while automatic inter-branch stock transfers are not part of the current implementation (FR-005).

- **Enhanced Security and Data Protection:** The layered approach separates sensitive business and clinical information from the frontend interface. The Application Layer enforces JWT-based authentication, role-based access control, input validation, and ownership restrictions for protected operations. HTTPS communication protects data in transit. These controls support the system's security requirements, while full PDPA compliance remains subject to independent verification.
- **Optimized Development and Maintenance:** The MERN stack enables JavaScript-based development across the frontend and backend. The modular implementation of routes, controllers, services, middleware, and Mongoose models improves maintainability and supports the implementation of business functions such as loyalty point calculations (FR-010) and inventory reorder alerts (FR-006).
- **Improved Fault Isolation and Reliability:** Separating application responsibilities helps isolate failures and simplifies troubleshooting and recovery. The architecture supports the system's original 99.9% availability target (NFR-002). However, actual uptime and fault tolerance depend on the deployed hosting infrastructure, external service availability, and operational monitoring.

1.4.2 Cloud-Native Hosting

The OSMS uses managed cloud services, including Vercel, Render.com, and MongoDB Atlas, to support centralized deployment and system operation.

The original AWS-based hosting architecture was revised through the approved architectural decision process, including ADR-015 and the subsequent deployment decisions.

1. **Scalability:** Vercel delivers the React frontend independently of the Render-hosted Node.js/Express.js backend. MongoDB Atlas provides managed database infrastructure. This separation allows application resources to be adjusted according to requirements, subject to provider capabilities, subscription plans, and deployment configuration. The original 50% scalability target remains subject to load testing and operational verification.

2. **Security and Data Protection:** The cloud-hosted architecture uses HTTPS communication, backend JWT authentication, role-based authorization, configured CORS restrictions, and environment-based credential management. PayHere handles external payment processing without requiring OSMS to store customer payment card details. These measures support security and privacy requirements, but do not independently establish PCI DSS or PDPA compliance.

3. **Reliability:** Managed hosting services reduce the need to maintain physical application servers at individual branches. The architecture supports centralized monitoring, deployment management, and recovery procedures. However, service availability depends on the selected Vercel, Render.com, and MongoDB Atlas configurations. Automatic failover and the original 99.9% uptime target cannot be guaranteed without appropriate infrastructure configuration and operational verification.

1.5 Scalability & Maintainability Considerations

To support the performance and maintainability requirements defined in the SRS, the OSMS architecture incorporates strategies for modular development, efficient data processing, cloud-based hosting, and controlled software delivery. These strategies were established within the original six-month development plan and refined according to the implemented architecture.

1.5.1 Scalability

The system is required to support a projected 50% increase in concurrent users and transactions without performance degradation (NFR-005). The following architectural considerations support this requirement:

- **Horizontal Scaling (Stateless Backend):** The Node.js/Express.js backend uses JWT-based authentication and RESTful APIs, reducing dependence on server-side user sessions. The application is hosted on Render.com using the existing Docker-based configuration. The architecture supports potential resource scaling, but automatic horizontal scaling and load balancing have not been independently verified in the current deployment.

- **Database Scalability:** MongoDB Atlas provides centralized database services with indexing and transaction support. The backend uses Mongoose schemas, database queries, and transactions to maintain consistency during operations such as checkout and appointment booking. Database performance depends on the provisioned resources, indexes, and workload characteristics. Sharding and automatic multi-cluster scaling are not confirmed features of the current deployment.

- **Asynchronous Processing:** The backend uses asynchronous operations for database access and external service communication. Order-status notifications are handled through an in-process event mechanism with email delivery processing and bounded retry behavior, reducing direct coupling between order-status changes and notification failures. Automated appointment reminders are excluded from the implementation under ADR-009.

- **Content Delivery Network (CDN):** Vercel provides managed delivery of the React frontend's static assets, including JavaScript and CSS files. Cloudinary provides hosted delivery of inventory product images. These services help reduce the need for application-managed static file delivery, although actual response times depend on network conditions and service configuration.

1.5.2 Maintainability

The system is designed to support the original maintainability requirement that 90% of minor bug fixes should be implemented and deployed within 24 hours (NFR-006). The following practices support this objective:

- **Modular Architecture:** The application is organized into functional modules, including Authentication, Inventory, Orders, Appointments, Prescriptions, Loyalty, and Reporting. The backend separates routes, controllers, services, models, middleware, and validation responsibilities. This structure supports targeted updates, testing, and maintenance while reducing unnecessary dependencies.

- **Unified Technology Stack:** The MERN stack uses JavaScript-based technologies across the frontend and backend. React.js, Node.js, Express.js, and MongoDB with Mongoose provide a consistent development environment, reducing context switching and supporting code maintenance.

- **Automated Documentation & Clean Code:** The project maintains technical documentation, coding standards, GitHub issues, pull requests, and accepted Architectural Decision Records (ADRs). API documentation and automated tests support the understanding and maintenance of existing functionality. These practices also help reduce knowledge-sharing risks identified under RSK-003.

- **Centralized Logging and Monitoring:** The backend provides application-level logging, error handling, and an API health-check endpoint to support operational troubleshooting. The hosting platforms provide operational service information. However, comprehensive production monitoring, automated alerting, and long-term availability tracking have not been independently verified.

- **CI/CD Pipeline:** GitHub Actions provides automated code-quality checks and testing for the frontend and backend. The frontend pipeline includes linting, formatting, test coverage, and build validation, while the backend pipeline includes linting, formatting, and automated tests. Vercel supports frontend deployment from the approved develop branch, and the backend is hosted on Render.com. Deployment readiness, required quality gates, rollback procedures, and production verification are managed through the applicable deployment processes. Passing CI checks alone does not guarantee a successful or defect-free production deployment.

Chapter 2: Database Design

2.1 ER Diagram

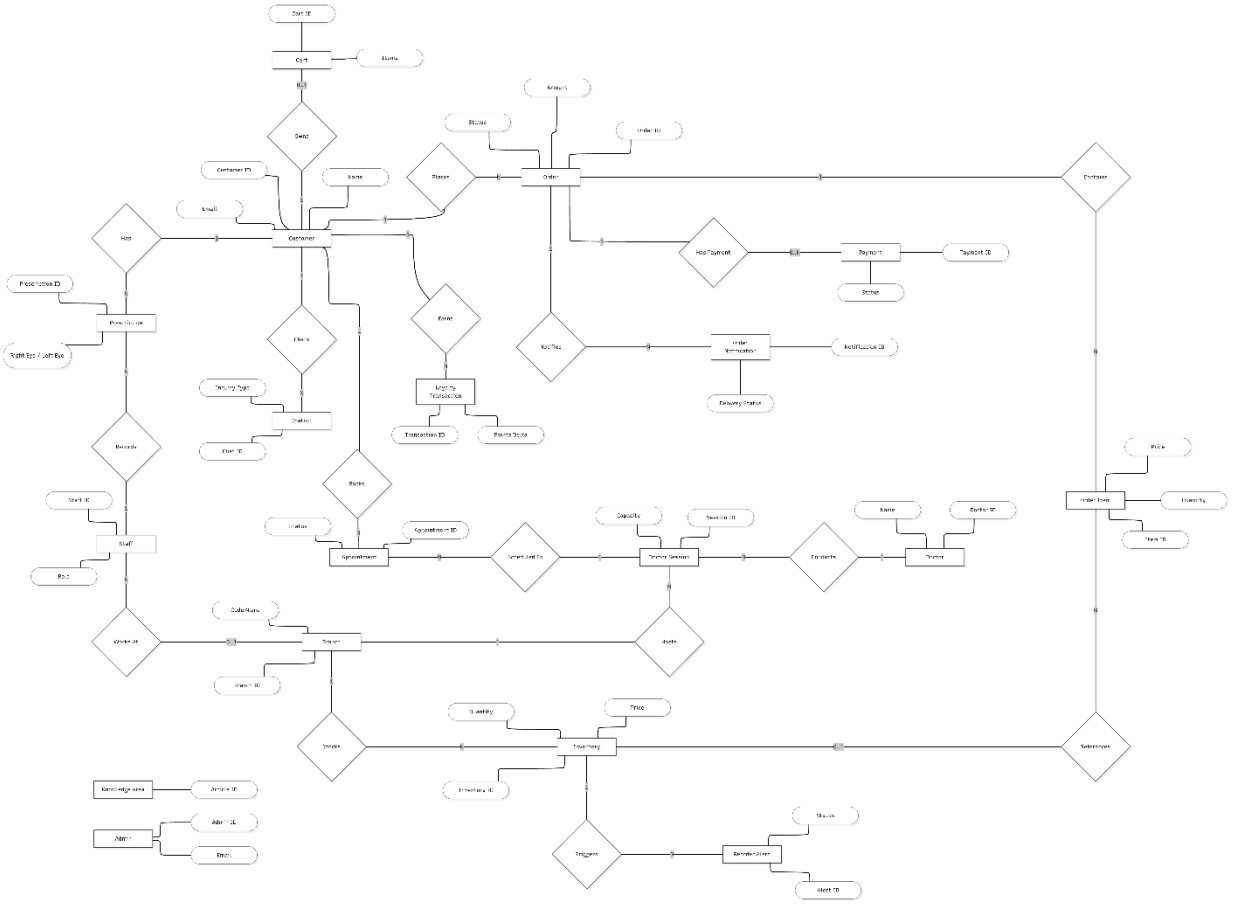


Figure 3: Logical Entity Relationship Diagram of the Optical Shop Management System (OSMS)

The logical Entity Relationship Diagram illustrates the structure and relationships of the core MongoDB collections used by the Optical Shop Management System (OSMS).

A centralized database architecture is used to maintain data consistency and support authorized business operations across branches. Mongoose schemas and MongoDB document references define the relationships between the main system entities.

- **Branch and Staff Relationship:** A One-to-Many (1) relationship exists between Branch and Staff records. Staff members may be assigned to a branch according to their operational responsibilities and authorized roles.

- **Customer and Order Relationship:** A One-to-Many (1) relationship exists between Customer and Order. Each customer can place multiple orders, while each order belongs to one customer. The Order and OrderItem entities maintain purchased items, quantities, prices, and relevant purchase-time snapshots.
- **Shopping Cart Relationship:** Each customer can have one active persisted Cart. Cart items are embedded within the Cart document and contain references to Inventory records.
- **Appointment and Doctor Session Relationship:** Customer appointments are associated with Doctor records and capacity-based Doctor Sessions. Doctor Sessions are linked to branches, and each session can accommodate multiple appointments according to its configured capacity.
- **Prescription Relationship:** The dedicated Prescription collection stores clinical prescription information associated with customers. Each prescription identifies the authorized Optometrist staff member responsible for recording it.
- **Inventory Management Relationship:** Inventory records are associated with individual branches. Branch Managers manage authorized stock information, while System Administrators have restricted price-override access. ReorderAlert records are linked to Inventory and Branch records for low-stock monitoring.
- **Payment and Loyalty Relationships:** Each Order may have multiple Payment attempts. LoyaltyTransaction records maintain eligible loyalty point earnings and approved reversals associated with customer transactions.
- **Chatbot and Knowledge Management:** Chatbot interaction metadata and customer usage information are stored in dedicated collections. The KnowledgeArea collection maintains the internal content used by the AI chatbot's knowledge retrieval process.
- **Audit and Notification Records:** AuditEvent records preserve relevant critical business actions, while OrderNotification records track supported customer notification activities.

Additional supporting collections maintain payment verification, payment retrieval, refund evidence, image-upload lifecycle information, and inventory configuration.

The diagram represents logical document relationships rather than relational database foreign-key constraints. Detailed field definitions, validation rules, indexes, and collection-specific structures are presented in Section 2.2.

2.2 Normalization Explanation

The database design of the Optical Shop Management System (OSMS) follows a document-oriented approach using MongoDB Atlas and Mongoose.

Relational database normalization principles were considered during logical data modelling to minimize unnecessary redundancy, maintain data consistency, and prevent update anomalies. However, the implemented system does not use a fully relational Third Normal Form (3NF) schema.

Instead, the system combines separate MongoDB collections, document references, embedded subdocuments, validation rules, indexes, and transactions according to the requirements of each business process.

First Normal Form (1NF)

- **Atomic Data:** Individual fields are used for information such as customer names, email addresses, product prices, and stock quantities where appropriate.
- **Unique Identification:** MongoDB assigns an _id identifier to documents. Additional unique indexes are used where required, including customer email addresses and branch codes.
- **Embedded Data:** Selected repeating structures, such as Cart items, Order status histories, and prescription measurements, are intentionally stored as embedded subdocuments or arrays.
- **Design Consideration:** These embedded structures support MongoDB document-oriented storage and are not claimed to satisfy strict relational 1NF requirements.

Second Normal Form (2NF)

- **Entity Separation:** Major business entities, including Customer, Inventory, Order, OrderItem, and Payment, are maintained in separate collections based on their responsibilities.
- **Data Relationships:** Mongoose ObjectId references associate related records without requiring the duplication of entire documents.

- **Transaction Separation:** Order records maintain purchasing and fulfillment information, while Payment records maintain payment attempts and provider-specific transaction details.

- **Design Consideration:** Because most collections use a single MongoDB _id identifier rather than a composite relational primary key, traditional partial-key dependency analysis is not directly applicable.

Third Normal Form (3NF)

- **Reduced Redundancy:** Related business information is separated into collections to reduce unnecessary duplication and support consistent updates.

- **Branch Separation:** Branch information is maintained in the Branch collection, while Staff and Inventory records reference the relevant branch rather than duplicating its details.

- **Clinical Data Separation:** Prescription records are maintained independently of Customer accounts, supporting historical clinical records and restricted access.

- **Appointment Separation:** Doctor and DoctorSession records are maintained separately from Appointment records to support capacity-based scheduling.

- **Controlled Denormalization:** Selected data, such as purchase-time OrderItem prices, classification snapshots, and Order status histories, is deliberately preserved within documents to support financial accuracy and historical consistency.

The implemented database design therefore applies normalization principles where appropriate while using MongoDB-specific data modelling techniques.

Implemented MongoDB Collection Schemas

The following tables summarize the original core database entities, updated to reflect the implemented Mongoose schemas.

The tables represent logical collection definitions and do not imply that the system uses relational SQL tables. MongoDB _id values serve as the primary document identifiers.

User & Customer Entities

Customer

Table 1: Customer Table

_id	name	email	address	phone	role	loyaltyBalance
ObjectId	String	String	String	String	String	Number

The Customer schema also maintains password hashes, login-attempt information, account-locking data, and password-reset token information. Password hashes and reset-token hashes are not exposed through normal customer API responses.

Staff

Table 2: Staff Table

_id	branchId	name	email	phone	address	role	isActive
ObjectId	ObjectId	String	String	String	String	String	Boolean

The branchId field references the Branch collection and may be optional according to the staff account configuration. Staff roles are restricted to the approved canonical staff roles. Authentication and password-recovery information is stored separately within the document.

Admin

Table 3: Admin Table

_id	name	email	phone	role
ObjectId	String	String	String	String

The Admin schema maintains System Administrator identity and authentication information. Password hashes and account-locking fields are stored securely and excluded from standard API responses.

Inventory & Branch Entities

Branch

Table 4: Branch Table

_id	codeName	address	contactNumber
ObjectId	String	String	String

The Branch collection maintains branch information. The codeName field has a unique constraint to distinguish branches.

Inventory

Table 5: Inventory Table

_id	branchId	itemName	category	brand	price	quantity
ObjectId	ObjectId	String	String	String	Number	Number

The Inventory schema additionally stores:

- itemClassification — product eligibility classification used by checkout and loyalty calculations.
- imageUrl — hosted product image URL.
- imagePublicId — Cloudinary asset identifier.

Inventory documents reference the Branch collection. Stock quantities are managed according to authorized branch operations. No dedicated global SKU field is defined in the implemented Inventory schema.

Transactional Entities

Order

Table 6: Order Table

_id	customerId	branchId	orderDate	orderAmount	status	orderAddress
ObjectId	ObjectId	ObjectId	Date	Number	String	String

The Order schema additionally stores:

- statusHistory — embedded history of order-status changes.
- statusUpdatedAt — timestamp of the most recent status update.
- reservationExpiresAt — expiry time for an unpaid stock reservation.
- inventoryRestoredAt — timestamp used to track stock restoration.
- claimedAt and claimedBy — fulfillment claim information.

The branchId field may initially be null for online orders and is assigned according to the approved fulfillment workflow. Order statuses follow the lifecycle defined in ADR-004.

Order Item

Table 7: Order Item Table

_id	orderId	inventoryId	itemName	price	quantity
ObjectId	ObjectId	ObjectId	String	Number	Number

The OrderItem schema additionally maintains:

- fulfilledInventoryId — reference to the inventory record associated with fulfillment.
- classificationSnapshot — embedded purchase-time classification information, including source inventory reference, category, item classification, and loyalty eligibility.

OrderItem financial information and approved purchase-time snapshots are protected from unauthorized modification after creation.

Payment

Table 8: Payment Table

_id	orderId	amount	status	method	provider	providerTransactionId
ObjectId	ObjectId	Number	String	String	String	String

The Payment schema additionally maintains provider order references, currency, payment and refund timestamps, reservation expiry, payment failure information, and reconciliation status.

Each Payment document references an Order. Multiple payment attempts may exist for the same Order, subject to the implemented uniqueness and payment-lifecycle constraints.

Service & Support Entities

Appointment

Table 9: Appointment Table

_id	customerId	doctorId	sessionId	branchId	dateTime	status	position
ObjectId	ObjectId	ObjectId	ObjectId	ObjectId	Date	String	Number

The Appointment collection associates customer bookings with Doctor records, DoctorSession records, and the relevant branch.

A legacy staffId reference is retained for historical compatibility. New appointment scheduling uses the approved Doctor and DoctorSession architecture rather than treating authenticated Optometrist staff accounts as appointment providers.

Chatbot

Table 10: Chatbot Table

_id	chatId	customerId	inquiryType	responseStatus
ObjectId	String	ObjectId	String	String

The Chatbot collection maintains interaction metadata for authenticated customer chatbot requests. It is not used as an unrestricted storage location for complete AI conversation histories.

Knowledge Area

Table 11: Knowledge Area Table

_id	articleId	category	contentTitle	contentBody	keywords	isActive
ObjectId	String	String	String	String	String Array	Boolean

The KnowledgeArea collection stores approved internal knowledge articles used by the chatbot's retrieval process. A text index supports keyword-based content retrieval.

Additional Implemented Collections

The original SDS defined the eleven core entities above. Approved architectural decisions and subsequent implementation introduced additional MongoDB collections.

The following collections extend the original schema without replacing the approved historical baseline.

Collection	Primary Fields	Purpose / Relationship
Cart	customerId, items	Stores one persisted cart per customer. Each embedded item contains inventoryId and quantity.
Doctor	name, specialization, isActive	Maintains the Doctor directory independently of authenticated staff accounts.
DoctorSession	doctorId, staffId, branchId, startAt, endAt, capacity, confirmedCount	Maintains capacity-based examination sessions. The staffId field supports historical compatibility.
Prescription	customerId, recordedBy, recordedAt, rightEye, leftEye, remarks, isArchived	Stores clinical optical measurements with customer ownership and recording Optometrist references.
LoyaltyTransaction	customerId, orderId, paymentId, eventType, pointsDelta, balanceAfter	Maintains traceable point earnings and approved reversals.
InventoryConfig	key, value, updatedBy	Maintains configurable inventory settings, including the organization-wide reorder threshold.
ReorderAlert	inventoryId, branchId, currentQuantity, threshold, status	Tracks low-stock alerts linked to branch inventory records.
ChatbotUsage	_id, windowStart, expiresAt, messageCount	Tracks the authenticated customer's chatbot usage window. A TTL index supports expiry.

ImageUpload	publicId, imageUrl, uploadedBy, branchId, status, inventoryId	Tracks Cloudinary inventory image upload and cleanup operations.
OrderNotification	orderId, customerId, recipientEmail, triggerStatus, deliveryStatus, attempts	Maintains transactional order-email notification outcomes.
PaymentEvent	eventKey, paymentId, orderId, provider, statusCode, verifiedAt	Records payment provider events and verified processing outcomes.
PaymentRetrievalEvidence	paymentId, orderId, observationKey, outcome, observedAt, recordedBy	Maintains provider payment-retrieval and reconciliation evidence.
ManualRefundEvidence	paymentId, orderId, amount, refundChannel, refundReference, recordedBy	Maintains approved evidence of externally handled refunds without implying automatic refund execution.
AuditEvent	eventId, action, occurredAt, actorId, actorRole, targetType, targetId, integrity	Maintains append-only audit records for authorized critical operations.

Data Integrity and Schema Constraints

The implemented MongoDB architecture applies several mechanisms to maintain consistency and preserve historical information:

- **Schema Validation:** Mongoose schemas define required fields, permitted values, data types, and validation rules.
- **Document References:** ObjectId references associate related collections, including Customers, Orders, Payments, Branches, DoctorSessions, and Prescriptions.
- **Indexes:** Unique, compound, and conditional indexes are used for relevant identity, lookup, scheduling, payment, and operational constraints.

- **Database Transactions:** MongoDB transactions coordinate applicable multi-document operations, including checkout-related inventory changes and appointment capacity updates.

- **Historical Data Preservation:** OrderItem snapshots, Order status histories, PaymentEvent records, and AuditEvent records preserve relevant historical business information.

- **Temporary Data Expiration:** ChatbotUsage records use a TTL index for session usage-window expiration. Production index provisioning must be verified.

- **Audit Retention:** ADR-012 defines a minimum seven-year retention policy for critical audit records. Long-term operational archival and retention guarantees remain subject to verification.

These mechanisms provide an implementation-aligned database structure while preserving the original requirements and approved architectural decisions.

Chapter 3: Class Diagram

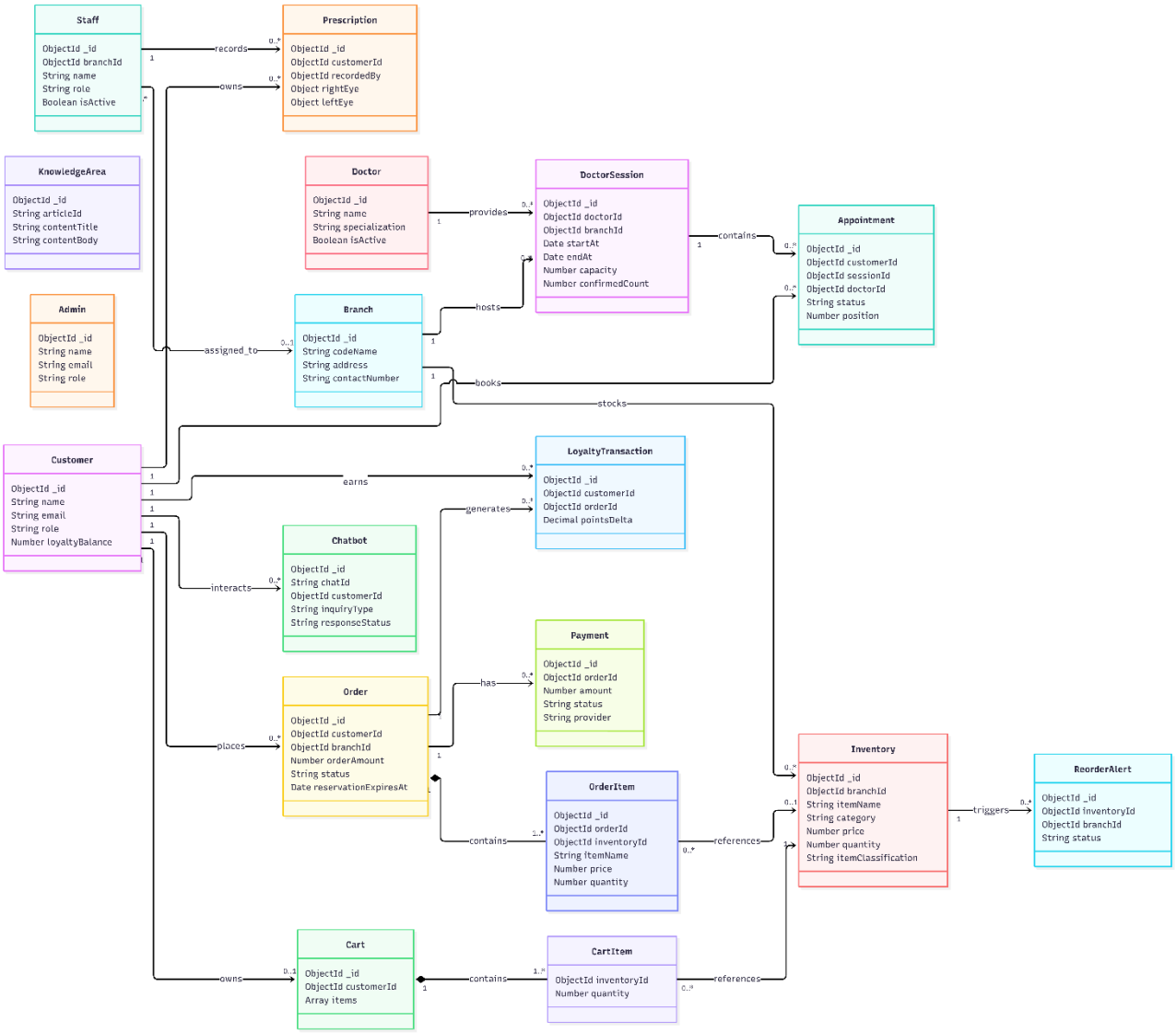


Figure 4(A): Core Domain Model Class Diagram

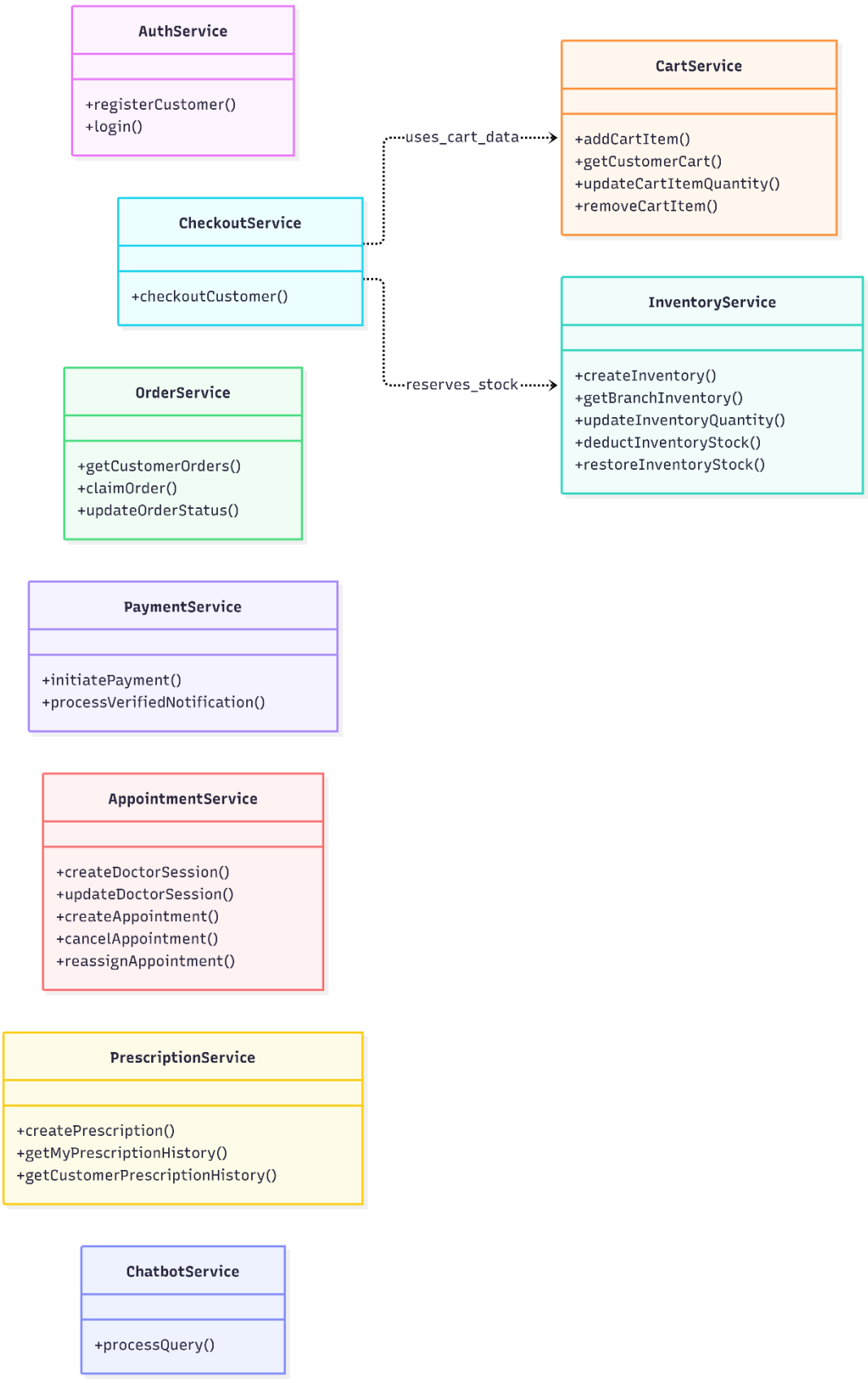


Figure 4(B): Application Service Operations

Figure 4: Class Diagram of the Optical Shop Management System (OSMS)

The Class Diagram presents the main domain models, their attributes, associations, and the application services responsible for business operations within the Optical Shop Management System (OSMS).

The implemented backend uses Mongoose schemas for domain models and separates business operations into service modules. The diagram therefore provides a logical UML representation of the implementation rather than implying that every MongoDB model is a conventional object-oriented class containing business methods.

Attribute and Method Visibility: Domain model attributes represent fields defined in the implemented Mongoose schemas, including document identifiers, business data, status fields, and related entity references. Business operations such as customer authentication, appointment creation, inventory updates, order processing, and payment verification are implemented primarily in application service modules. Public operation notation (+) is used to identify the service functions exposed to other backend modules; it does not imply that MongoDB document fields are JavaScript private properties.

Association and Multiplicity: Relationships between domain models are represented through their implemented Mongoose references and embedded document structures. For example, a Customer can place multiple Orders, an Order contains multiple OrderItems, and an Order can have multiple Payment attempts. DoctorSessions support multiple customer Appointments, subject to configured session capacity.

Inheritance and Roles: Customer, Staff, and Admin are maintained as separate MongoDB models rather than subclasses of a shared User model. Staff roles are controlled through the approved Role-Based Access Control (RBAC) architecture, including Branch Assistant, Branch Manager, and Optometrist. System Administrator permissions are maintained separately. Doctor records represent clinical service providers and are not authenticated staff accounts.

System Automation: Business operations are executed through dedicated backend services. The appointment service manages DoctorSessions, validates booking constraints, and creates confirmed appointments. The inventory service performs authorized stock updates, while checkout and payment services coordinate order creation, stock reservations, and verified PayHere payment processing. The chatbot service processes customer inquiries using internal knowledge retrieval and Gemini AI. Automated appointment reminders and human-agent chatbot escalation are excluded from the current implementation.

The class diagrams focus on the principal application models and services. Additional supporting schemas for auditing, notifications, payment reconciliation, image uploads, and system configuration are documented in Chapter 2 and the relevant architectural decision records.

Chapter 4: Sequence Diagrams

4.1 User Visit and Homepage

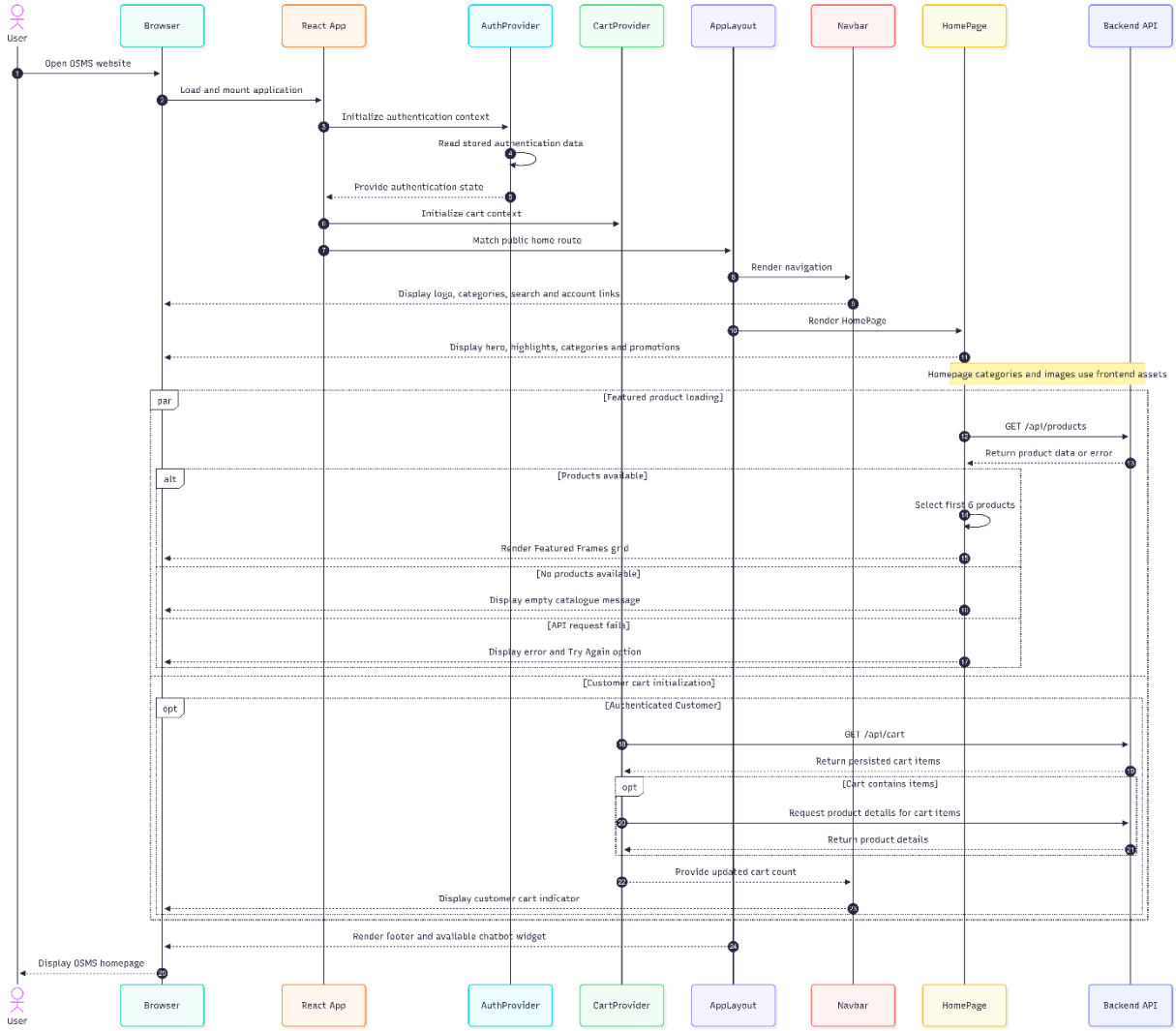


Figure 5: User Visit and Home Page Sequence Diagram

The sequence diagram illustrates the process of accessing and displaying the OSMS homepage.

When a user opens the website, the browser loads the React application. The application initializes the routing, authentication, and cart contexts before displaying the appropriate page.

The AppLayout renders the navigation bar and homepage components. The homepage displays the hero section, product category cards, shopping highlights, promotional content, and footer.

The HomePage component retrieves product information through the backend REST API (GET /api/products) and displays up to six products in the Featured Frames section. Product categories and homepage images are provided through frontend configuration and assets.

For authenticated Customers, the CartProvider retrieves the persisted shopping cart and updates the cart information displayed in the navigation bar.

The homepage includes appropriate loading, empty-result, and error states to handle product availability and API failures.

Public visitors can access the homepage and browse products without authentication. Protected features, including shopping cart management and checkout, require an authenticated Customer account.

4.2 Authentication (Register and Login)

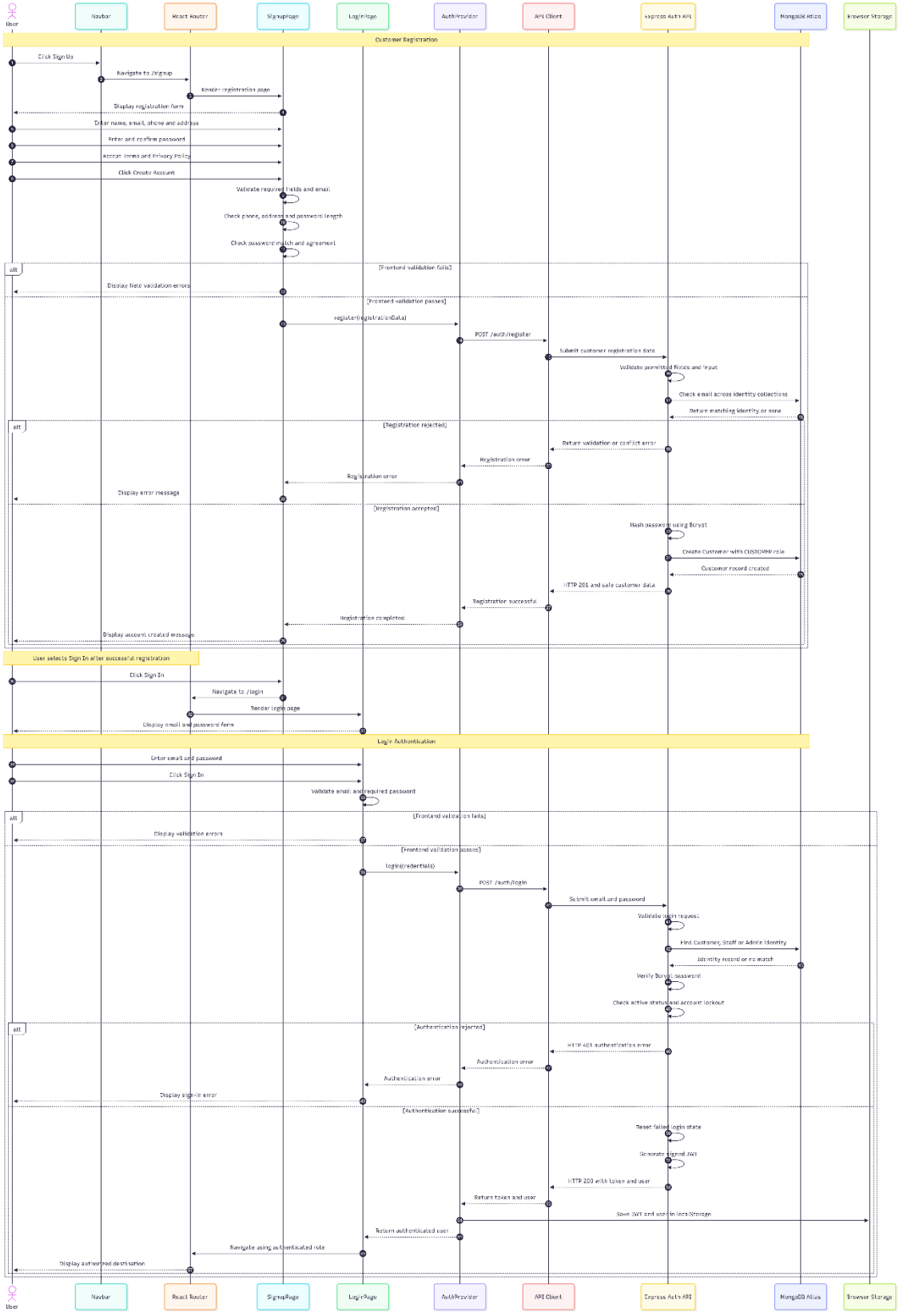


Figure 6: Authentication Sequence Diagram

The Authentication Sequence Diagram illustrates the customer registration and user login processes implemented in the Optical Shop Management System (OSMS).

Customer Registration:

1. A new customer accesses the registration page through the application's navigation.
2. The system displays a registration form requesting the customer's first name, last name, email address, phone number, address, password, and password confirmation.
3. The frontend validates the required fields, email and phone formats, minimum password length, password confirmation, and agreement to the Terms of Service and Privacy Policy.
4. If validation succeeds, the AuthProvider submits the registration information to the backend through the POST /api/auth/register endpoint.
5. The backend validates the submitted data and checks whether the email address already exists in the Customer, Staff, or Admin collections.
6. If the registration is valid, the backend hashes the password using Bcrypt and creates a new Customer record in MongoDB with the CUSTOMER role.
7. The backend returns a successful registration response. The frontend displays an account-creation confirmation, allowing the customer to proceed to the login page.
8. If validation fails or the email is already registered, the system displays an appropriate error message.

User Login:

1. The user opens the login page and enters an email address and password.
2. The frontend validates the required login information.
3. The AuthProvider submits the credentials to the backend through the POST /api/auth/login endpoint.
4. The backend searches the Customer, Staff, and Admin collections for the matching identity.
5. The authentication service verifies the password using Bcrypt and checks account status and applicable account-lockout conditions.
6. If authentication succeeds, the backend generates a signed JSON Web Token (JWT) containing the authorized identity and role information.

7. The frontend stores the authentication token and user information in browser localStorage and updates the authentication context.
8. The user is redirected to the appropriate application page according to the authenticated role.
9. If authentication fails, the system displays an error message without granting access to protected functionality.

Security Considerations:

- Customer registration creates accounts with the CUSTOMER role only.
- Staff and System Administrator accounts are managed through authorized administrative processes.
- Passwords are stored as Bcrypt hashes rather than plaintext values.
- JWT authentication is used for protected backend API requests.
- Role-based access control is enforced by the backend, while frontend protected routes manage authorized navigation.
- Repeated unsuccessful login attempts are subject to account-lockout controls.
- The current frontend stores JWT credentials in localStorage rather than HTTP-only authentication cookies.

4.3 Product Discovery (Search, Filter, Detail)

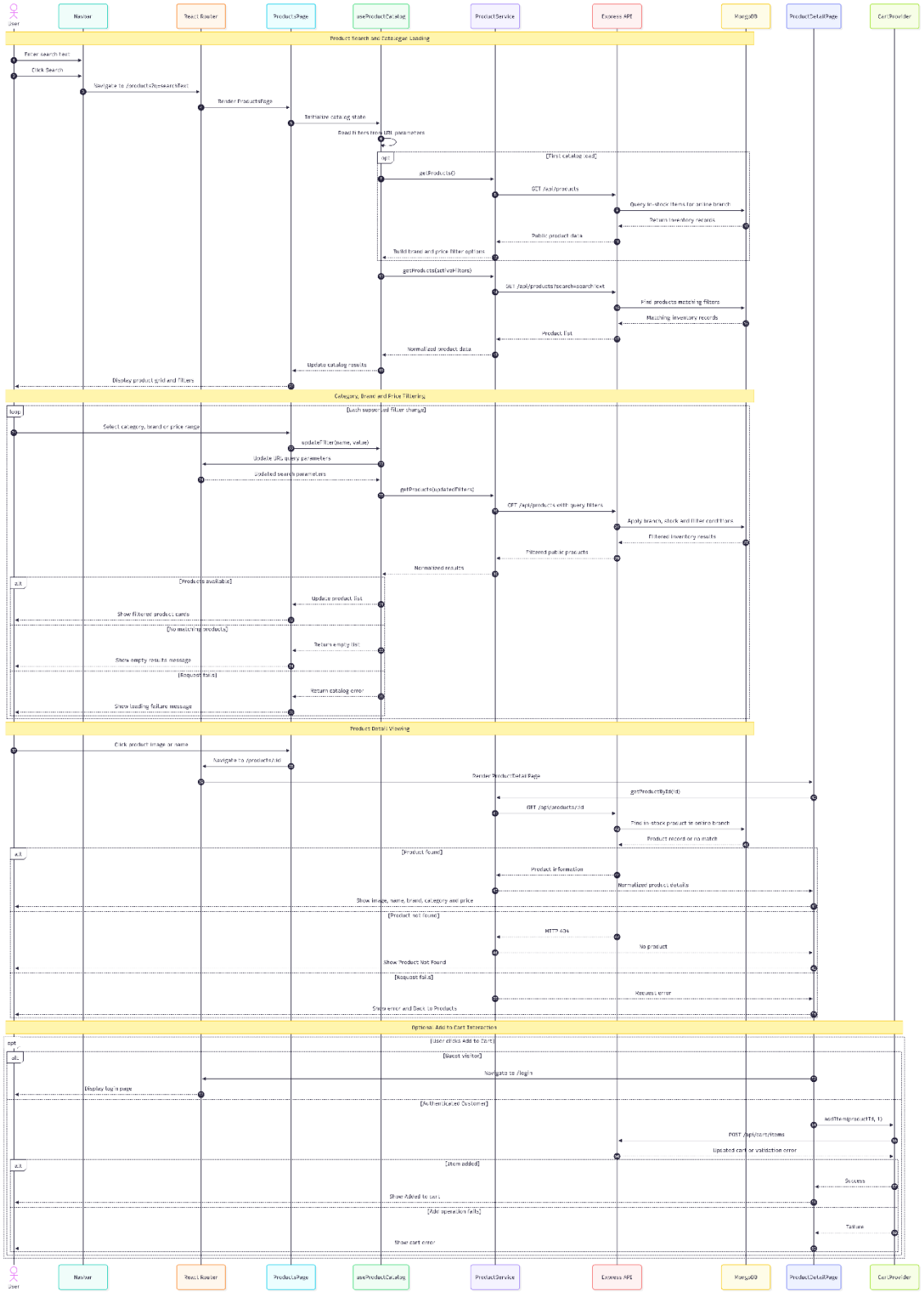


Figure 7: Product Discovery Sequence Diagram

The Product Discovery Sequence Diagram illustrates the process of searching, filtering, and viewing eyewear products within the Optical Shop Management System (OSMS).

Product Search:

1. The user enters a search term through the navigation bar and submits the search request.
2. React Router navigates to the product catalogue page with the search term included in the URL query parameters.
3. The ProductsPage component uses the useProductCatalog hook to manage catalogue state, filters, and loading status.
4. The frontend requests product information from the backend through the GET /api/products endpoint.
5. The backend retrieves available inventory items associated with the configured online fulfillment branch.
6. Matching product records are returned to the frontend and displayed as product cards.

Product Filtering:

1. The user can filter products by category, brand, minimum price, and maximum price.
2. The frontend updates the relevant URL query parameters.
3. The useProductCatalog hook requests updated results from the backend.
4. The backend applies the supported search and filtering conditions to the available inventory records.
5. The frontend updates the displayed product list based on the returned results.
6. If no products match the selected filters, the system displays an empty-result message. An appropriate error message is displayed if the request fails.

Product Detail Viewing:

1. The user selects a product from the catalogue.
2. React Router navigates to the ProductDetailPage using the selected product identifier.

3. The frontend requests the selected product through the GET /api/products/:id endpoint.
4. The backend verifies that the product exists, belongs to the configured online fulfillment branch, and is available in stock.
5. The frontend displays the product image, name, brand, category, and price.
6. If the product cannot be found or retrieved, the system displays an appropriate message.

Shopping Cart Interaction:

The product catalogue and product details are publicly accessible. However, adding items to the shopping cart requires an authenticated Customer account.

If an unauthenticated visitor selects Add to Cart, the application redirects the visitor to the login page. For authenticated Customers, the CartProvider submits the add-item operation to the backend and updates the shopping cart based on the response.

Implementation Constraints:

The current public product API supports searching by product name or brand and filtering by category, brand, and price range. Product sorting controls, colour filtering, frame-type filtering, wishlists, product reviews, and lens-type selection are not implemented in the current product discovery workflow.

Product information is retrieved from centralized inventory records rather than a static frontend product dataset.

4.4 Cart Management

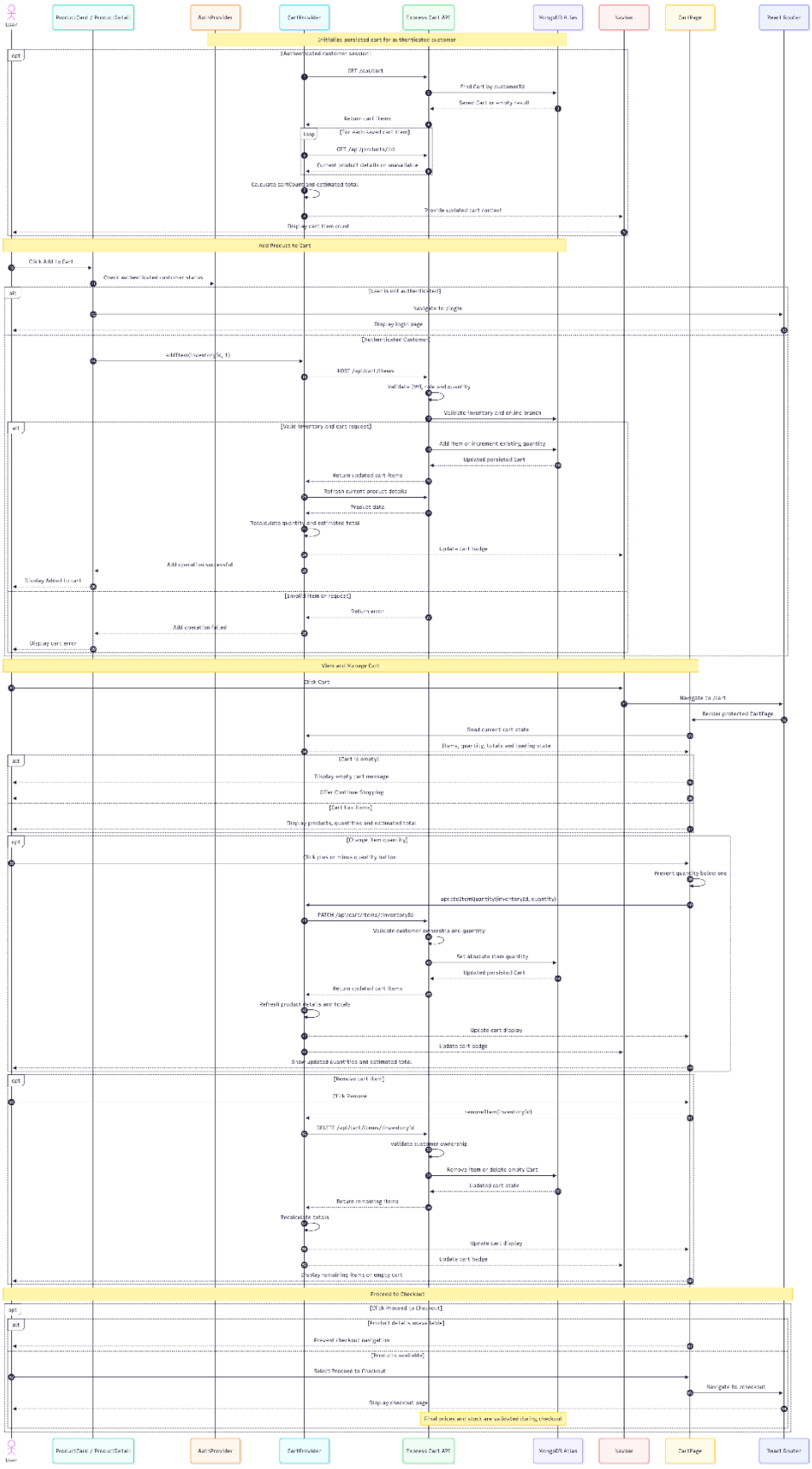


Figure 8: Cart Management Sequence Diagram

The Cart Management Sequence Diagram illustrates the process of adding products to a shopping cart, viewing saved items, modifying quantities, removing items, and proceeding to checkout within the Optical Shop Management System (OSMS).

Adding Products to Cart:

1. The Customer selects Add to Cart from a product card or product details page.
2. The frontend verifies whether the user is authenticated with the CUSTOMER role.
3. If the visitor is unauthenticated, the application redirects the visitor to the login page.
4. For an authenticated Customer, the CartProvider submits the selected inventory identifier and quantity through the POST /api/cart/items endpoint.
5. The backend validates the customer's authorization, inventory identifier, requested quantity, and applicable online fulfillment branch.
6. If the product already exists in the customer's Cart, the backend increases its quantity. Otherwise, it creates a new cart item.
7. The updated Cart is stored in MongoDB and returned to the frontend.
8. The CartProvider refreshes the relevant product information, updates its state, and recalculates the cart item count and estimated total.
9. The navigation bar displays the updated cart count, and the Customer receives a success or error message.

Viewing the Shopping Cart:

1. The Customer selects the Cart option from the navigation bar.
2. React Router displays the protected CartPage component.
3. The CartPage retrieves the current cart state from the CartProvider.
4. For authenticated Customers, the CartProvider loads the persisted cart through the GET /api/cart endpoint when the customer session is initialized.
5. The CartPage displays the available product details, item quantities, line subtotals, and estimated total.
6. If the Cart is empty, the system displays an empty-cart message and allows the Customer to continue shopping.

Updating Item Quantities:

1. The Customer increases or decreases an item's quantity using the available controls.
2. The CartPage prevents the quantity from being reduced below one.

3. The CartProvider submits the updated quantity through the PATCH /api/cart/items/:inventoryId endpoint.
4. The backend validates the request and updates the customer's persisted Cart.
5. The frontend refreshes the cart information and recalculates the displayed quantities and estimated total.
6. The navigation bar updates its cart indicator to reflect the current item count.

Removing Cart Items:

1. The Customer selects Remove for an item.
2. The CartProvider sends a request through the DELETE /api/cart/items/:inventoryId endpoint.
3. The backend removes the selected item from the persisted Cart.
4. If the removed item was the final item, the backend deletes the empty Cart document.
5. The frontend displays the updated cart or empty-cart state.

Proceeding to Checkout:

The CartPage allows the Customer to proceed to the checkout module after reviewing the selected items.

The displayed cart total is an estimate calculated using currently retrieved product prices. The backend performs authoritative stock and price validation during checkout.

If product details are unavailable, the frontend prevents the Customer from proceeding until the issue is resolved.

Implementation Constraints:

- Shopping cart operations are restricted to authenticated Customers.
- Cart items are persisted in MongoDB and associated with the authenticated customer identity.
- The Cart stores inventory identifiers and requested quantities rather than fixed purchase prices.
- Adding or modifying Cart items does not reserve or deduct inventory stock.
- CartDrawer functionality, lens-specific product variants, promotional code application, and separate cart-level tax calculations are not implemented in the current Cart Management workflow.

4.5 Checkout and Order Confirmation

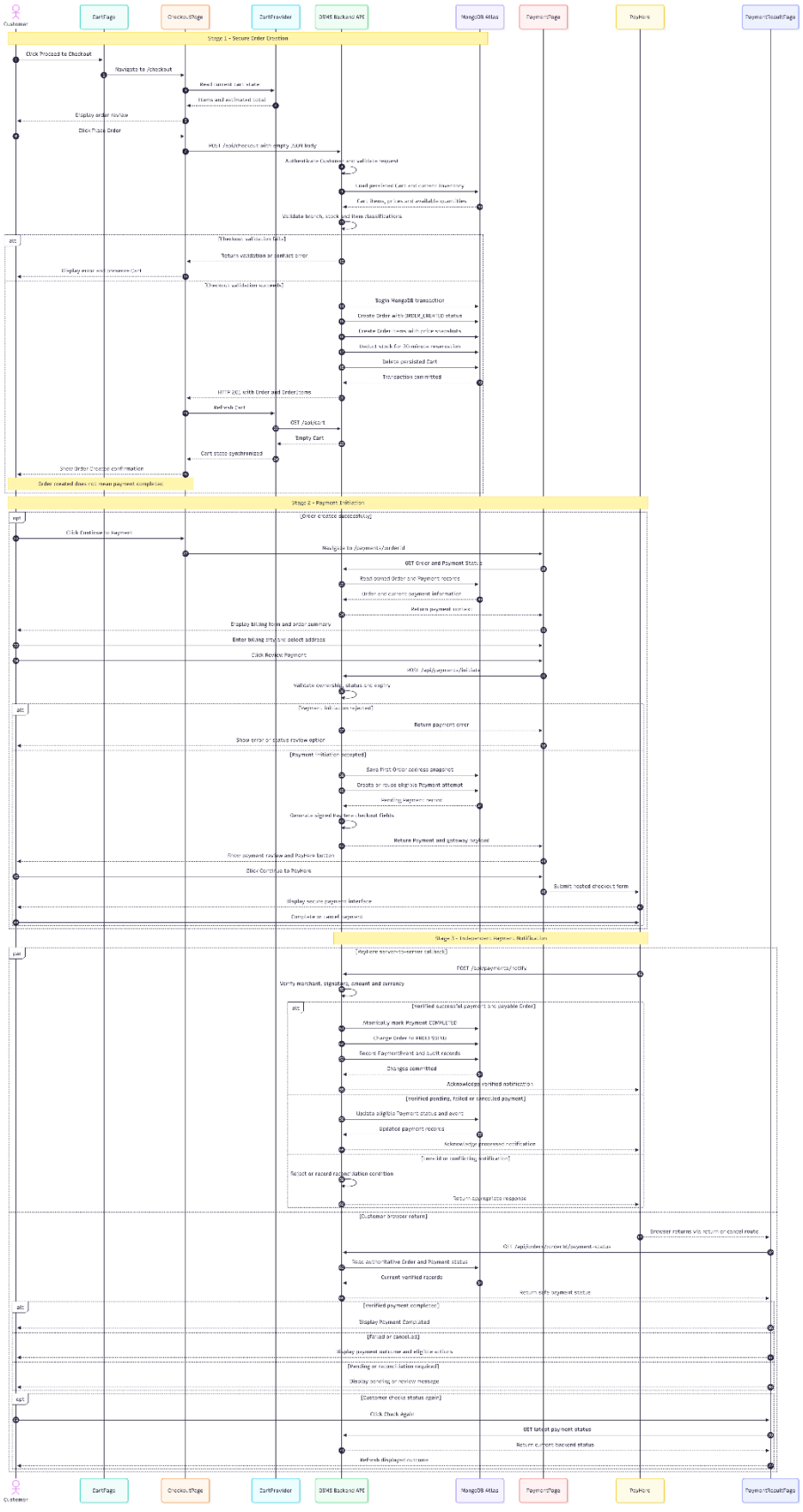


Figure 9: Checkout and Order Confirmation Sequence Diagram

The sequence diagram illustrates the secure checkout, order creation, PayHere payment processing, and payment confirmation workflows of the Optical Shop Management System (OSMS).

Order Review and Creation:

1. The authenticated Customer selects Proceed to Checkout from the shopping cart.
2. The CheckoutPage displays the selected items, quantities, provisional prices, and estimated order total.
3. The Customer reviews the order and selects Place Order.
4. The frontend submits an authenticated request to the POST /api/checkout endpoint without supplying client-calculated prices or order ownership information.
5. The backend retrieves the Customer's persisted Cart and current Inventory records from MongoDB.
6. The checkout service validates inventory ownership, available stock, product prices, and item classifications.
7. If validation succeeds, a MongoDB transaction creates an Order with the ORDER_CREATED status and the associated OrderItem records.
8. The backend deducts the reserved stock quantities, establishes a 30-minute reservation period, and deletes the persisted Cart within the same transaction.
9. Following successful order creation, the frontend refreshes the CartProvider state and displays an Order Created confirmation with the order identifier, items, total amount, and payment navigation.
10. If checkout fails, the transaction is rolled back and the existing Cart is preserved.

Payment Initiation:

1. The Customer selects Continue to Payment.
2. The PaymentPage retrieves the owned Order information and its current payment status.
3. The Customer confirms the billing and delivery address and provides the required billing city.

4. The frontend submits a payment initiation request through POST /api/payments/initiate.
5. The backend validates order ownership, payment eligibility, reservation expiry, and applicable reconciliation conditions.
6. During the first eligible payment initiation, the selected address is saved as the Order's billing and delivery address snapshot. Subsequent attempts reuse the saved Order address.
7. The backend creates or reuses an eligible Payment attempt and generates the signed PayHere checkout information.
8. The Customer reviews the payment information and continues to the PayHere-hosted payment interface.
9. Card and other payment credentials are entered directly through PayHere and are not collected or stored by the OSMS.

Payment Verification and Confirmation:

1. PayHere sends a server-to-server Instant Payment Notification (IPN) to the backend payment notification endpoint.
2. The backend verifies the merchant identity, notification signature, transaction amount, currency, and associated payment record.
3. Duplicate callbacks, inconsistent payment information, expired payment reservations, and conflicting financial states are handled according to the approved payment lifecycle.
4. When a successful payment notification is valid and the Order remains eligible, the backend updates the Payment status to completed and transitions the Order from ORDER_CREATED to PROCESSING.
5. Relevant payment events and audit records are maintained to support payment traceability.
6. The Customer may return to the application through PayHere's return or cancellation URL.

7. The PaymentResultPage retrieves the authoritative payment status through GET /api/orders/:orderId/payment-status.
8. The system displays the verified payment result or an appropriate pending, failed, cancelled, or reconciliation-required message.
9. If the payment result is not yet confirmed, the Customer may request the latest status again.

Reservation Expiry and Failure Handling:

Unpaid Orders are subject to the original 30-minute stock reservation window. Eligible expired unpaid reservations are cancelled, and their reserved inventory quantities are restored through backend recovery procedures. Automatic expiry restoration may be deferred when payment reconciliation or other financial conflicts require investigation.

Payment failures do not automatically establish a new Order or extend the original reservation period. Any permitted payment retry must satisfy the backend's payment eligibility and reconciliation controls.

Implementation Constraints:

- Order creation and payment completion are separate operations.
- Final order amounts and stock reservations are determined by the backend.
- Customer card information is handled by PayHere rather than the OSMS frontend.
- The browser return URL is not authoritative evidence of successful payment.
- The Cart is cleared only after successful backend Order creation.
- The current checkout does not implement a separate 8% tax calculation, card-entry form, or automatic homepage redirect.
- The Order lifecycle follows ADR-004, while payment verification and settlement boundaries follow ADR-011.

4.6 Appointment Booking

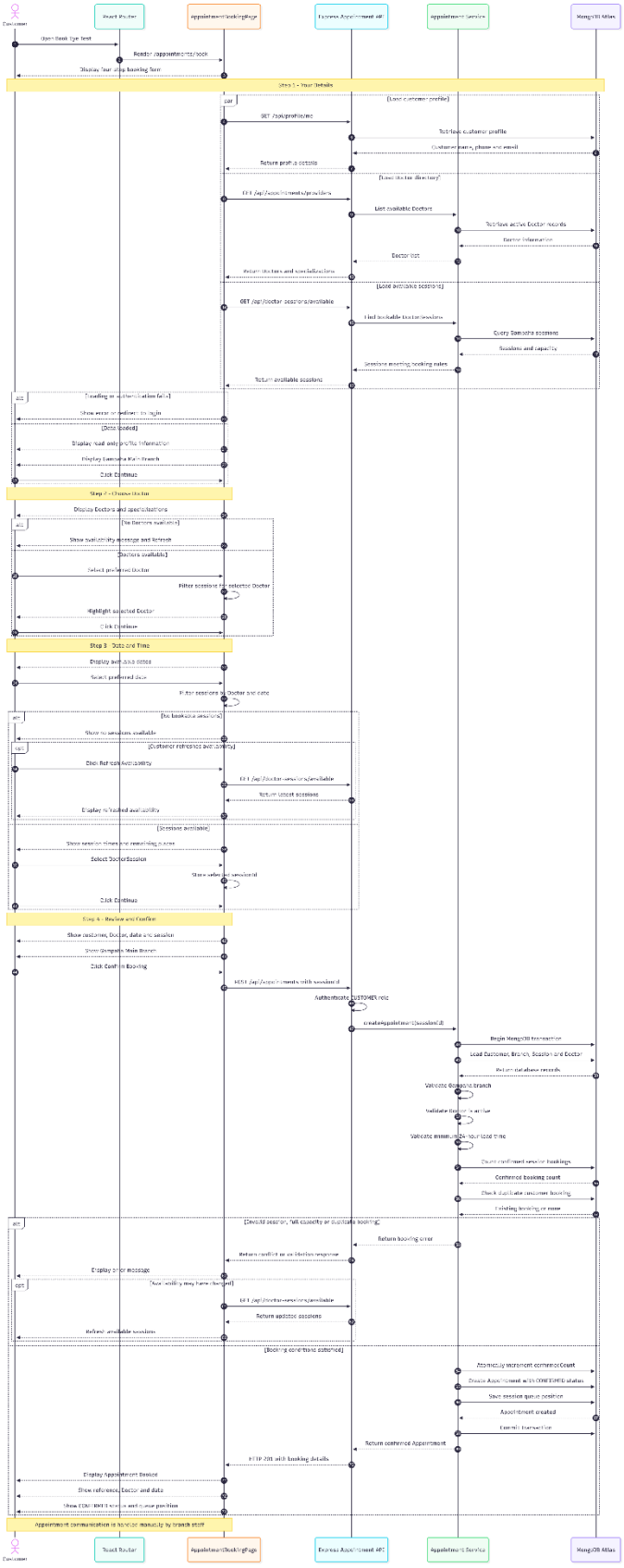


Figure 10: Appointment Booking Sequence Diagram

The Appointment Booking Sequence Diagram illustrates the four-step process used by authenticated Customers to schedule an eye examination through the Optical Shop Management System (OSMS).

The system follows a capacity-based Doctor Session scheduling model, restricted to the approved Gampaha Main Branch.

Step 1 — Your Details

1. The authenticated Customer accesses the Book Eye Test page through /appointments/book.
2. The frontend retrieves the customer's profile, available Doctors, and bookable Doctor Sessions through the backend REST APIs.
3. The customer's name, phone number, and email address are displayed as read-only fields using the registered customer profile.
4. The system displays Gampaha Main Branch as the approved appointment location.
5. The Customer reviews the information and continues to the Doctor selection step.

Step 2 — Choose Doctor

1. The system displays available Doctors and their specializations.
2. The Customer selects a preferred Doctor.
3. The frontend filters the available Doctor Sessions according to the selected Doctor.
4. The Customer proceeds to the Date and Time selection step.

Step 3 — Date and Time

1. The system displays available dates based on the selected Doctor's bookable sessions.
2. The Customer selects a preferred date.
3. The frontend displays the available Doctor Sessions for that date, including their start time, end time, and remaining capacity.
4. The Customer selects an available Doctor Session.
5. If no sessions are available, the system displays an appropriate message and provides an option to refresh availability.
6. The Customer continues to the booking review step.

Step 4 — Review and Confirm

1. The system displays a summary containing the Customer's details, selected Doctor, appointment date, Doctor Session time, and Gampaha Main Branch location.
2. The Customer selects Confirm Booking.
3. The frontend submits the selected sessionId to the backend through the POST /api/appointments endpoint.
4. The backend authenticates the Customer and validates the submitted session identifier.

5. The Appointment Service validates that the selected Doctor Session belongs to the approved Gampaha branch and references an active Doctor.
6. The system verifies that the session begins at least 24 hours after the booking request.
7. The backend checks the session's remaining capacity and prevents duplicate confirmed bookings for the same Customer and Doctor Session.
8. If all conditions are satisfied, a MongoDB transaction updates the session's confirmed booking count and creates the Appointment record.
9. The newly created Appointment receives the CONFIRMED status and a queue position within the Doctor Session.
10. The backend returns the appointment reference and confirmation details.
11. The frontend displays the Appointment Booked screen with the Doctor, date, location, booking reference, confirmation status, and queue position.

Alternative and Exception Flows

- **A1: No Available Doctors or Sessions:** The system displays an availability message and allows the Customer to refresh the available sessions.
- **A2: Session Capacity Reached:** If the selected session becomes full before confirmation, the backend rejects the booking and the frontend refreshes the available sessions.
- **A3: Booking Window Violation:** If the selected session is less than 24 hours away, the system rejects the booking request.
- **A4: Duplicate Booking:** If the Customer already has a confirmed booking for the same session, the backend prevents a duplicate appointment.
- **A5: Invalid or Inactive Doctor:** If the selected session is not associated with an active Doctor, the request is rejected.
- **A6: Authentication or Server Error:** The system displays an appropriate error message. Authentication failures redirect the Customer to the login page.

Appointment Communication

Successfully created appointments immediately receive the CONFIRMED status. The booking details are displayed within the OSMS application.

In accordance with ADR-009, appointment-related communication is handled manually by authorized Branch Assistants. Automated appointment confirmation emails, SMS reminders, and scheduled notification services are not included in the current implementation.

Implementation Constraints

- Appointment bookings are restricted to authenticated Customers.
- The appointment location is restricted to Gampaha Main Branch.
- Bookings must be made at least 24 hours before the Doctor Session begins.

- Multiple Customers may book the same Doctor Session until its configured capacity is reached.
- MongoDB transactions are used to maintain booking and session-capacity consistency.
- Appointment statuses are restricted to CONFIRMED and CANCELLED.
- The frontend submits the selected session identifier rather than customer-supplied Doctor, branch, date, or queue-position values.

4.7 Chatbot Interaction

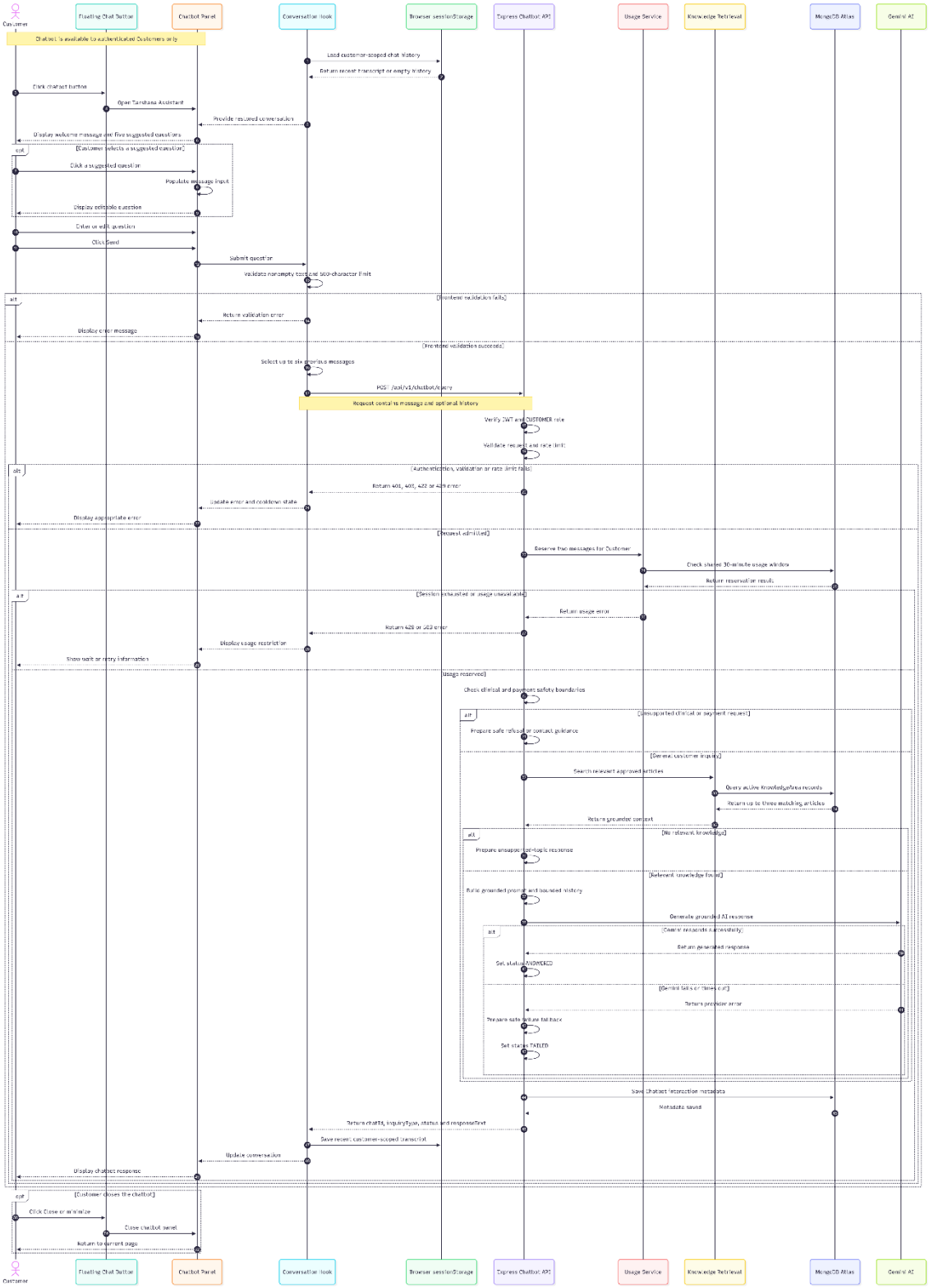


Figure 11: Chatbot Interaction Sequence Diagram

The Chatbot Interaction Sequence Diagram illustrates how authenticated Customers interact with the AI-powered chatbot within the Optical Shop Management System (OSMS).

The implemented chatbot combines a React-based chat interface, backend authentication and usage controls, MongoDB Knowledge Area retrieval, and Google Gemini AI to provide grounded customer assistance.

Opening the Chatbot:

1. An authenticated Customer accesses the OSMS website.
2. The system displays the floating Darshana Assistant chatbot button.
3. The Customer selects the button to open the chatbot panel.
4. The interface displays a welcome message, message input field, and five suggested questions: Frame Recommendations, Track My Order, Opening Hours, Our Services, and Location & Contact.
5. The application restores available customer-scoped conversation history from browser sessionStorage.

Submitting a Customer Inquiry:

1. The Customer enters a question or selects a suggested question.
2. Selecting a suggested question places its text in the message input field without automatically submitting the request.
3. The Customer submits the question.
4. The frontend validates the input and restricts individual questions to a maximum of 500 characters.
5. The conversation hook prepares the question and up to six previous conversation messages as optional context.
6. The frontend submits the authenticated request to POST /api/v1/chatbot/query.
7. The backend validates the Customer's JWT, role, request content, and applicable rate limits.

Session Usage Validation:

1. The backend identifies the Customer using the authenticated request.
2. The ChatbotUsageService checks the Customer's shared 30-minute usage window in MongoDB.
3. The service reserves two message units for each admitted question-and-response exchange.
4. A maximum of 20 message units, equivalent to 10 exchanges, is permitted within the 30-minute window.
5. If the usage limit is reached, the system rejects further requests until the applicable window expires.
6. Refreshing the page, signing out and back in, or clearing conversation history does not reset the backend-controlled usage allowance.

Knowledge Retrieval and AI Response Generation:

1. The ChatbotService checks whether the question involves restricted clinical interpretation or payment credentials.
2. Restricted requests receive an appropriate safe response without being forwarded to Gemini AI.
3. For supported general inquiries, the KnowledgeRetrievalService searches active KnowledgeArea documents in MongoDB.
4. The service retrieves up to three matching knowledge articles using keyword-based matching.
5. If no relevant information is available, the system returns an unsupported-topic response with appropriate contact guidance.
6. If matching knowledge is found, the backend constructs a grounded prompt containing the retrieved content and bounded conversation history.
7. The GeminiChatbotAdapter submits the request to Google Gemini AI.
8. The generated response is returned through the backend to the frontend chatbot interface.

9. If the AI provider fails or times out, the chatbot returns a controlled fallback response.

Response Handling and Conversation Storage:

1. The backend stores Chatbot interaction metadata, including the chat identifier, Customer reference, inquiry type, and response status.
2. The backend returns the generated response text and relevant metadata to the frontend.
3. The frontend displays the response in the chatbot conversation.
4. Recent conversation messages are maintained in browser sessionStorage, scoped to the authenticated Customer.
5. Complete customer conversation transcripts are not persisted in the MongoDB Chatbot collection.
6. The Customer can continue the conversation, submit another question, or close the chatbot panel.

Error and Exception Handling:

- **Invalid Input:** Empty questions and messages exceeding the permitted length are rejected.
- **Unauthorized Access:** Unauthenticated users and users without the CUSTOMER role cannot access the protected chatbot API.
- **Rate Limit Exceeded:** The system displays an appropriate error or cooldown message.
- **Session Limit Reached:** The system prevents additional requests until the shared usage window becomes available again.
- **Unsupported Inquiry:** Requests outside the available knowledge or approved chatbot scope receive a safe fallback response.
- **AI Provider Failure:** A controlled failure response is displayed rather than exposing internal provider errors.

Implementation Constraints:

- The chatbot is available only to authenticated Customers.
- Suggested questions are interface shortcuts and do not invoke independent scripted response workflows.
- Chatbot responses are grounded in internal KnowledgeArea records.
- The chatbot provides general order-tracking guidance but does not independently retrieve a Customer's live order records.
- Automated human-agent escalation is not implemented; appropriate store contact information may be provided.
- Clinical diagnosis, prescription interpretation, and payment credential processing are excluded from the chatbot's permitted functions.
- Backend interaction metadata and usage counters are stored separately from the browser-managed conversation transcript.

4.8 Staff Order Processing and Tracking

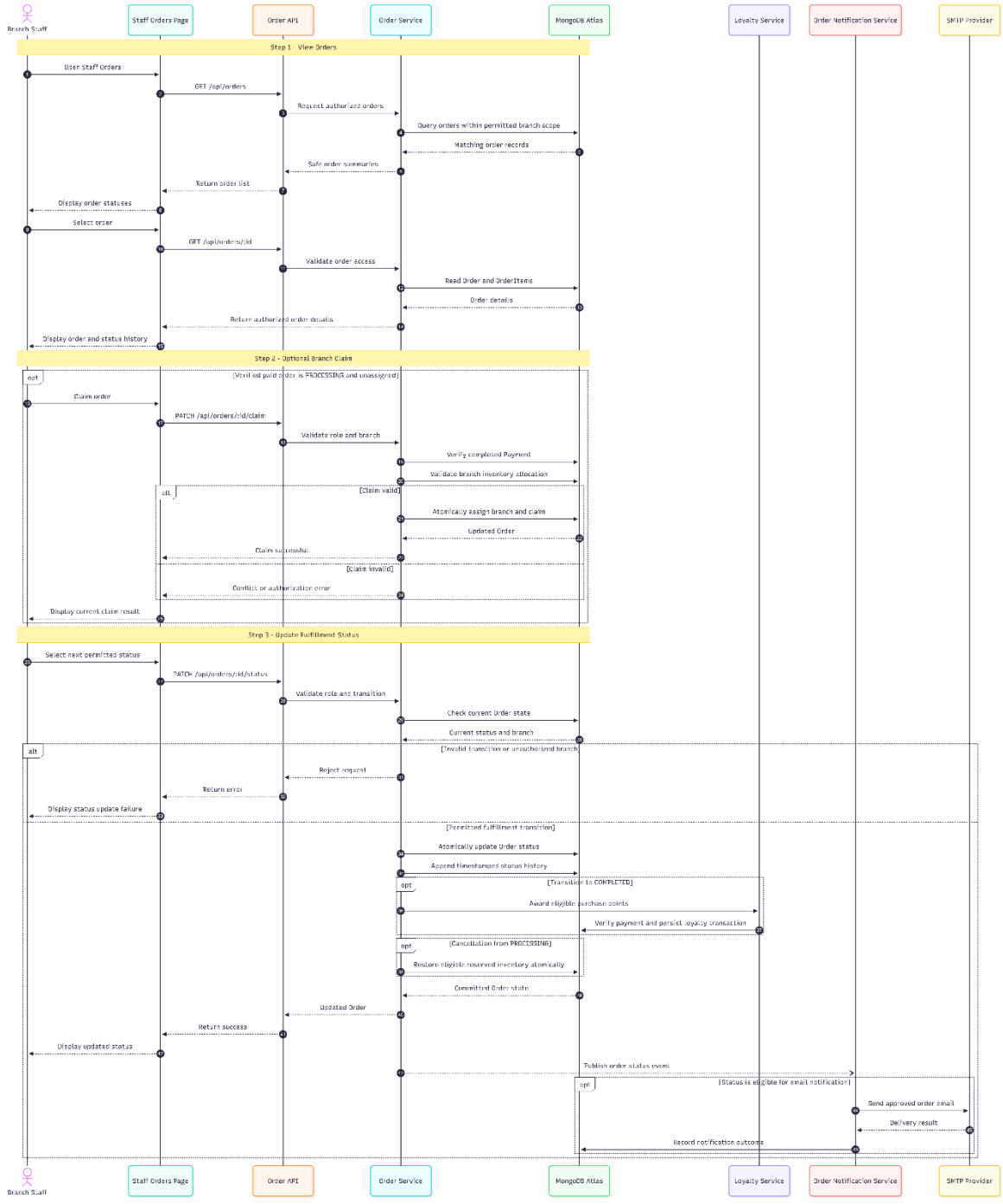


Figure 12: Staff Order Processing and Tracking Sequence Diagram

The sequence diagram illustrates how authorized Branch Assistants and Branch Managers retrieve, claim, and update customer orders during branch fulfillment operations.

Order Retrieval and Tracking

1. Authorized branch staff access the Staff Orders page.
2. The frontend requests orders through GET /api/orders.
3. The backend validates the authenticated role and branch assignment.
4. The Order Service retrieves authorized order records and returns the permitted details.
5. Staff can view individual orders, purchased items, current status, and status history.

Branch Claim and Fulfillment

1. When a verified paid Order is in PROCESSING status but has not been assigned to a branch, authorized staff may claim it.
2. The backend validates the staff member's branch, verifies successful payment, and checks the applicable inventory allocation rules.
3. If validation succeeds, the Order is assigned to the branch through a MongoDB transaction.
4. Claims are rejected when branch ownership, payment status, or inventory conditions are invalid.

Order Status Management

1. Authorized staff select the next permitted Order status.
2. The frontend submits the request through PATCH /api/orders/:id/status.
3. The backend validates the requested status transition and branch authorization.
4. The Order Service updates the status and records the change in the Order's status history.
5. The supported fulfillment sequence is PROCESSING, READY_FOR_PICKUP, PICKED_UP, and COMPLETED.
6. Eligible cancellation operations restore applicable reserved inventory.
7. When an eligible paid Order reaches COMPLETED, the backend calculates and persists the approved loyalty points.

8. Approved status changes can trigger asynchronous order notification processing.

Implementation Constraints

- Operational status updates are restricted to Branch Assistants and Branch Managers.
- Order ownership and branch restrictions are enforced by the backend.
- An Order cannot progress to fulfillment without satisfying the applicable payment and branch-allocation requirements.
- COMPLETED and CANCELLED are terminal Order states.
- Direct third-party courier tracking is not implemented.
- Order notification delivery is separate from the authoritative Order status update.

4.9 Inventory Management and Reorder Alerts

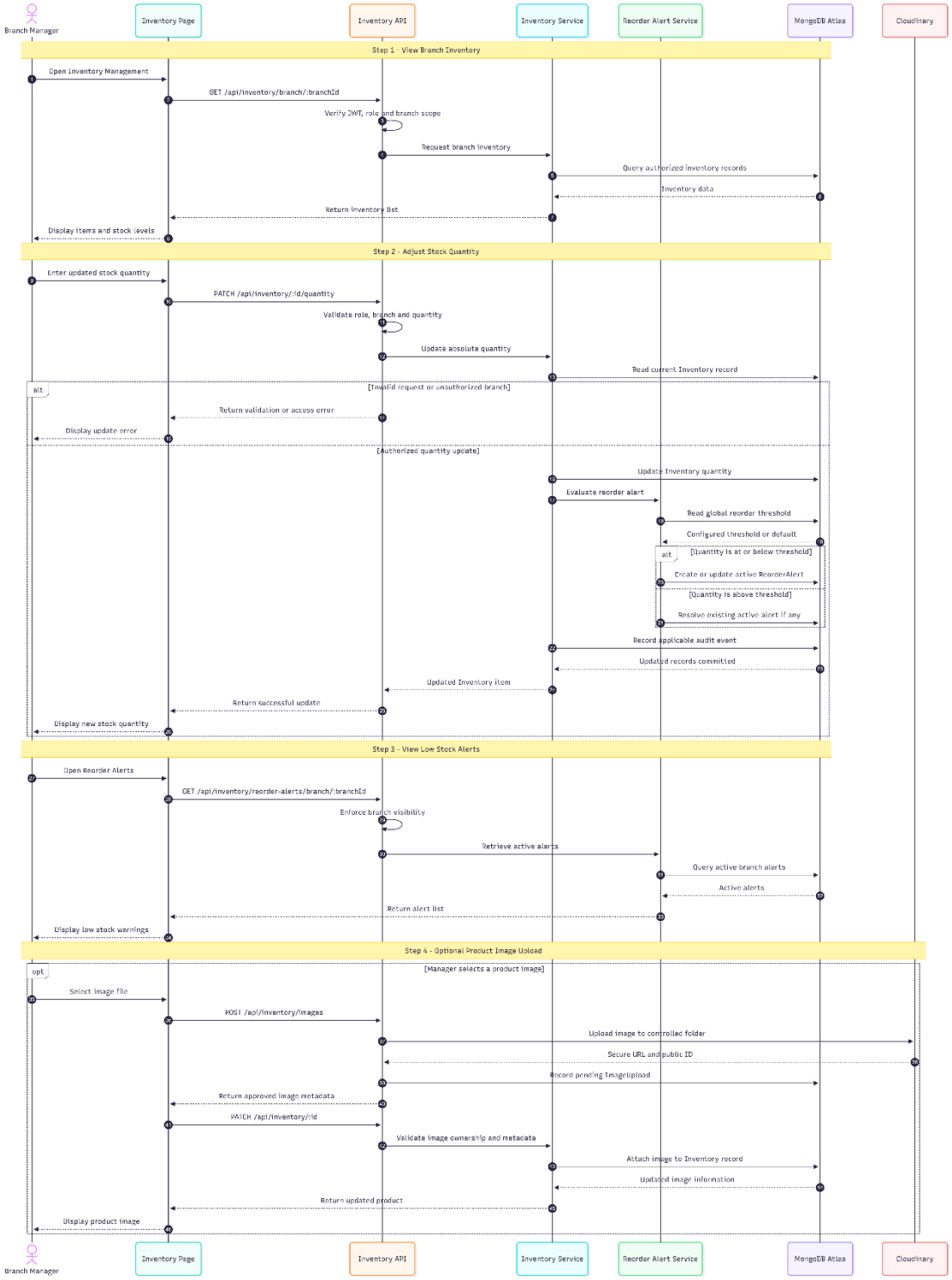


Figure 13: Inventory Management and Reorder Alerts Sequence Diagram

The sequence diagram illustrates the branch-specific inventory management process, reorder alert evaluation, and optional product image management in the Optical Shop Management System (OSMS).

Inventory Retrieval

1. The Branch Manager opens the Inventory Management page.
2. The frontend requests inventory information through GET /api/inventory/branch/:branchId.
3. The backend authenticates the user and validates the applicable branch authorization.
4. Inventory records associated with the authorized branch are retrieved from MongoDB.
5. The frontend displays the inventory items and their current stock quantities.

Stock Quantity Updates

1. The Branch Manager selects an inventory item and enters its new stock quantity.
2. The frontend submits the absolute quantity through PATCH /api/inventory/:id/quantity.
3. The backend validates the item identifier, quantity, role, and branch scope.
4. The Inventory Service updates the stored stock quantity.
5. The ReorderAlertService evaluates the updated quantity against the organization's configured reorder threshold.
6. If the quantity is at or below the threshold, an active reorder alert is created or updated.
7. If the quantity exceeds the threshold, an existing active alert is resolved.
8. Applicable audit information is recorded for the inventory adjustment.
9. The backend returns the updated inventory record.

Reorder Alert Monitoring

1. The Branch Manager opens the Reorder Alerts section.
2. The frontend retrieves active alerts for the authorized branch.

3. The backend returns the permitted alert records, including relevant inventory quantities and threshold information.
4. The frontend displays the current low-stock warnings.

Product Image Management

1. An authorized Branch Manager may select an image for an inventory item.
2. The frontend uploads the file through POST /api/inventory/images.
3. The backend uploads the image to the controlled Cloudinary inventory folder.
4. Cloudinary returns an approved image URL and public identifier.
5. The backend records image-upload metadata and validates ownership before attaching the image to an inventory item.
6. The Inventory record is updated with the approved image reference.
7. Image replacement and deletion operations apply the implemented asset-cleanup procedures.

Implementation Constraints

- Branch Managers perform authorized branch-specific stock management.
- System Administrators have restricted price-override access rather than unrestricted stock-quantity management.
- The reorder threshold is organization-wide and defaults to five units unless changed through an authorized configuration operation.
- Reorder alerts represent low-stock conditions, not automatically generated supplier purchase orders.
- Inventory image storage uses Cloudinary rather than Amazon S3.
- Direct automatic inter-branch stock transfers are not included in the current implementation.

4.10 Doctor Session Management

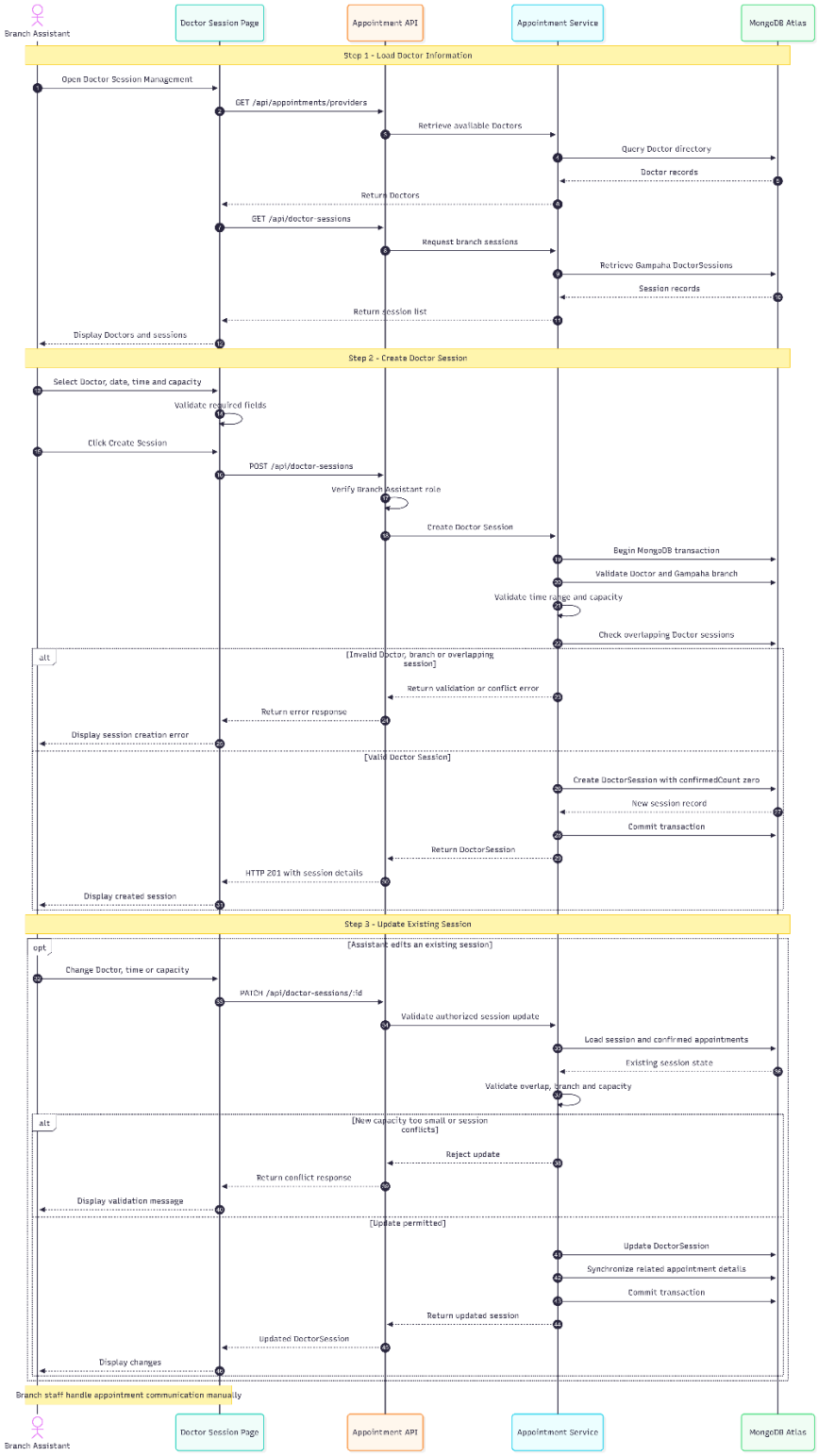


Figure 14: Doctor Session Management Sequence Diagram

The sequence diagram illustrates how an authorized Branch Assistant creates and updates Doctor Sessions for the Optical Shop Management System (OSMS).

Doctor and Session Retrieval

1. The Branch Assistant accesses the Doctor Session Management interface.
2. The frontend retrieves Doctor information through GET /api/appointments/providers.
3. Existing Doctor Sessions are retrieved through GET /api/doctor-sessions.
4. The backend validates the staff member's role and branch assignment before returning authorized information.

Doctor Session Creation

1. The Branch Assistant selects a Doctor from the approved Doctor directory.
2. The Branch Assistant specifies the session start time, end time, and maximum patient capacity.
3. The frontend submits the session details through POST /api/doctor-sessions.
4. The backend verifies that the requester is an authorized Branch Assistant assigned to Gampaha Main Branch.
5. The Appointment Service validates the selected Doctor's active status, branch restrictions, session timing, and capacity.
6. The backend checks whether the Doctor has another session overlapping the requested period.
7. If the request is valid, a MongoDB transaction creates the DoctorSession record with an initial confirmed booking count of zero.
8. The frontend displays the successfully created session.
9. If validation fails, the request is rejected and the existing session records remain unchanged.

Doctor Session Updates

1. The Branch Assistant selects an existing session for editing.
2. The frontend submits approved modifications through PATCH /api/doctor-sessions/:id.
3. The backend verifies the staff member's authorization and the session's branch.
4. The service validates the proposed Doctor, timing, and capacity changes.
5. The session capacity cannot be reduced below the number of confirmed bookings.
6. Overlapping Doctor Sessions are rejected.
7. When a permitted change affects the session date, time, or Doctor, the backend synchronizes the relevant information in associated Appointment records.
8. The updated session information is returned to the frontend.

Implementation Constraints

- Only authorized Branch Assistants can create or modify Doctor Sessions.
- Branch Managers may view permitted appointment information but do not create Doctor Sessions.
- New Doctor Sessions must reference approved Doctor directory records.
- Doctor Session creation is restricted to Gampaha Main Branch.
- A Doctor cannot be assigned to overlapping sessions.
- Confirmed Appointment capacity must remain valid when a session is modified.
- Appointment communication is handled manually under ADR-009.

4.11 Prescription Recording and History

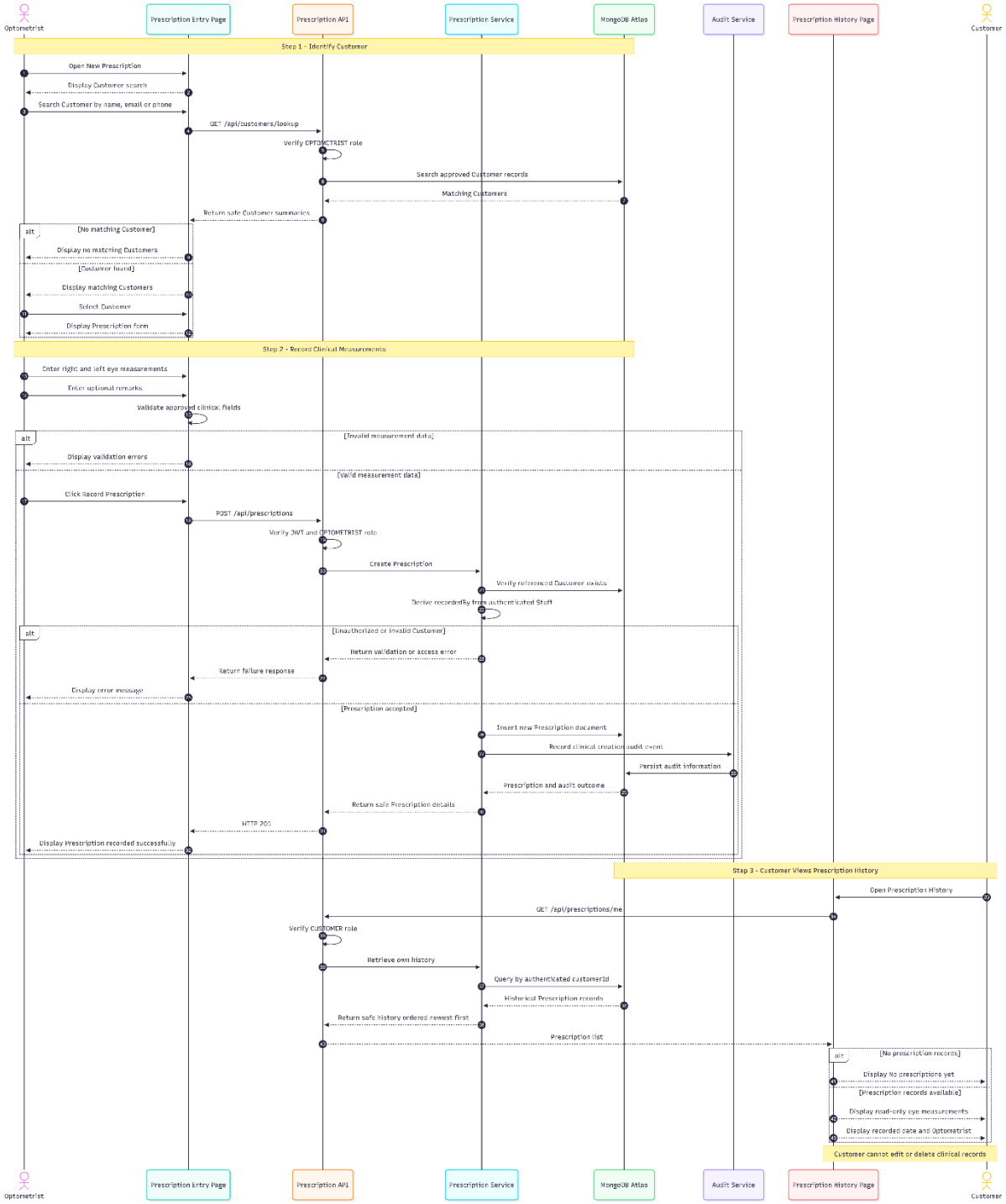


Figure 15: Prescription Recording and History Sequence Diagram

The sequence diagram illustrates the clinical prescription recording process performed by authorized Optometrists and the read-only prescription history provided to Customers.

Customer Identification

1. An authenticated Optometrist opens the New Prescription page.
2. The Optometrist searches for a Customer using their name, email address, or phone number.
3. The frontend submits the search request through the authorized Customer lookup API.
4. The backend validates the Optometrist's permissions and retrieves matching Customer records.
5. The Optometrist selects the appropriate Customer before entering clinical measurements.

Prescription Recording

1. The Optometrist enters the applicable right-eye and left-eye measurements, including distance and reading information.
2. Optional clinical remarks may also be recorded.
3. The frontend validates the supplied measurement values and permitted data fields.
4. The frontend submits the prescription through POST /api/prescriptions.
5. The backend authenticates the Optometrist and validates the target Customer.
6. The recording Optometrist's identifier is obtained from the authenticated JWT context rather than client-supplied information.
7. The Prescription Service creates a new Prescription document in MongoDB.
8. A corresponding audit event is recorded for the clinical creation operation.
9. The backend returns a safe prescription response, and the frontend displays a success confirmation.
10. Existing historical prescriptions are preserved rather than overwritten.

Customer Prescription History

1. An authenticated Customer opens the Prescription History page.
2. The frontend submits a request through GET /api/prescriptions/me.
3. The backend identifies the Customer from the verified authentication token.
4. The Prescription Service retrieves records belonging only to that Customer.
5. The results are returned in descending order of recording date.
6. The frontend displays the recorded measurements, relevant Optometrist information, recording date, and remarks.
7. If no prescriptions exist, the system displays an appropriate empty-history message.

Security and Data Integrity

- Prescription creation is restricted to authenticated Optometrists.
- Customers can view their own prescription histories but cannot create, modify, or delete clinical records.
- The backend applies Customer ownership restrictions when retrieving prescription history.
- Prescription records maintain the identity of the recording Optometrist.
- Relevant clinical creation events are recorded through the Audit Service.
- Prescription editing, deletion, and scanned prescription uploads are not included in the current implementation.

Chapter 5: UI/UX Design

5.1 Screen Mock-ups for Key User Interfaces

The user interface of the Optical Shop Management System (OSMS) was initially designed using Figma to establish the visual layout, navigation structure, branding, and interaction patterns for the main application modules.

The original interactive design prototype is available through the following link:

Figma Design: https://www.figma.com/proto/OecN23wEqdSIWsY4xjWwUQ/Darshana-Opticals

The initial mock-ups covered customer registration and login, the homepage, eyewear catalogue, product details, shopping cart, checkout, appointment booking, and chatbot interaction.

During implementation, the original designs were refined to align with approved requirements, architectural decisions, security controls, and backend functionality.

The final React-based application maintains the core visual identity of Darshana Opticals, including its blue-and-white branding, navigation patterns, responsive layouts, and customer-oriented interface design.

The implemented user interface includes the following main functional areas:

- **Homepage:** Provides category navigation, product discovery, featured eyewear, promotional content, and links to major customer services.
- **Authentication:** Provides customer registration, login, password recovery, and role-based navigation.
- **Customer Account:** Supports profile management, order history, prescription history, loyalty point visibility, and registered address information.
- **Product Catalogue:** Supports category, brand, and price-range filtering, with product information retrieved from backend inventory records.
- **Product Details:** Displays available product information and provides authenticated Customers with Add to Cart functionality.

- **Shopping Cart:** Uses a dedicated CartPage to manage persisted cart items, quantities, removal operations, and estimated totals.
- **Checkout and Payment:** Separates backend-authorized Order creation from PayHere-hosted payment processing. Customer card credentials are not collected by the OSMS.
- **Appointment Booking:** Provides a four-step Doctor Session booking process with appointment availability, capacity validation, and booking confirmation.
- **AI Chatbot:** Provides an interactive customer assistance interface using approved Knowledge Area content and Gemini AI.
- **Staff Interfaces:** Support authorized order fulfillment, inventory management, Doctor Session administration, prescription recording, and branch-level reports.
- **Administrative Interfaces:** Support authorized staff and Doctor management and organization-wide reporting.

The original Figma mock-ups remain available as historical design references. Any screens that differ from the implemented application are distinguished from the final interface design.

The final SDS should use representative screenshots from the implemented application to demonstrate the delivered user experience. Functionality shown only in the original mock-ups, including wishlists, product reviews, virtual try-on, direct card-entry checkout, and automated appointment reminders, must not be presented as implemented features.

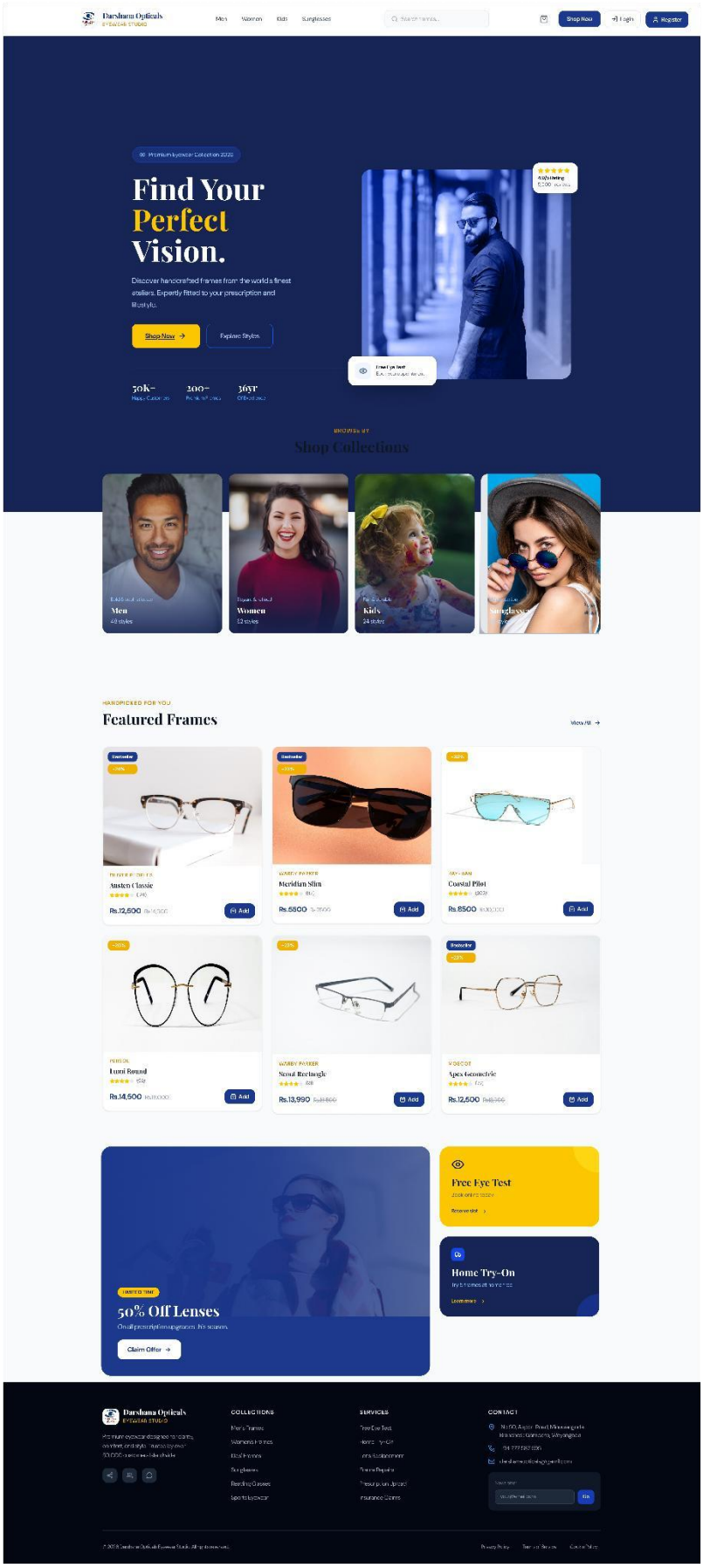


Figure 16: Landing Page

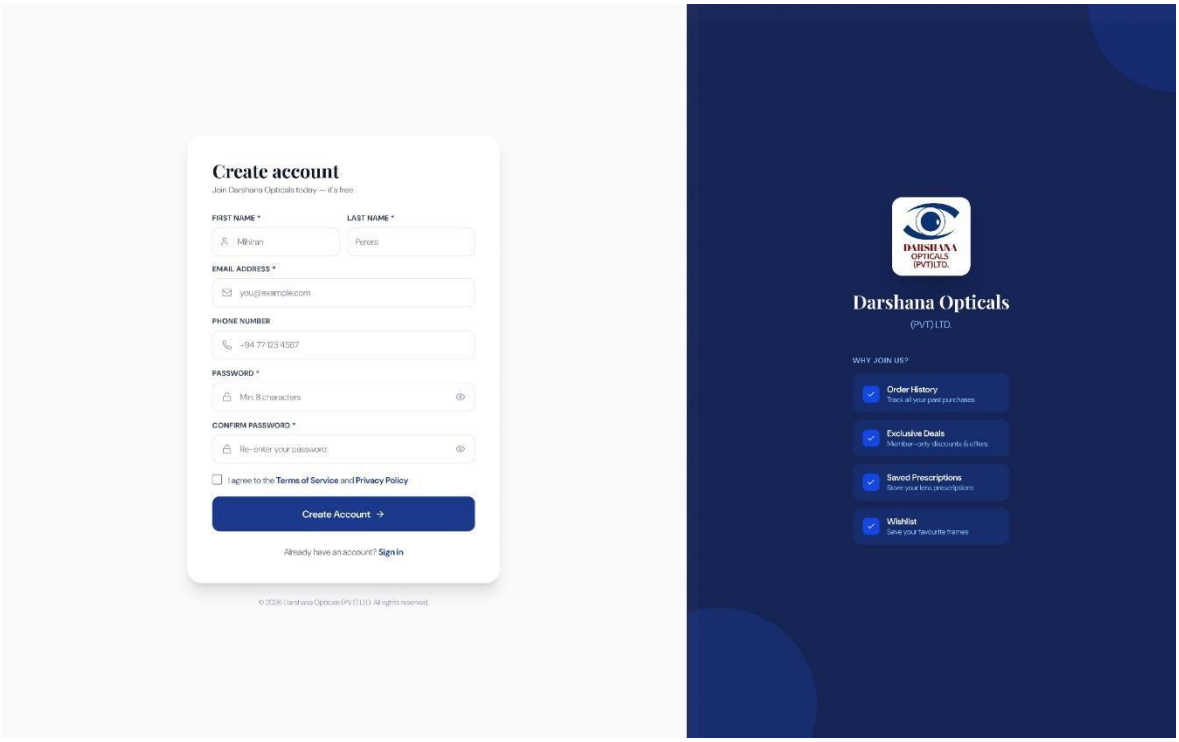


Figure 17: Sign Up Page

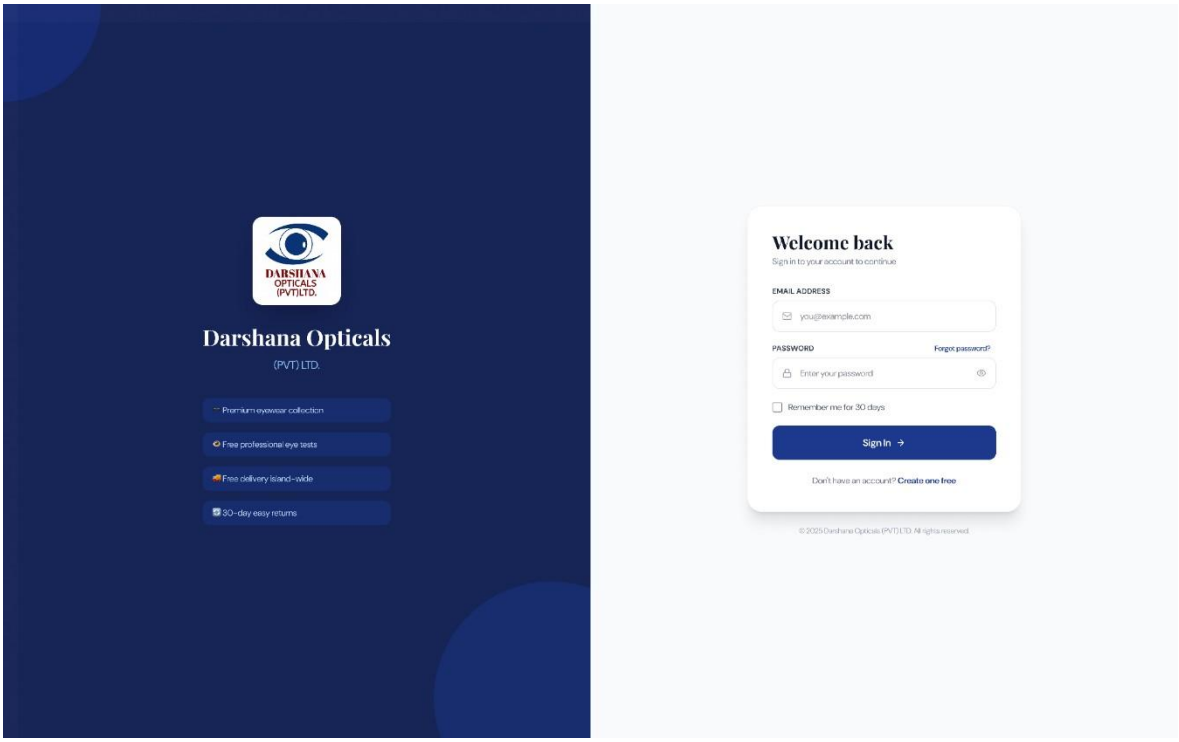


Figure 18: Login Page

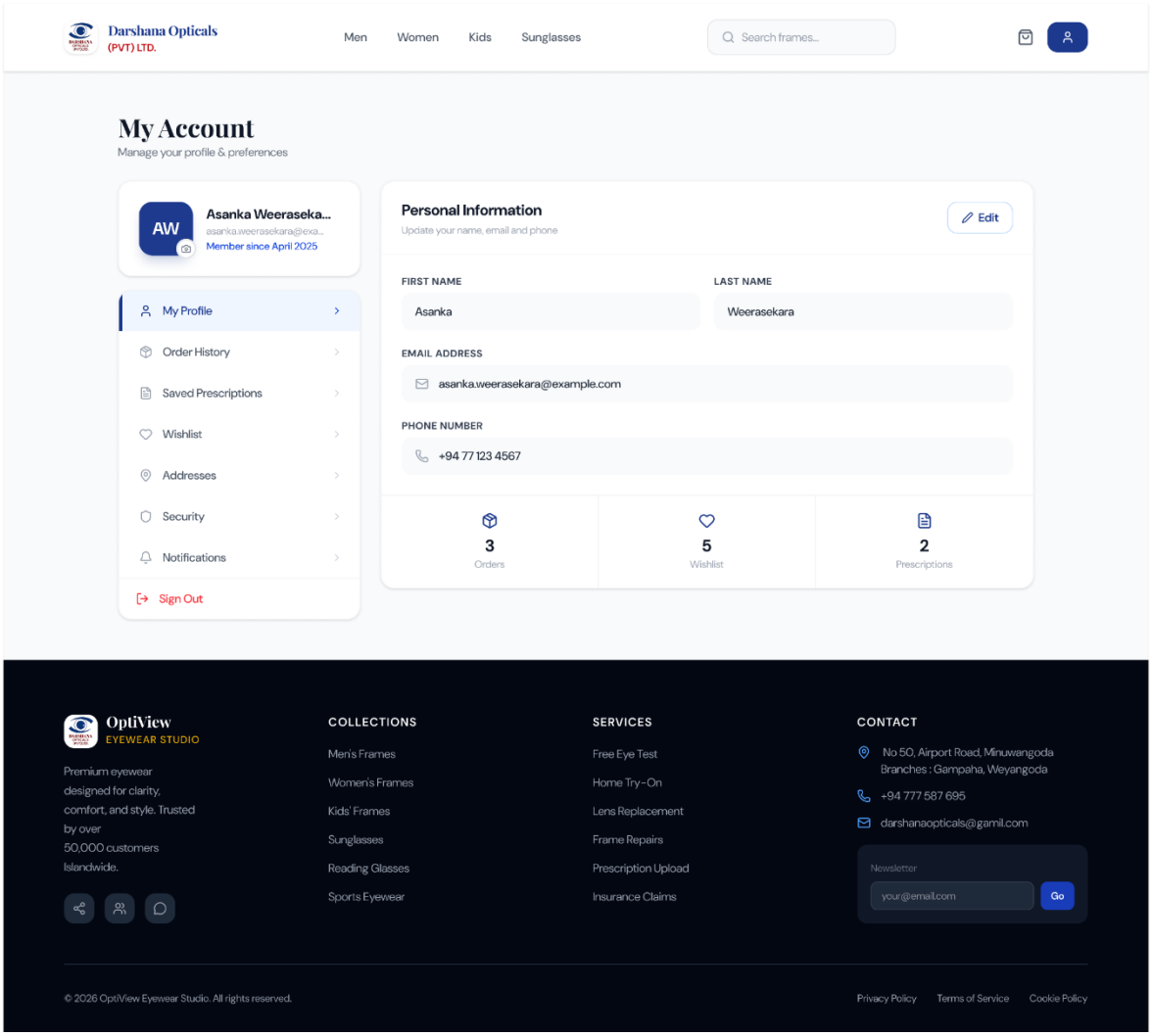


Figure 19: Profile Page

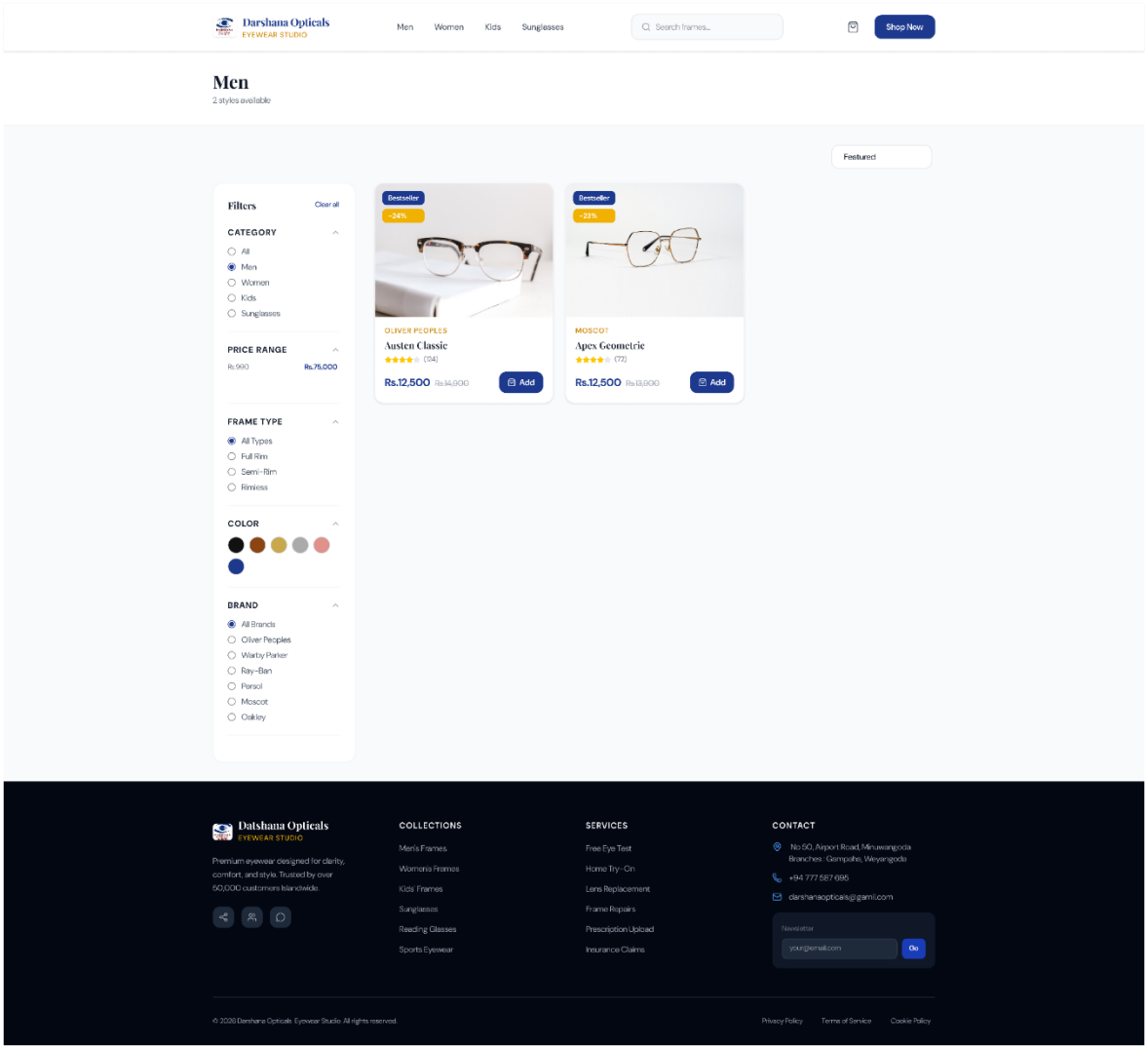


Figure 20: Men's Collection Page

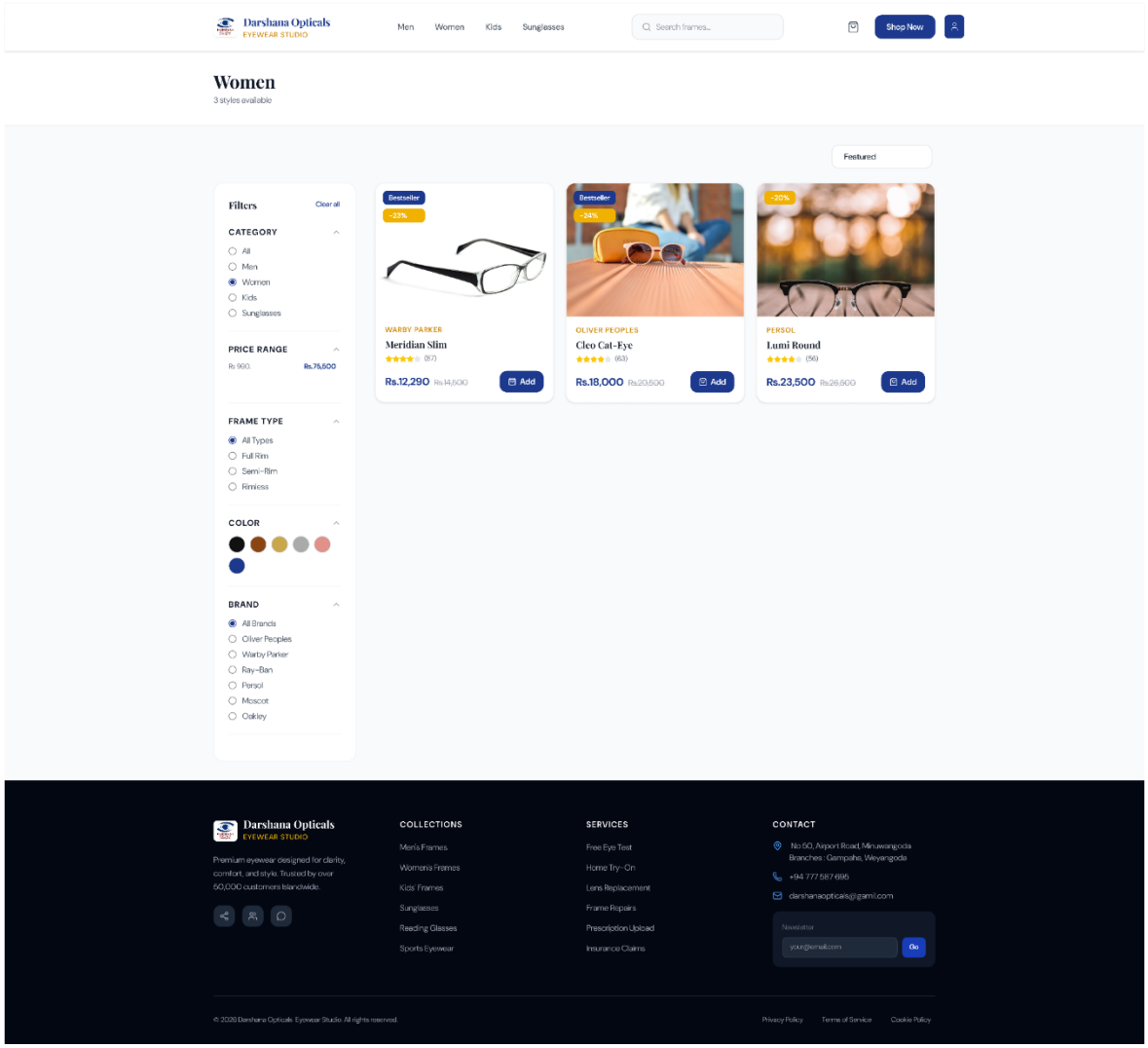


Figure 21: Women's Collection Page

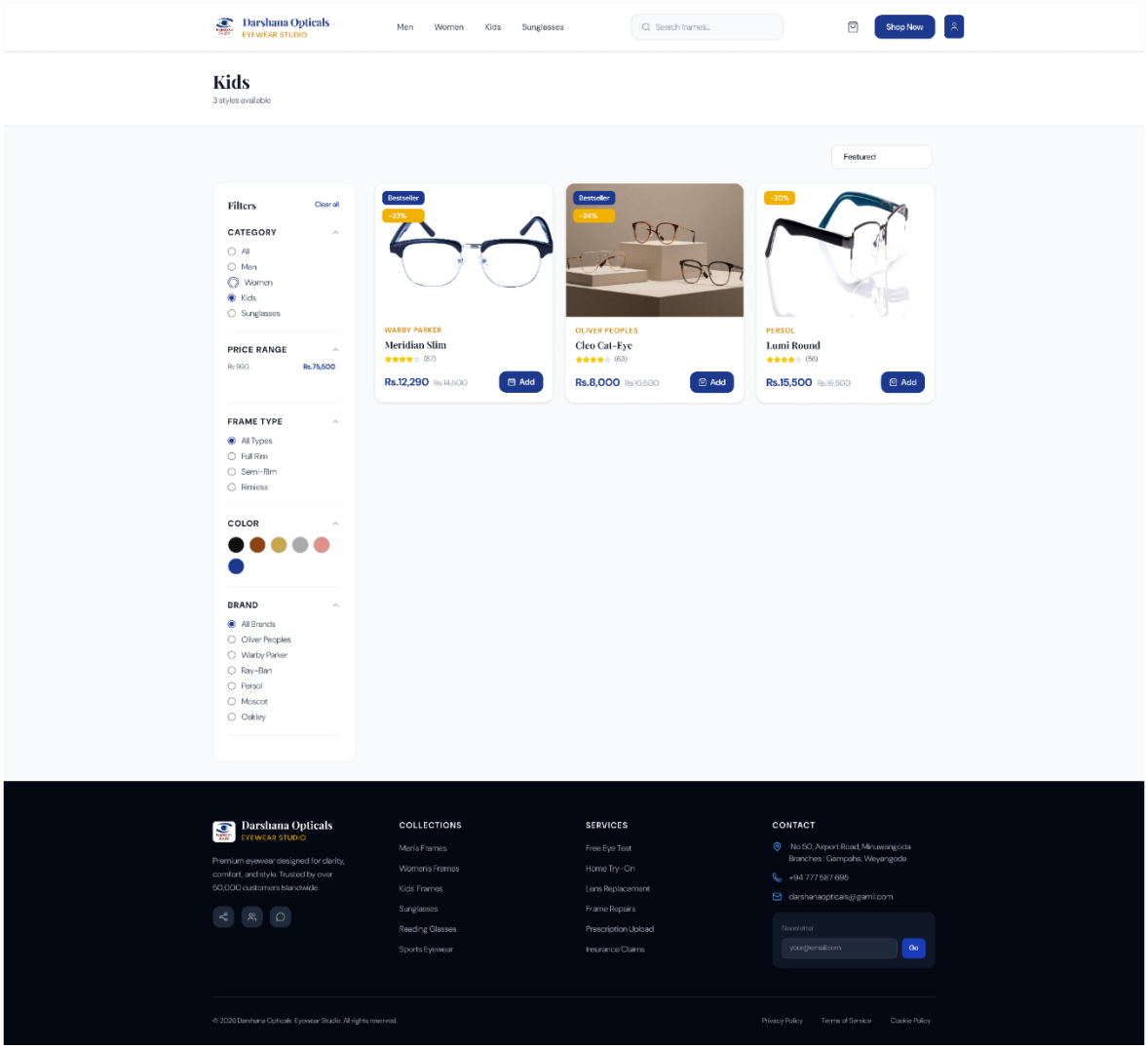


Figure 22: Kids' Collection Page

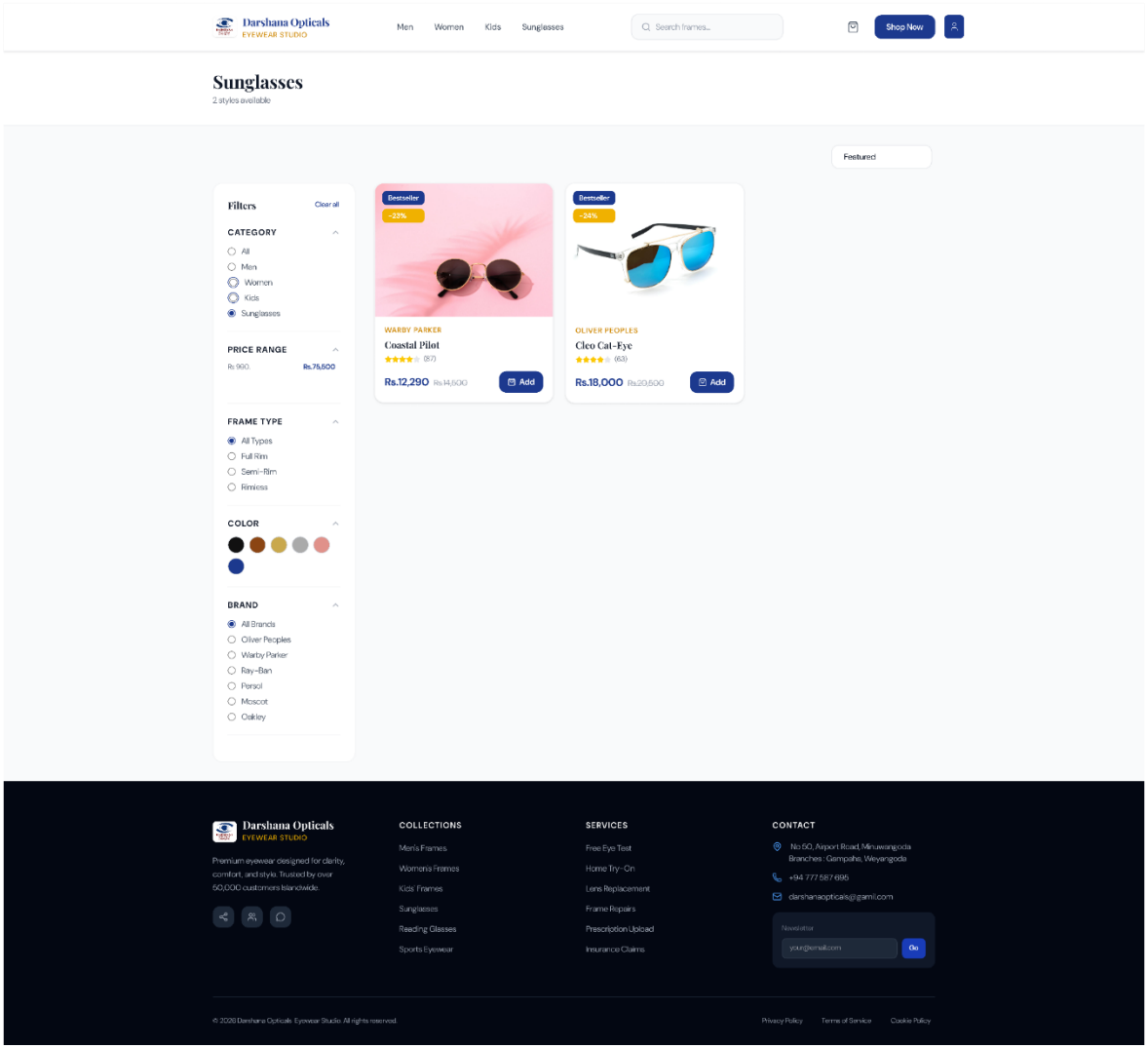


Figure 23: Sunglasses Collection Page

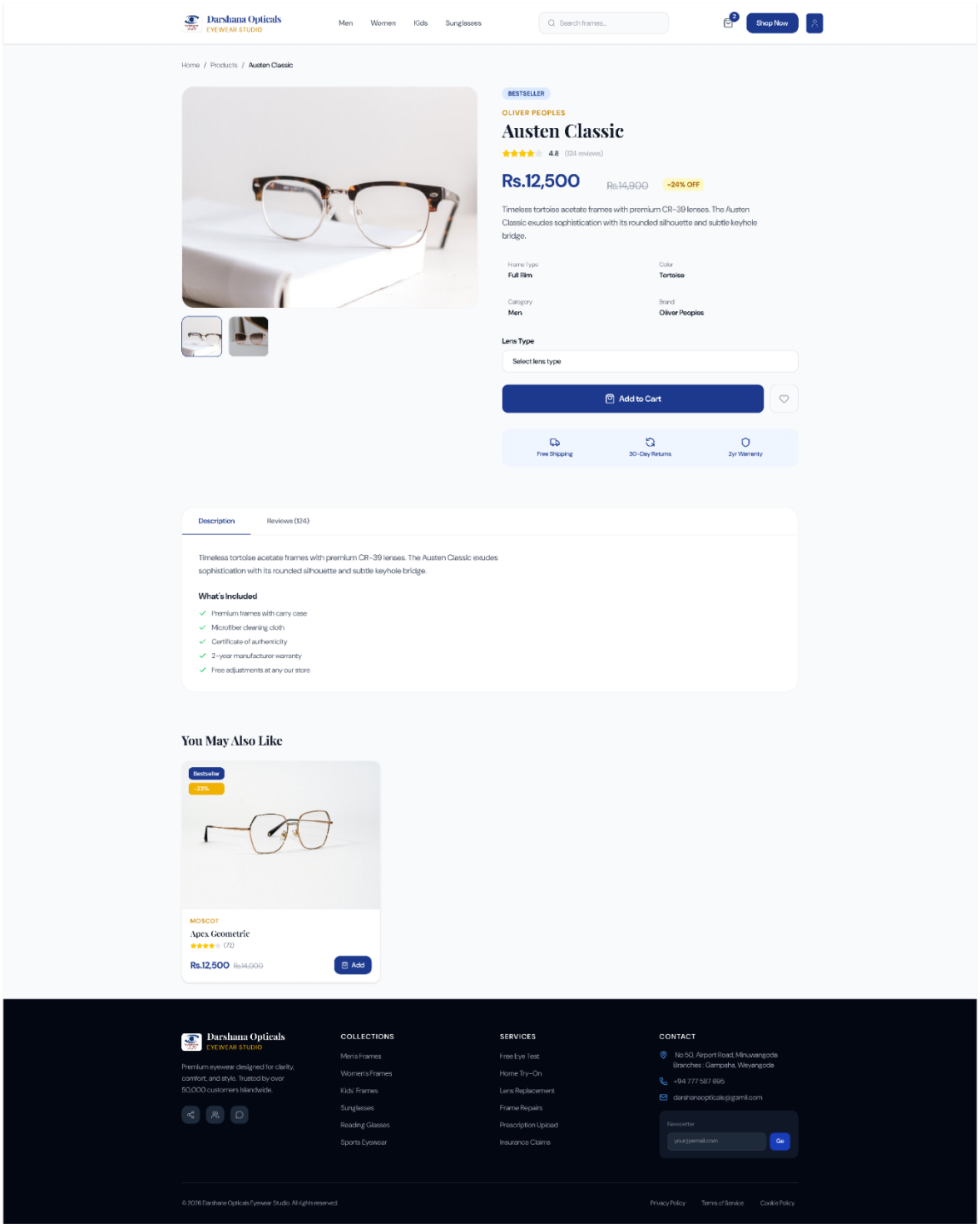


Figure 24: Product Information Page

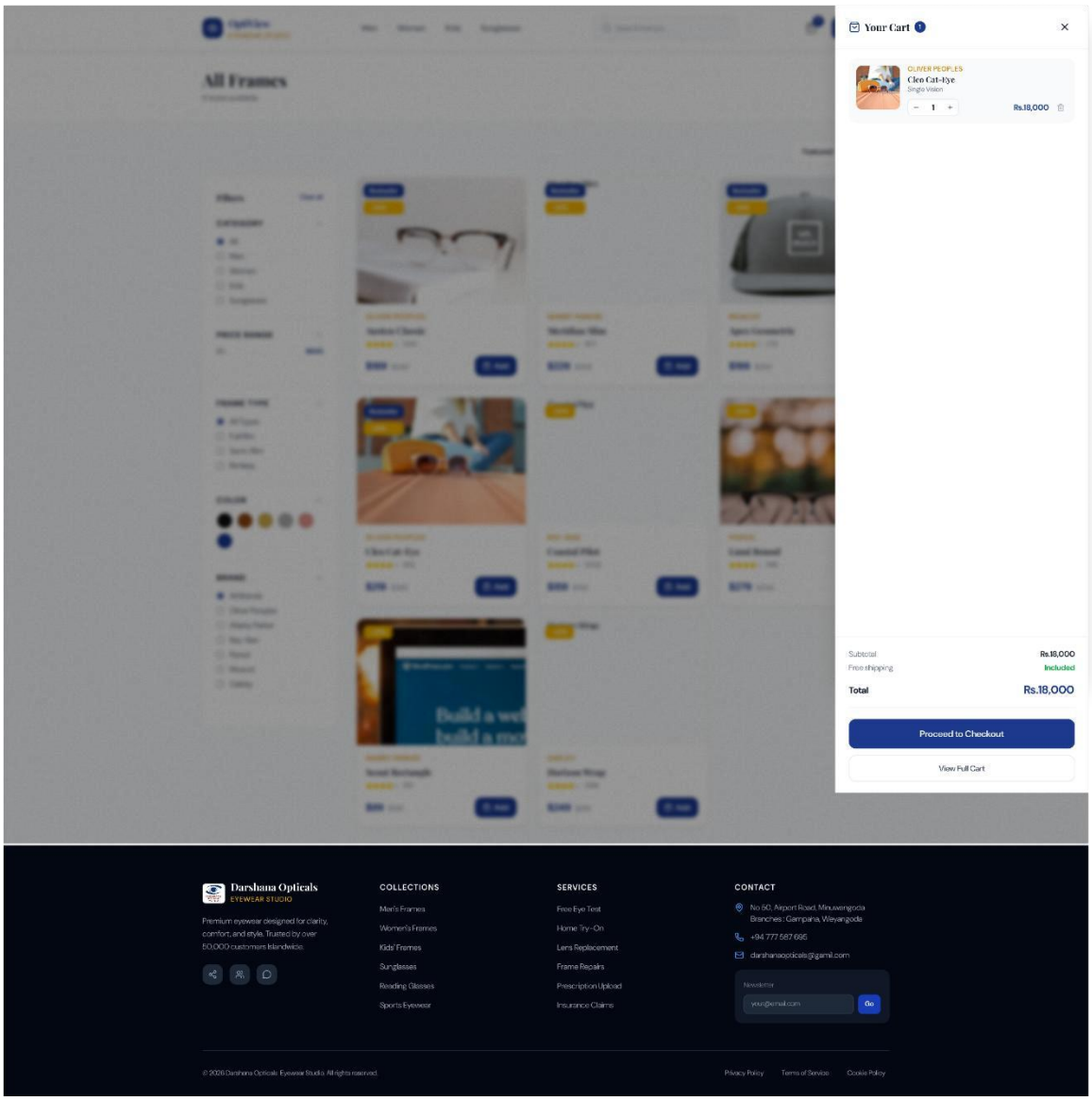


Figure 25: Cart Drawer

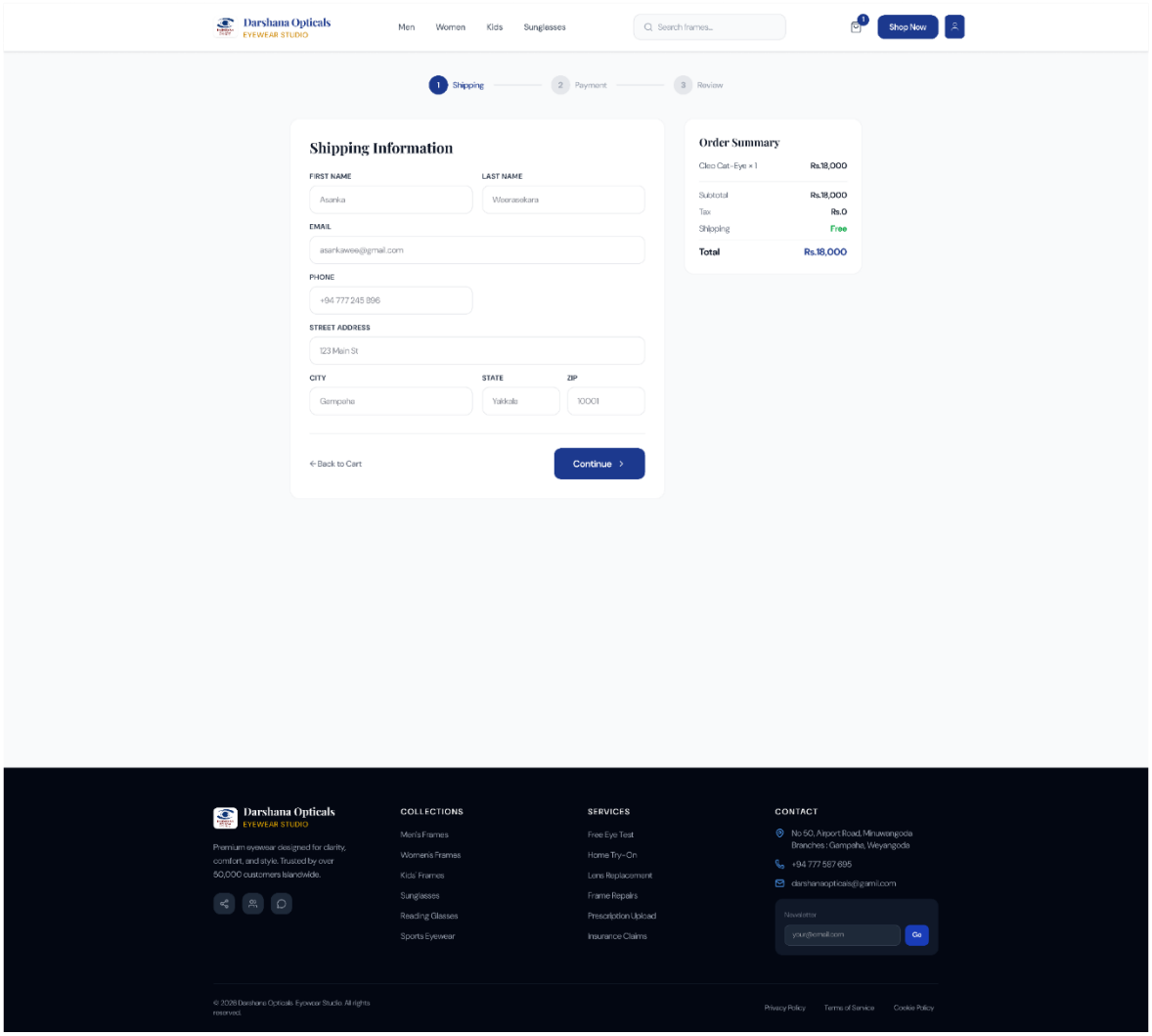


Figure 26: Shipping Information Page

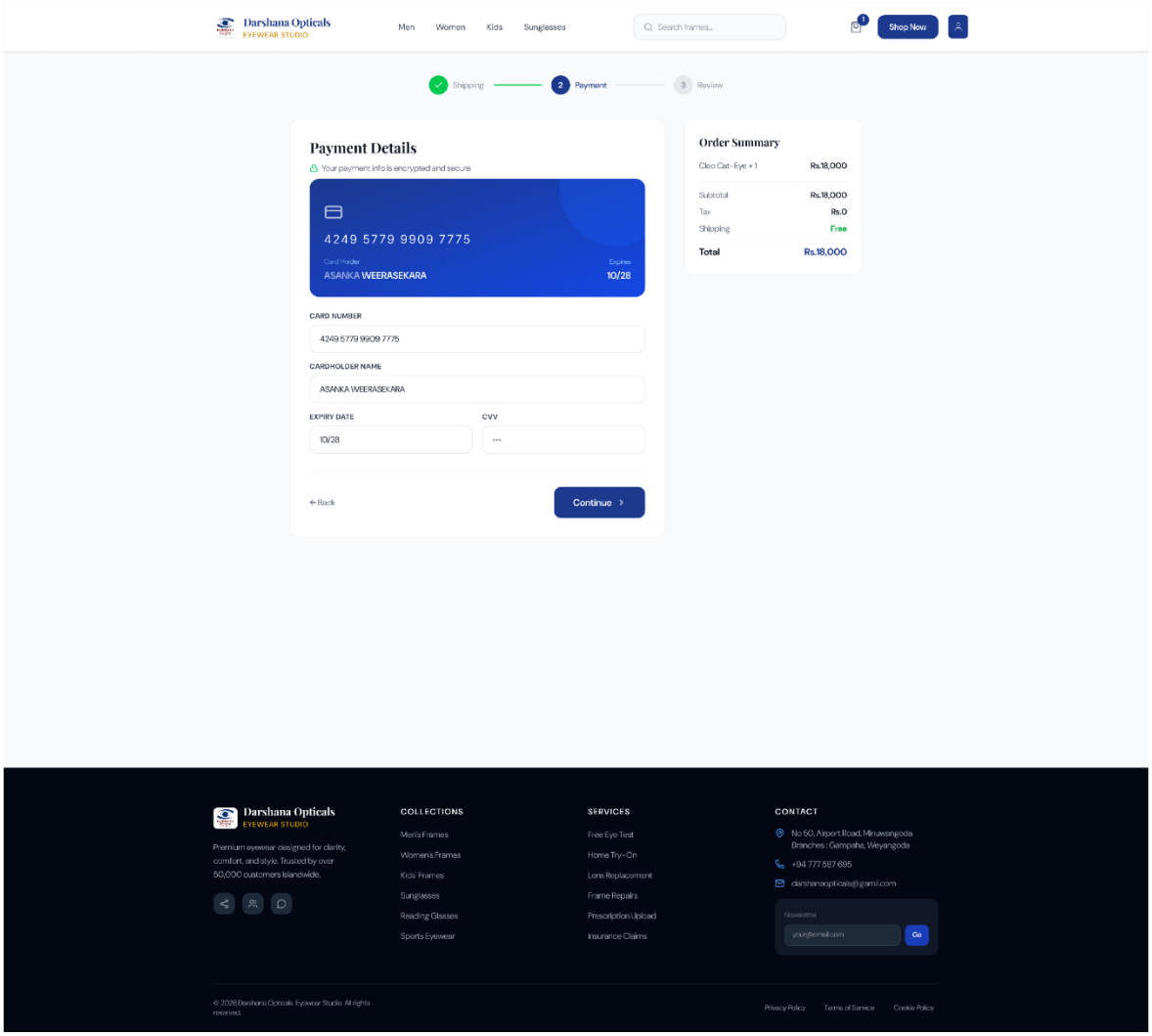


Figure 27: Payment Details Page

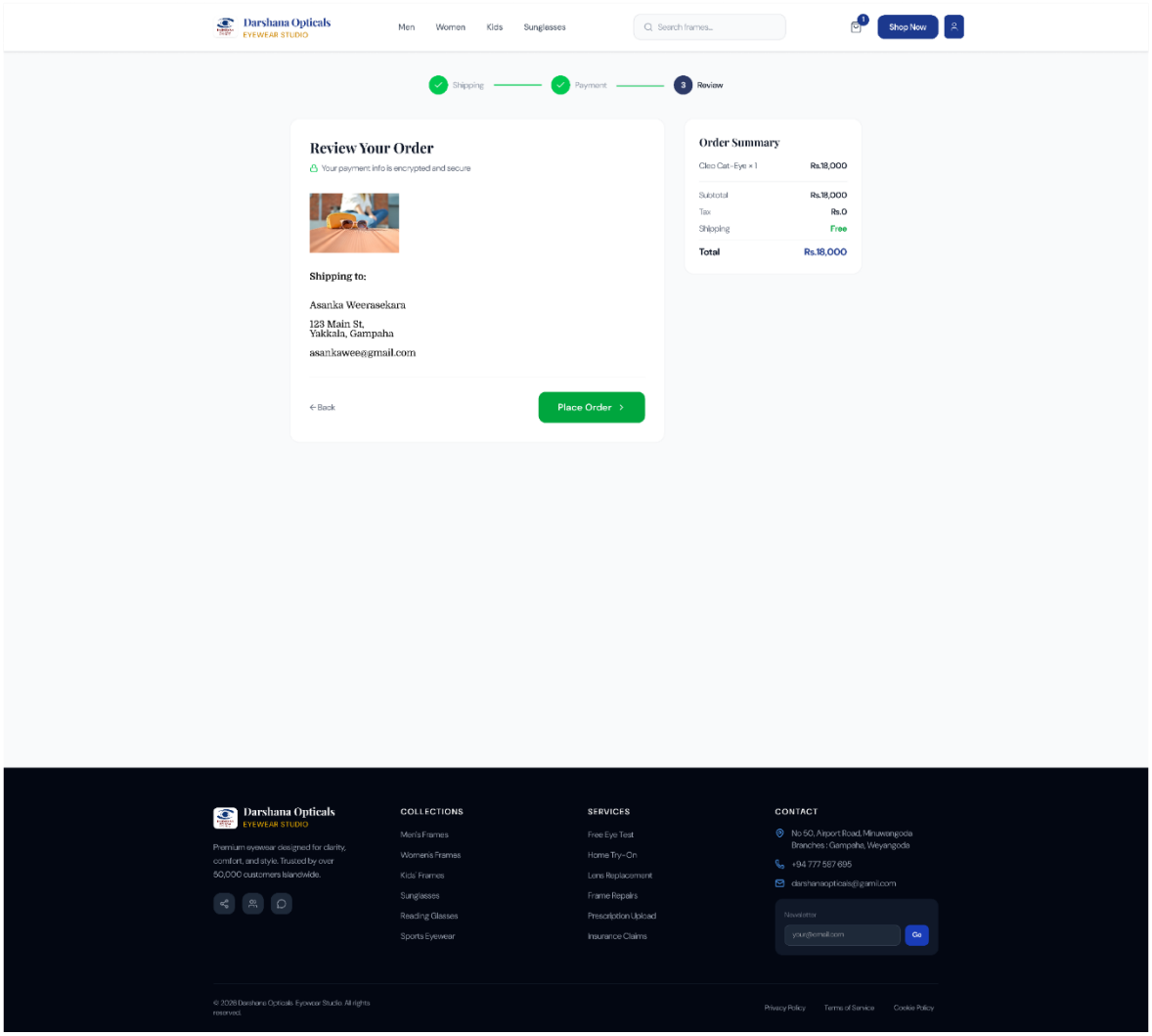


Figure 28: Order Review Page

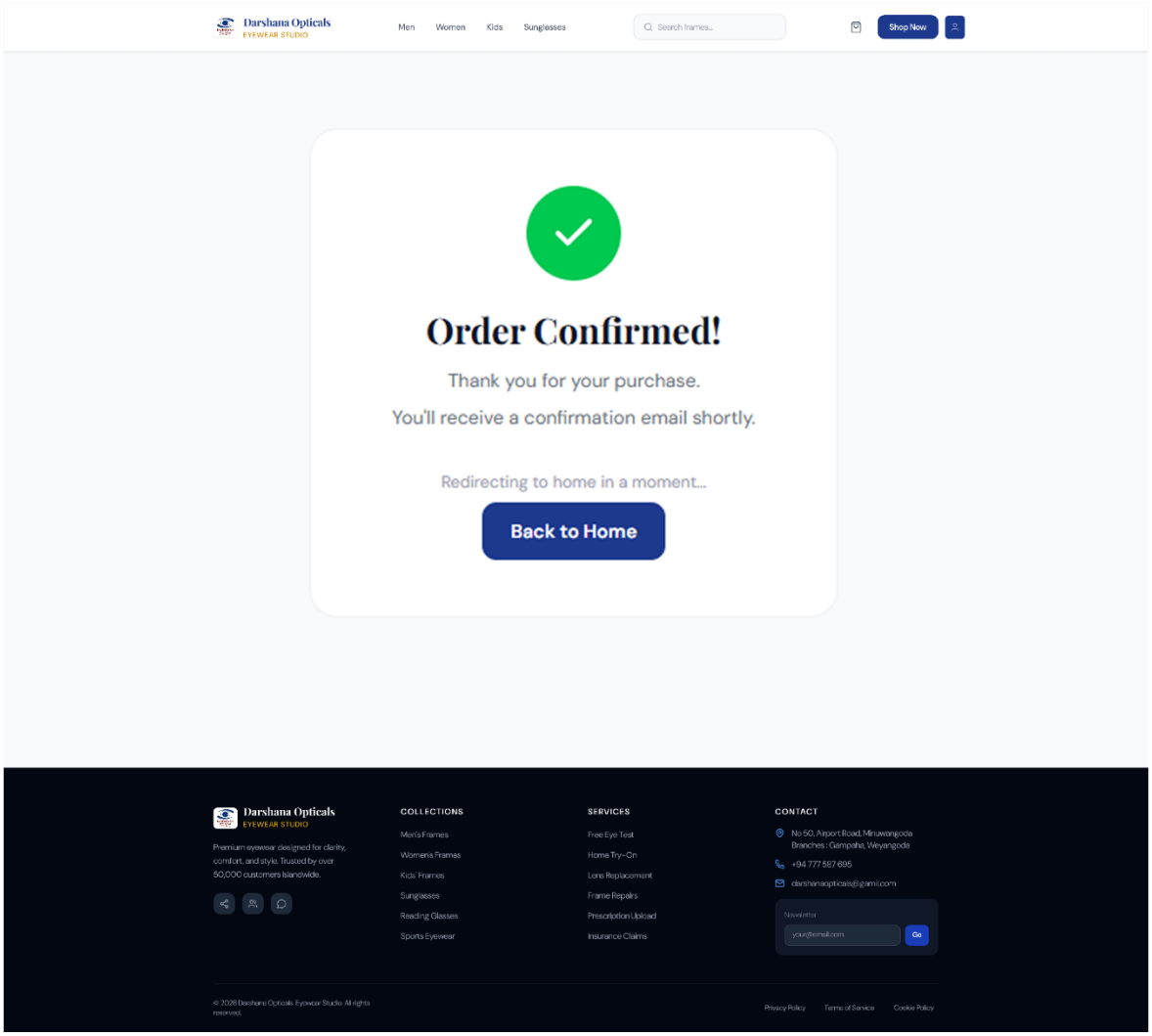


Figure 29: Order Confirmation Page

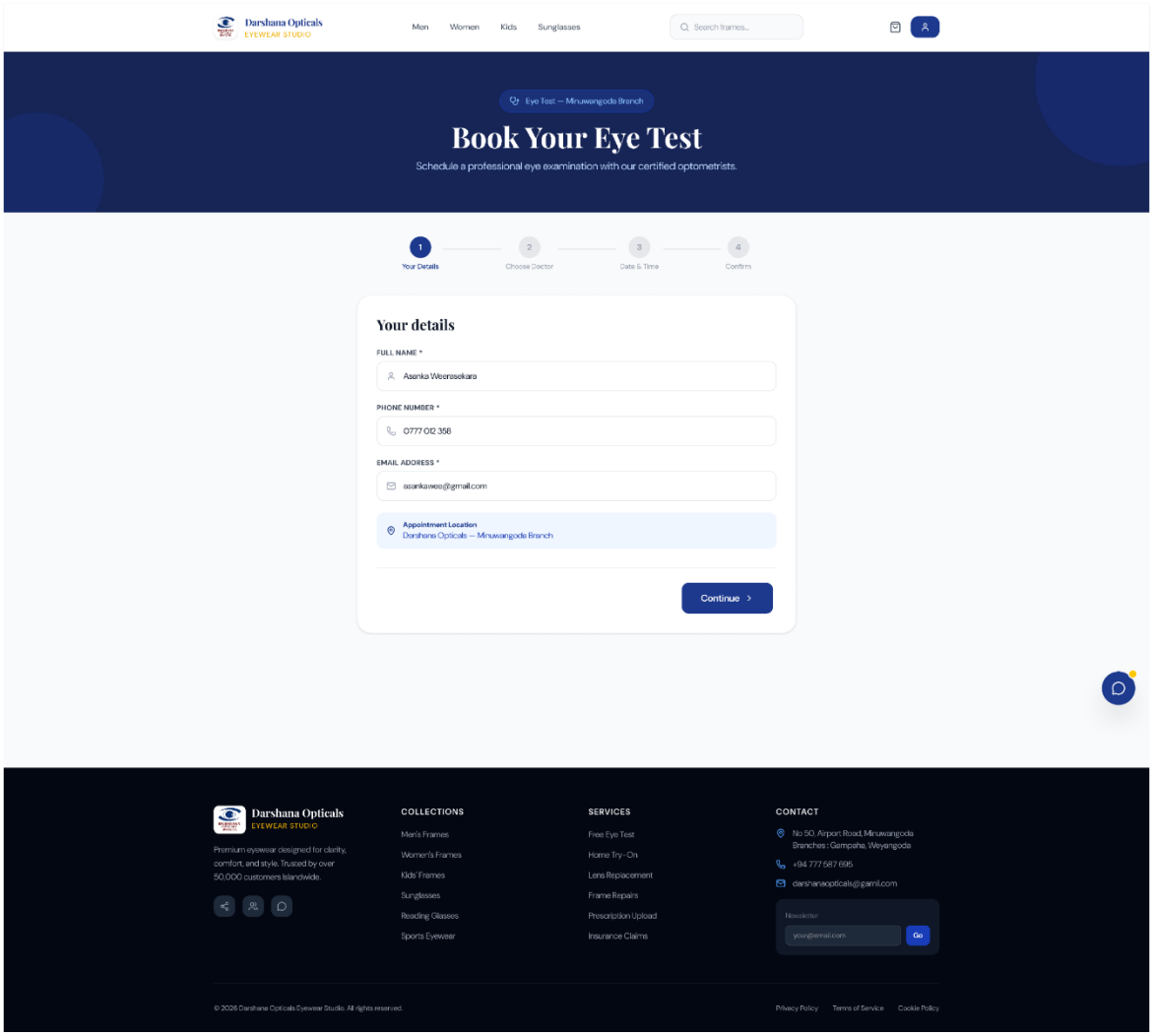


Figure 30: Patient Details Page

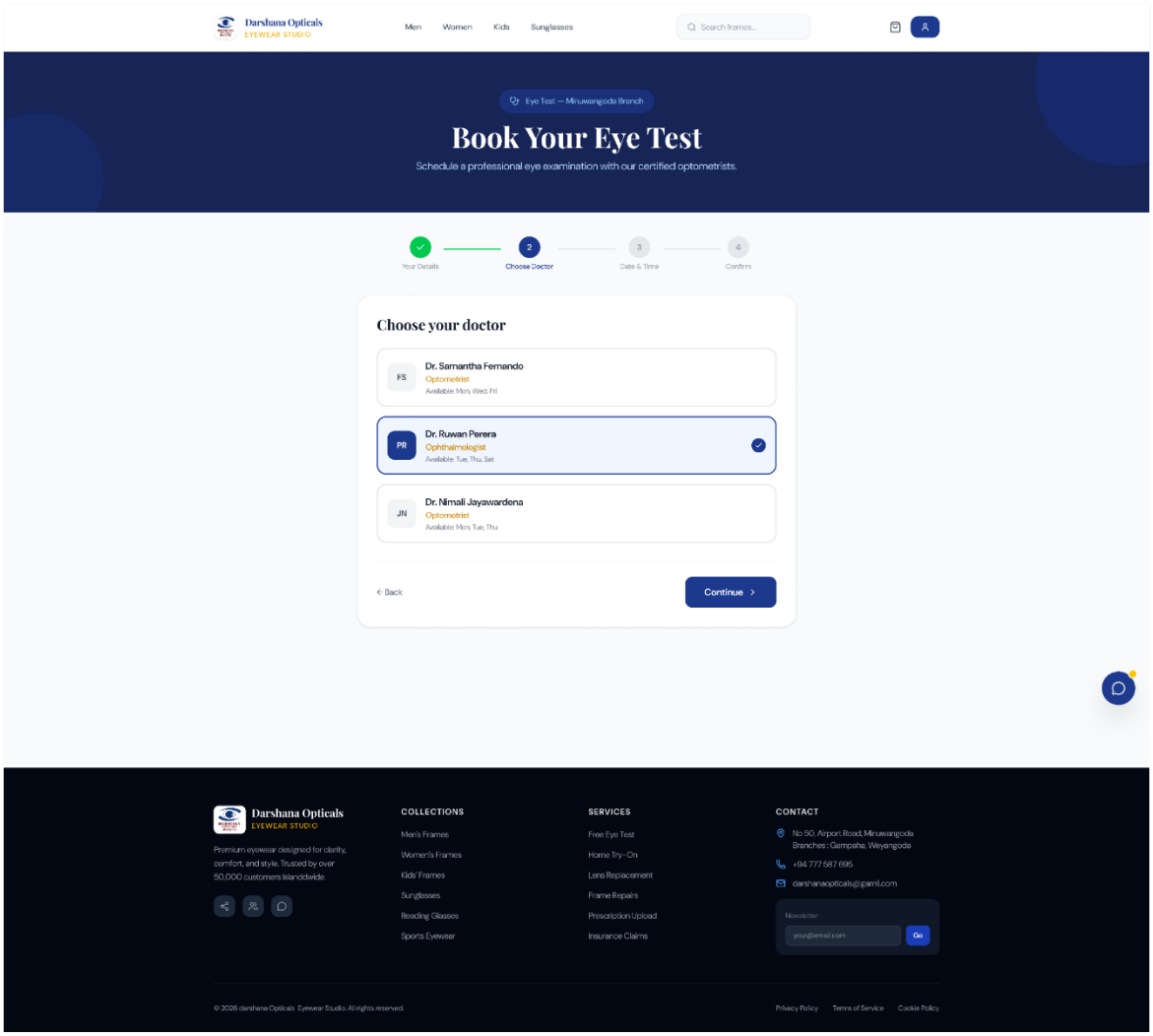


Figure 31: Doctor Selection Page

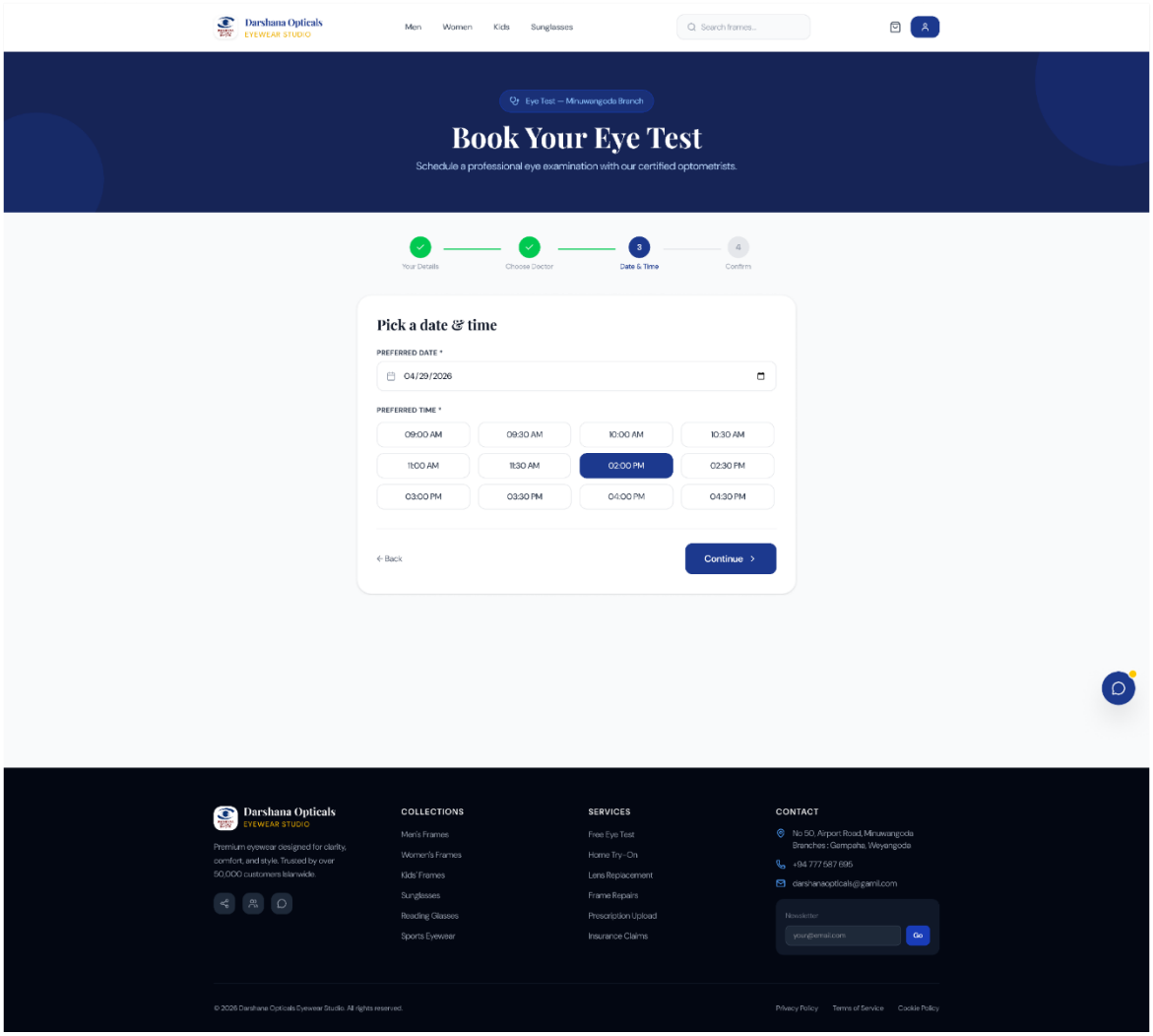


Figure 32: Appointment Scheduling Page

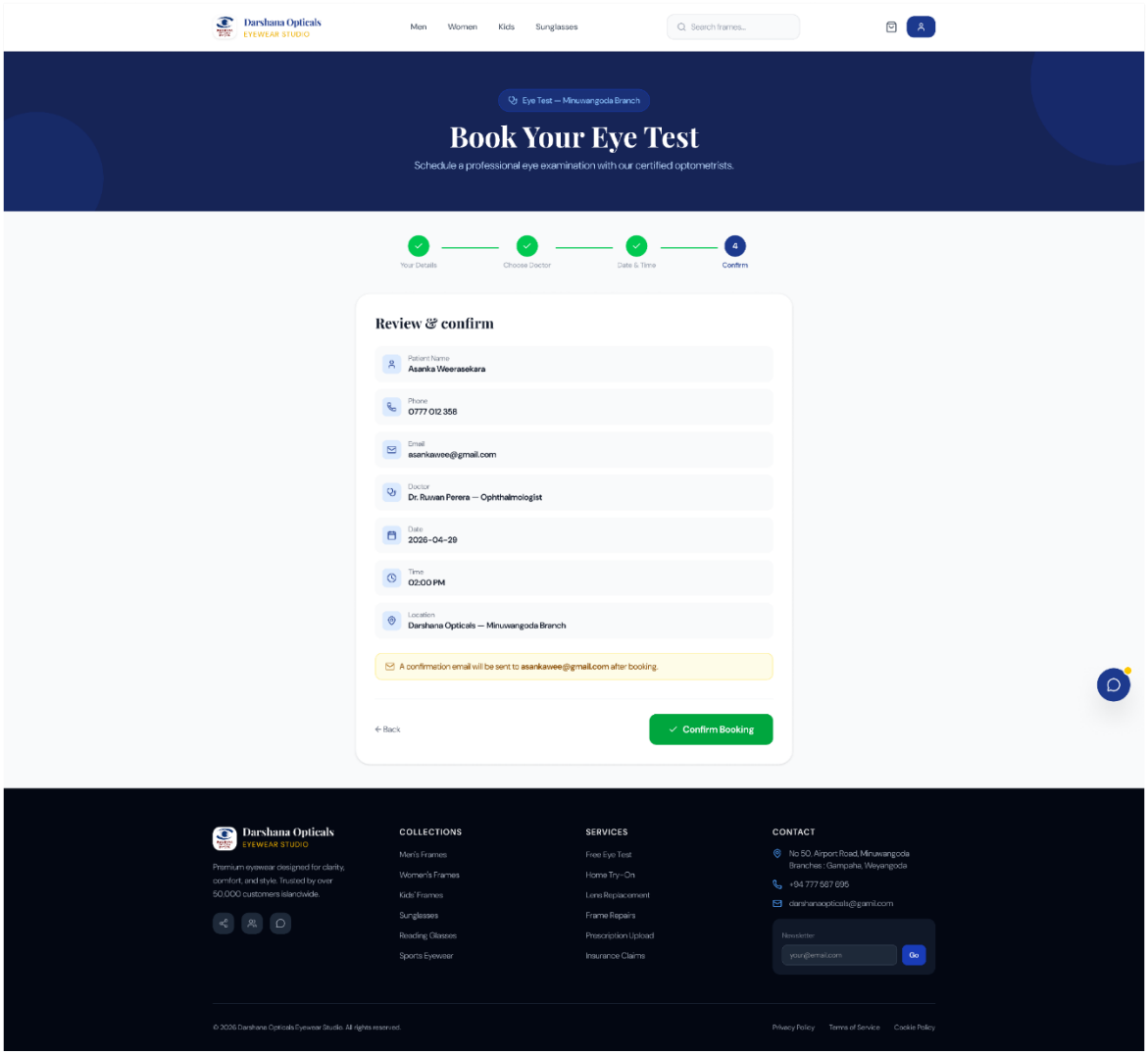


Figure 33: Appointment Review Page

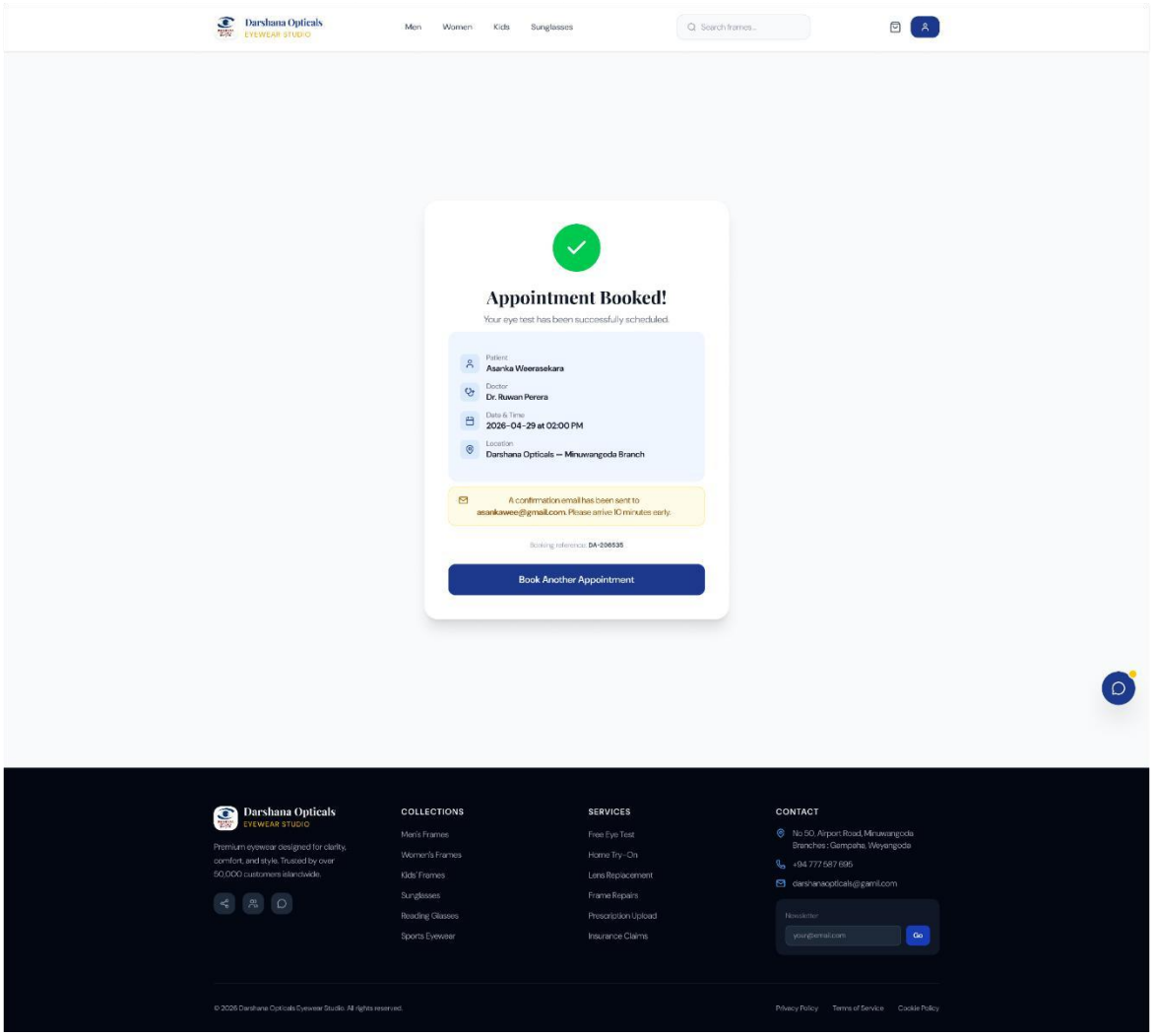


Figure 34: Appointment Confirmation Page

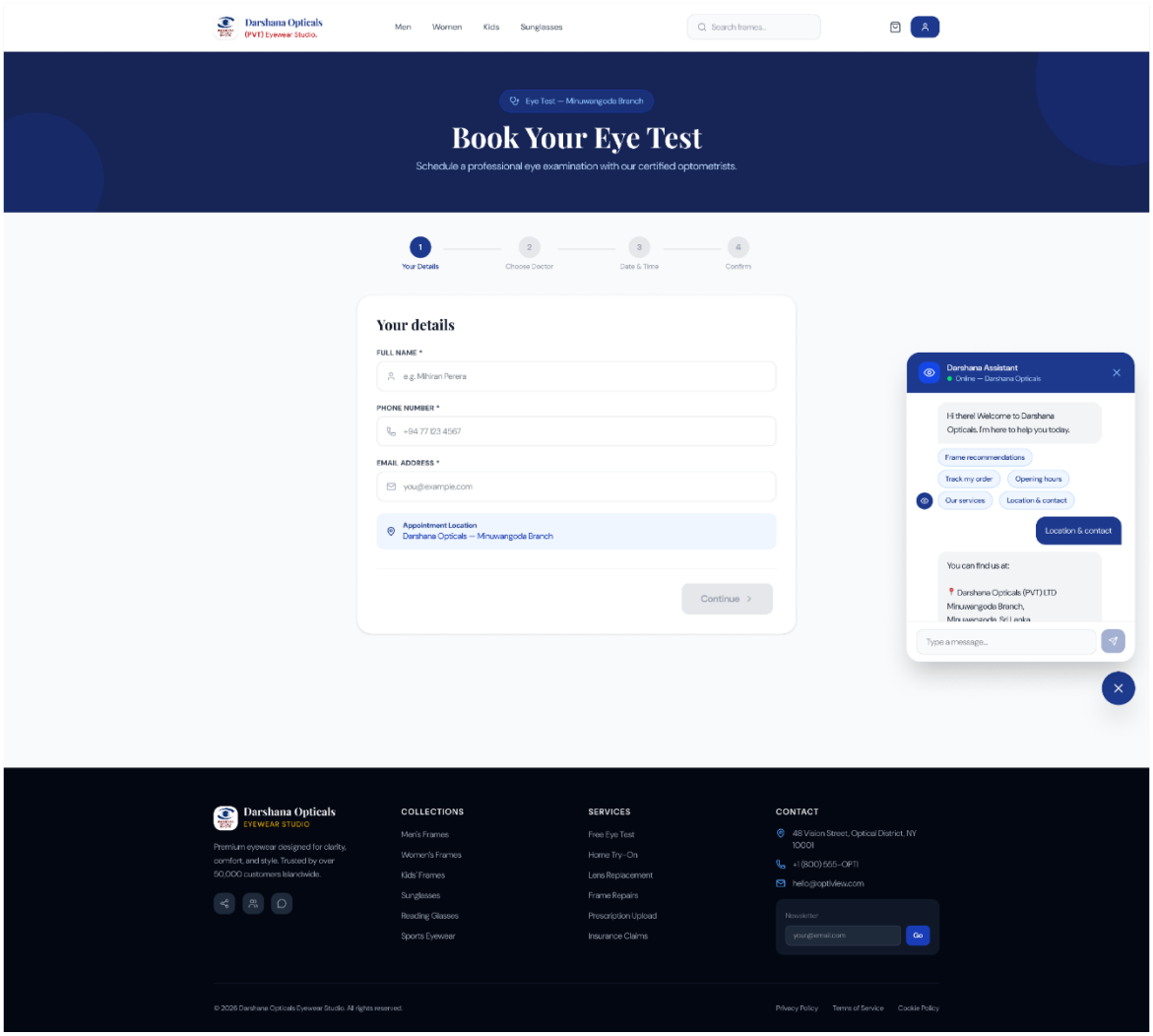


Figure 35: Chatbot Widget

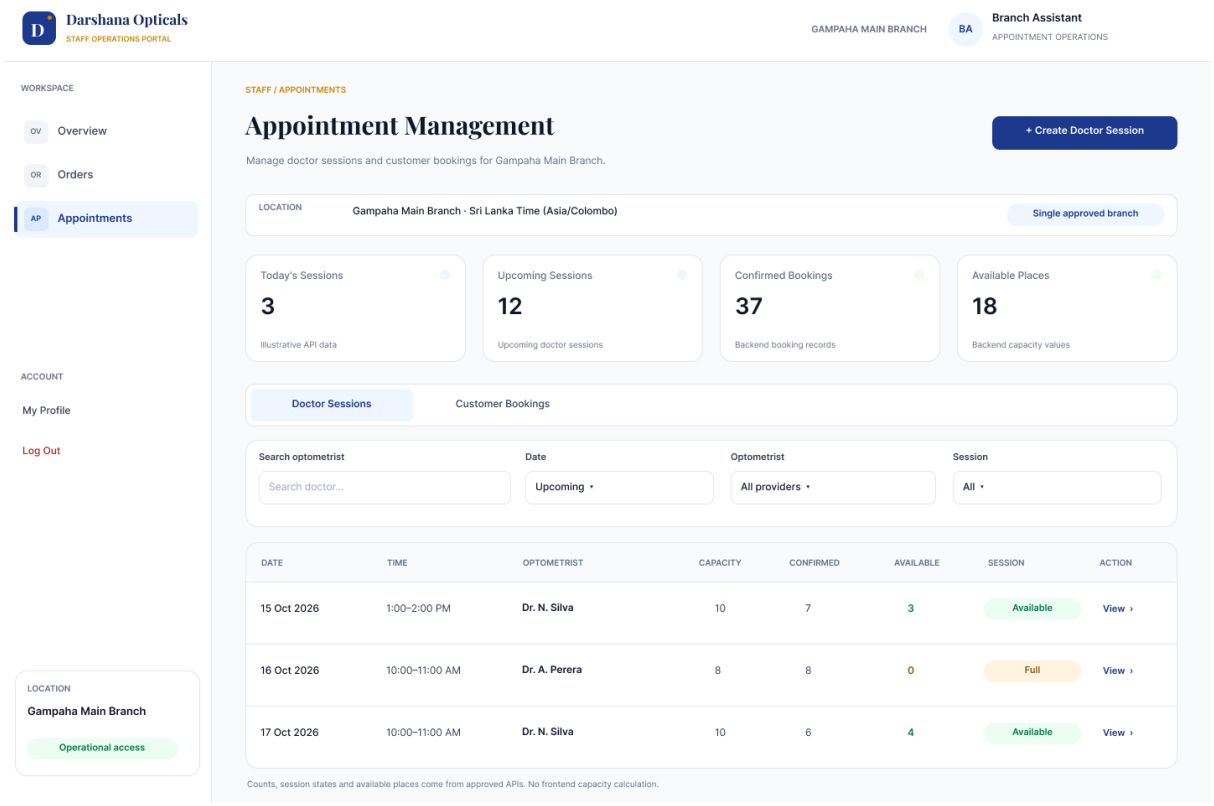


Figure 36: Doctor Sessions Overview

5.2 User flow / Navigation Diagrams

The user flow diagrams illustrate the main navigation paths, functional interactions, decision points, and possible outcomes within the Optical Shop Management System (OSMS).

The diagrams were updated to reflect the implemented React frontend, RESTful backend services, MongoDB persistence, approved user roles, and relevant Architectural Decision Records (ADRs).

The user flows cover both customer-facing services and internal operational functions performed by authorized branch staff, Optometrists, Branch Managers, and System Administrators.

5.2.1 Role-Based User Flow

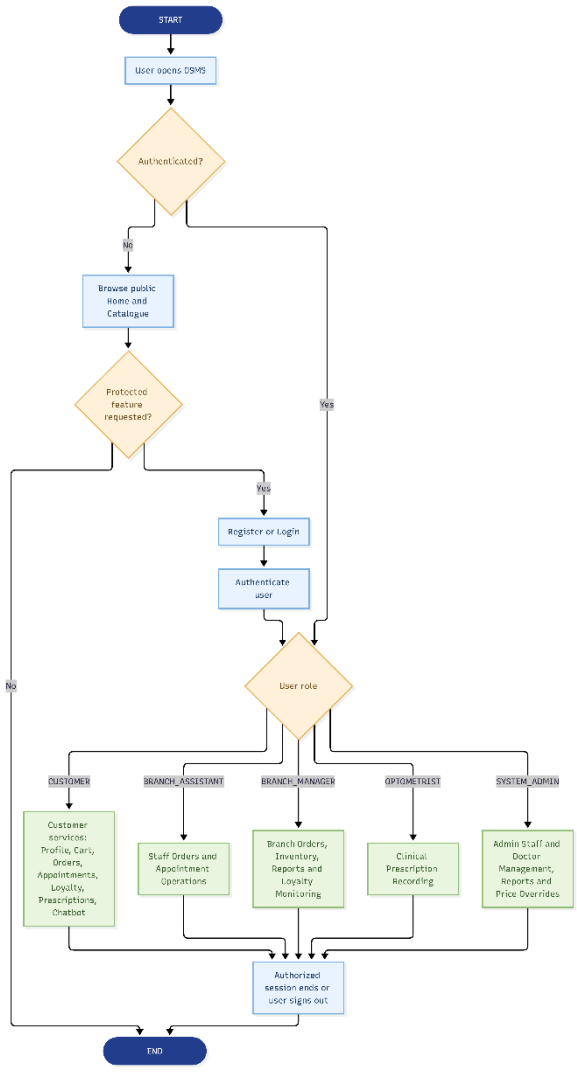


Figure 37: Role-Based User Flow

Illustrates public access to the OSMS, authentication requirements, and navigation according to the five implemented user roles. Protected functions are accessible only to appropriately authorized users.

5.2.2 Browse and Product Discovery Flow

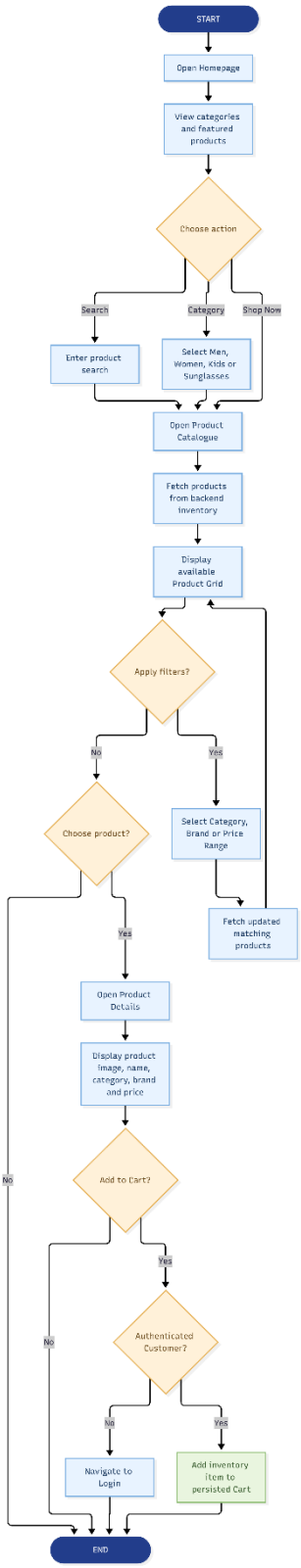


Figure 38: Browse and Product Discovery Flow

Illustrates product browsing, searching, category selection, brand and price filtering, product detail viewing, and authenticated Add to Cart functionality.

5.2.3 Cart Management Flow

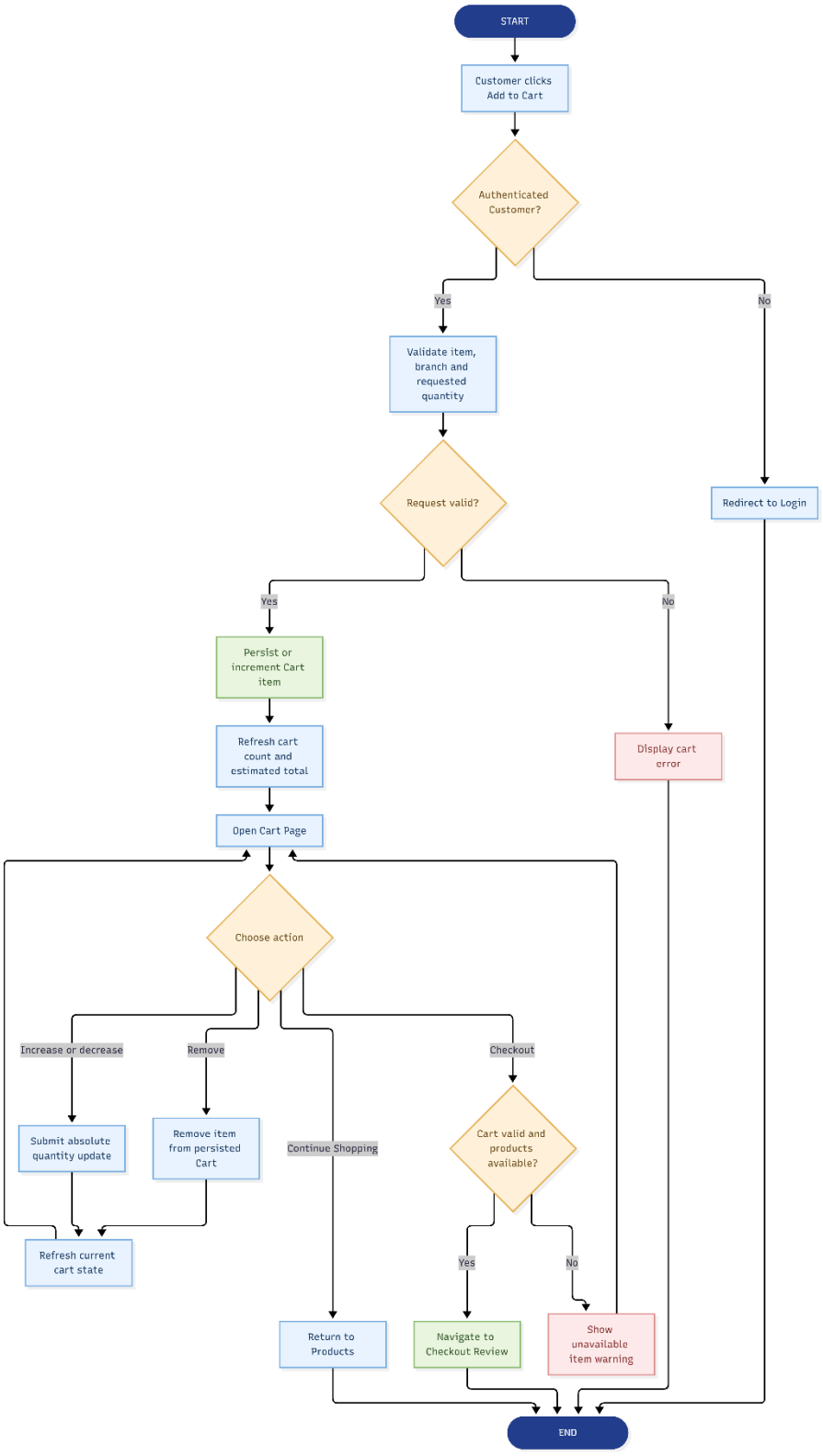


Figure 39: Cart Management Flow

Illustrates adding items to the persistent shopping cart, modifying quantities, removing items, viewing estimated totals, and proceeding to checkout.

5.2.4 Checkout and Order Confirmation Flow

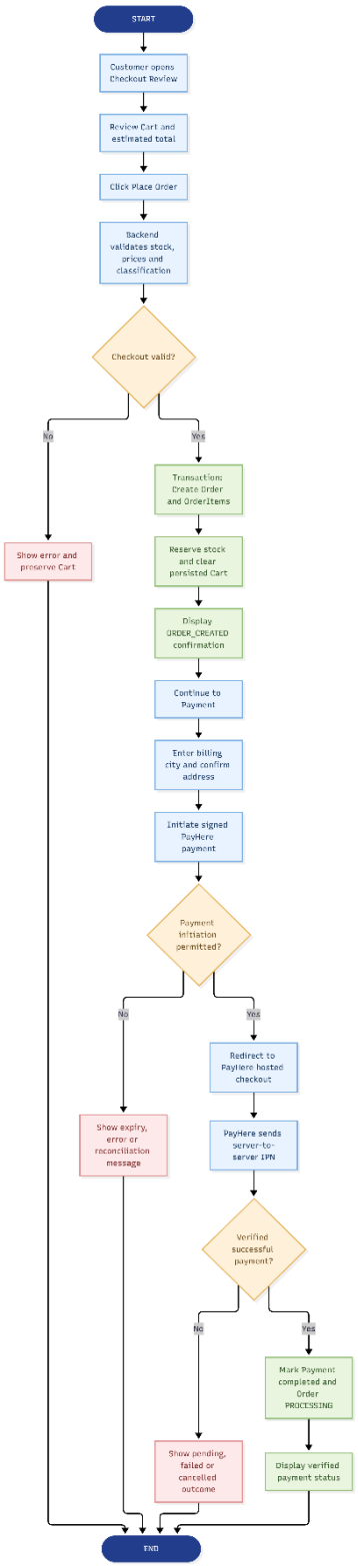


Figure 40: Checkout and Order Confirmation Flow

Illustrates backend-authorized Order creation, stock reservation, PayHere-hosted payment processing, verified payment confirmation, and payment error handling.

5.2.5 User Profile Management Flow

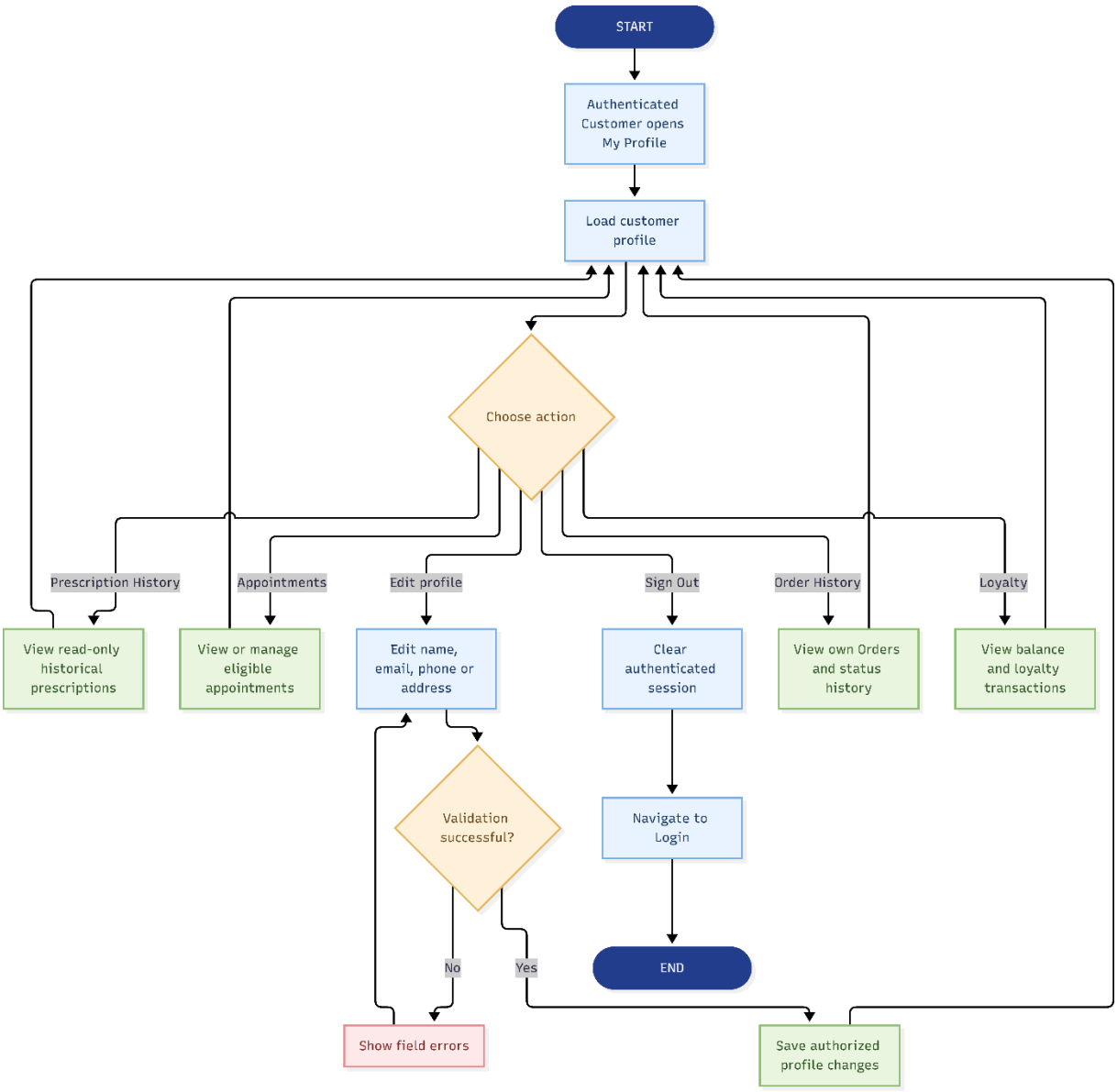


Figure 41: User Profile Management Flow

Illustrates customer profile editing and navigation to Order History, Prescription History, Loyalty, and Appointment History.

5.2.6 Authentication Flow

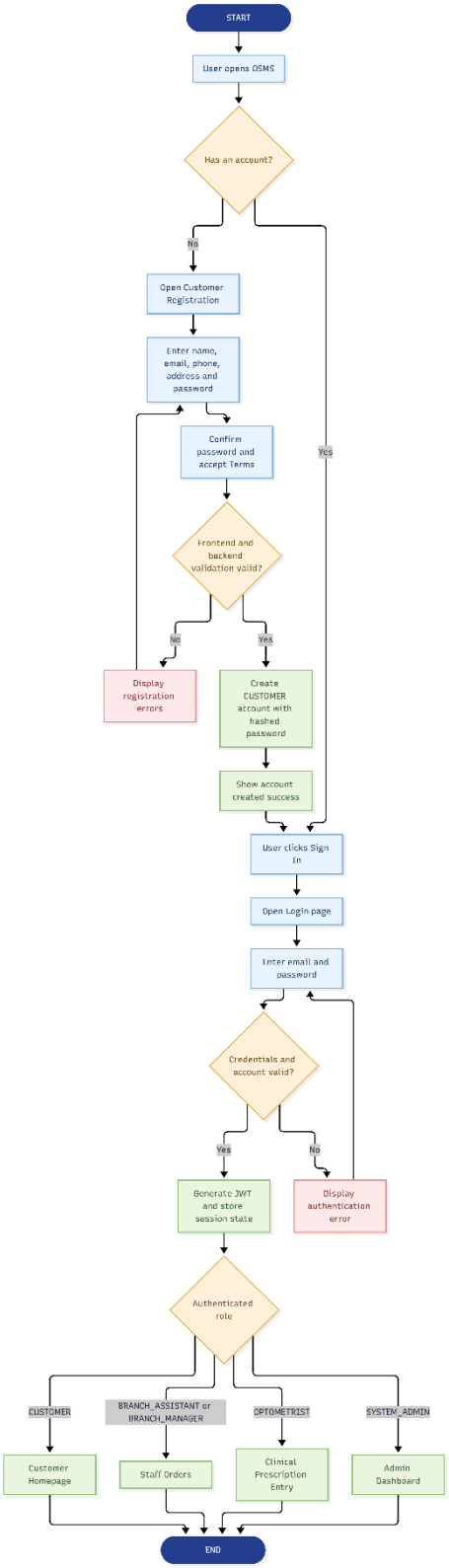


Figure 42: Authentication Flow

Illustrates customer registration, credential validation, JWT-based login, authentication errors, and role-specific navigation after successful login.

5.2.7 Appointment Booking Flow

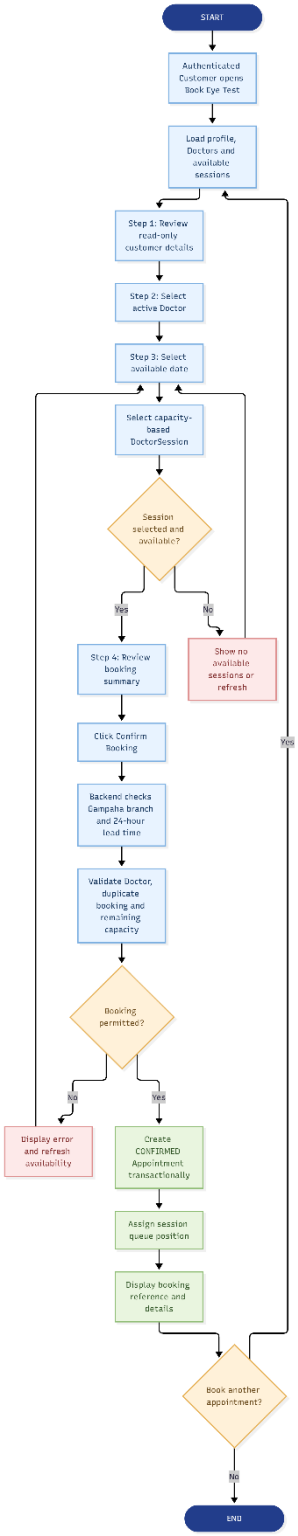


Figure 43: Appointment Booking Flow

Illustrates the four-step Doctor Session booking process, including customer profile retrieval, Doctor selection, available session selection, booking validation, and confirmation.

5.2.8 Chatbot Interaction Flow

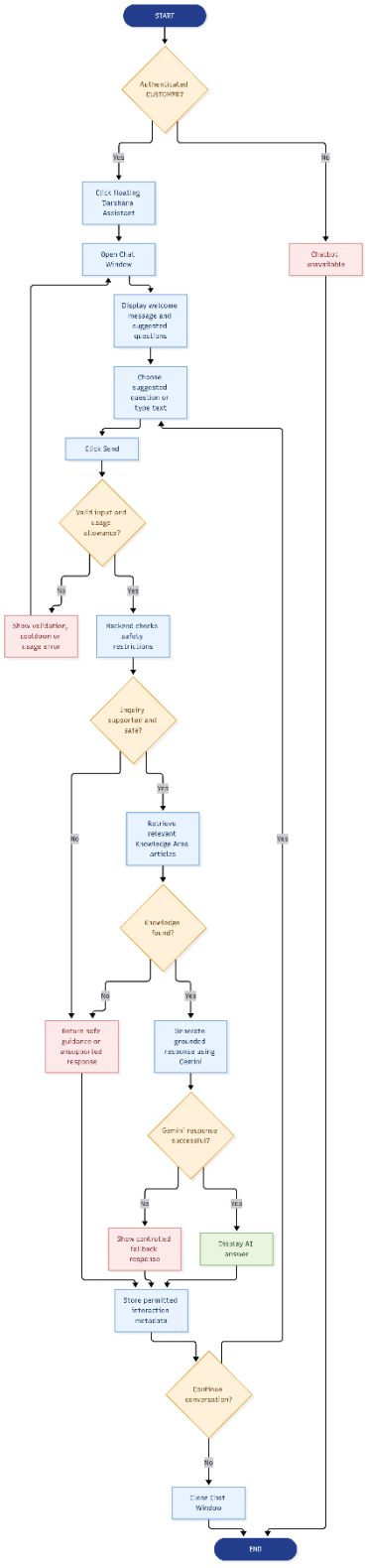


Figure 44: Chatbot Interaction Flow

Illustrates chatbot access, suggested questions, customer inquiries, usage-limit validation, Knowledge Area retrieval, Gemini AI response generation, and fallback handling.

5.2.9 Staff Order Processing and Tracking Flow

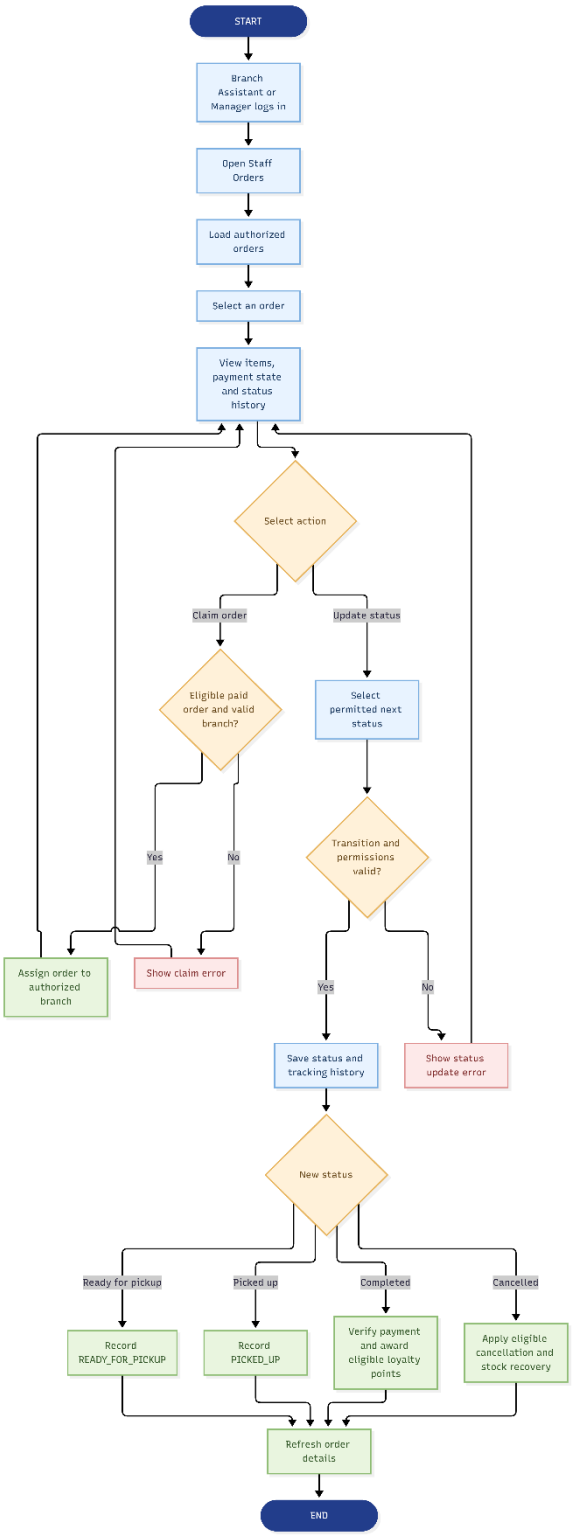


Figure 45: Staff Order Processing and Tracking Flow

Illustrates authorized staff access to customer orders, branch allocation, fulfillment status transitions, applicable cancellation handling, and loyalty point calculations after eligible Order completion.

5.2.10 Inventory Management and Reorder Alerts Flow

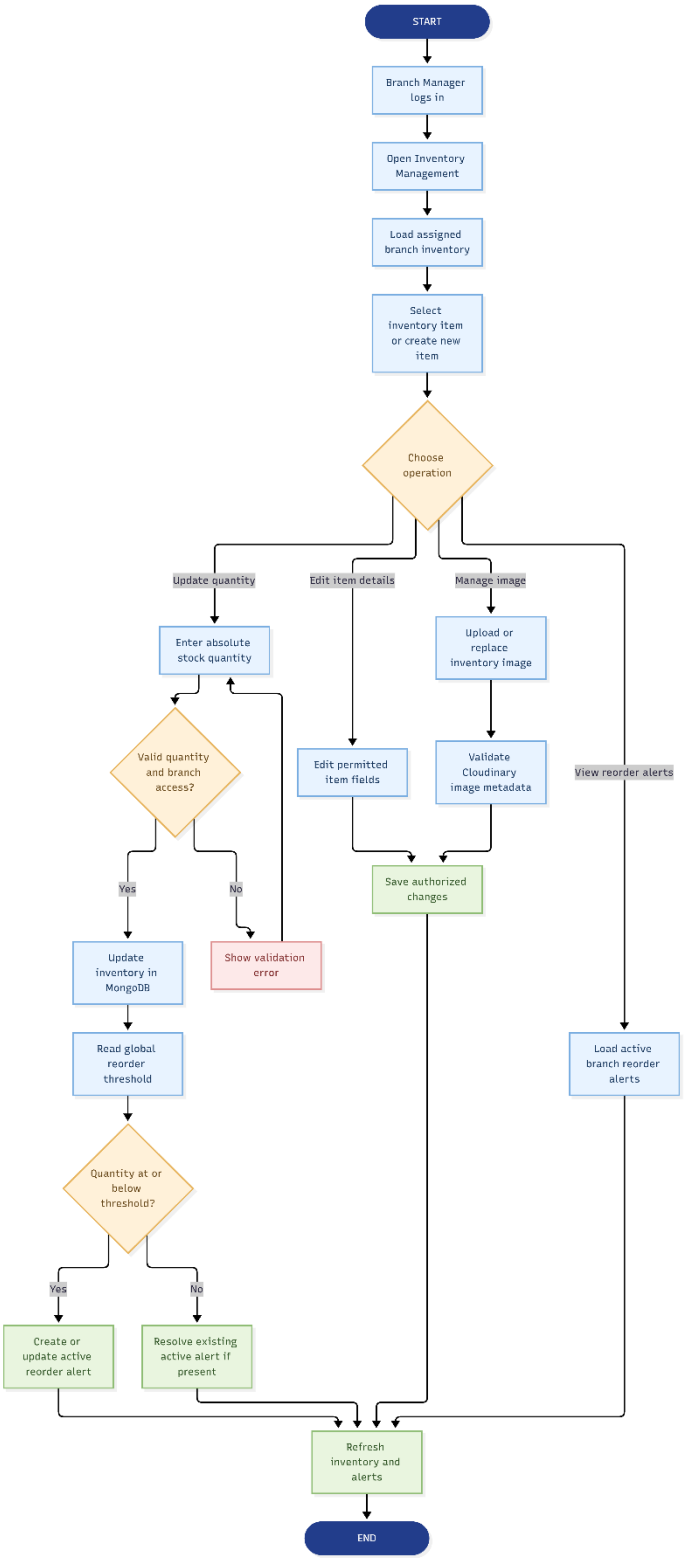


Figure 46: Inventory Management and Reorder Alerts Flow

Illustrates branch-specific inventory retrieval, stock updates, low-stock alert evaluation, inventory image management, and authorized inventory operations.

5.2.11 Doctor Session and Appointment Administration Flow

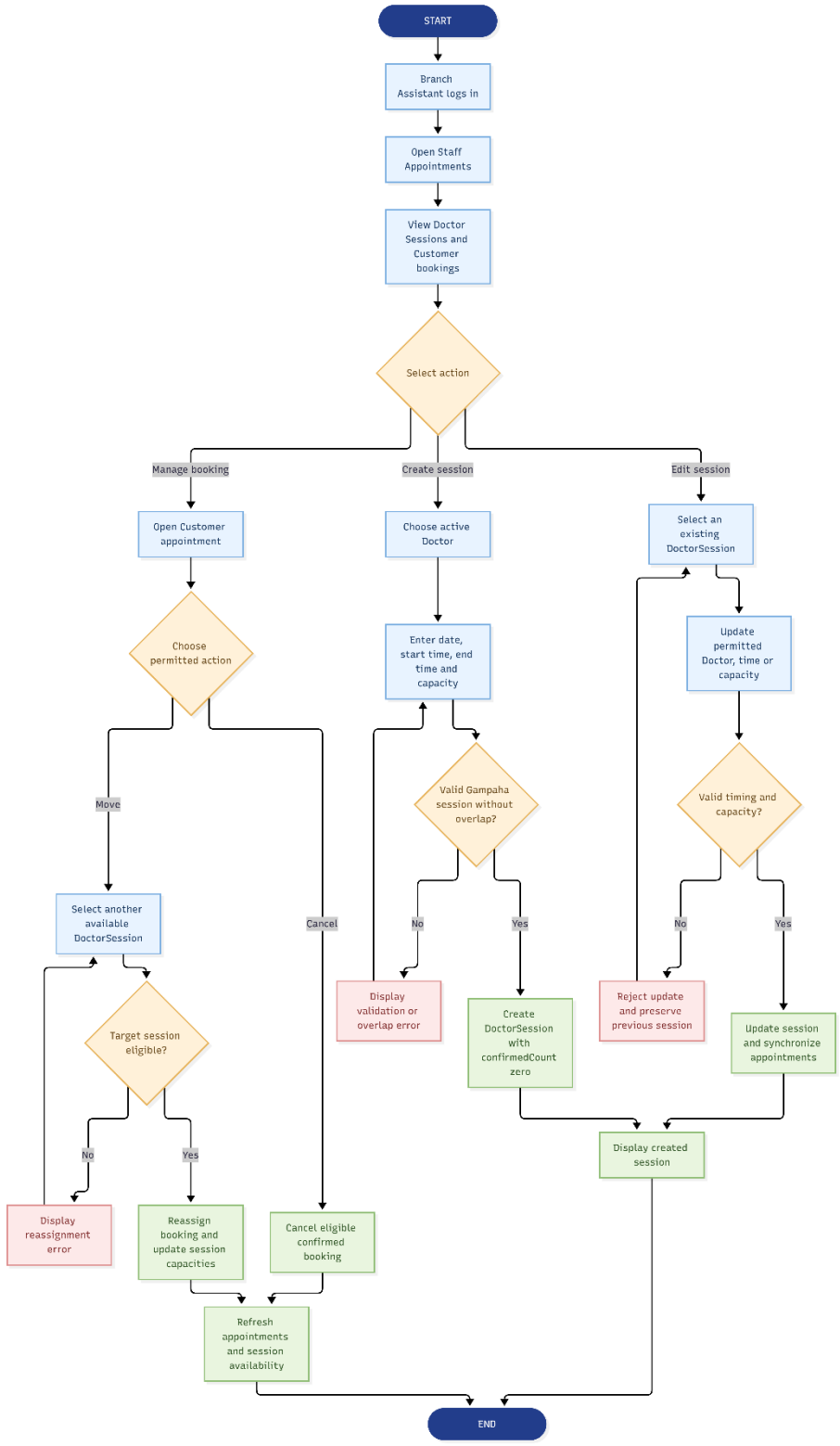


Figure 47: Doctor Session and Appointment Administration Flow

Illustrates Branch Assistant management of Doctor Sessions, including creation, updates, overlapping-session validation, appointment cancellation, and permitted booking reassignment.

5.2.12 Clinical Prescription Management Flow

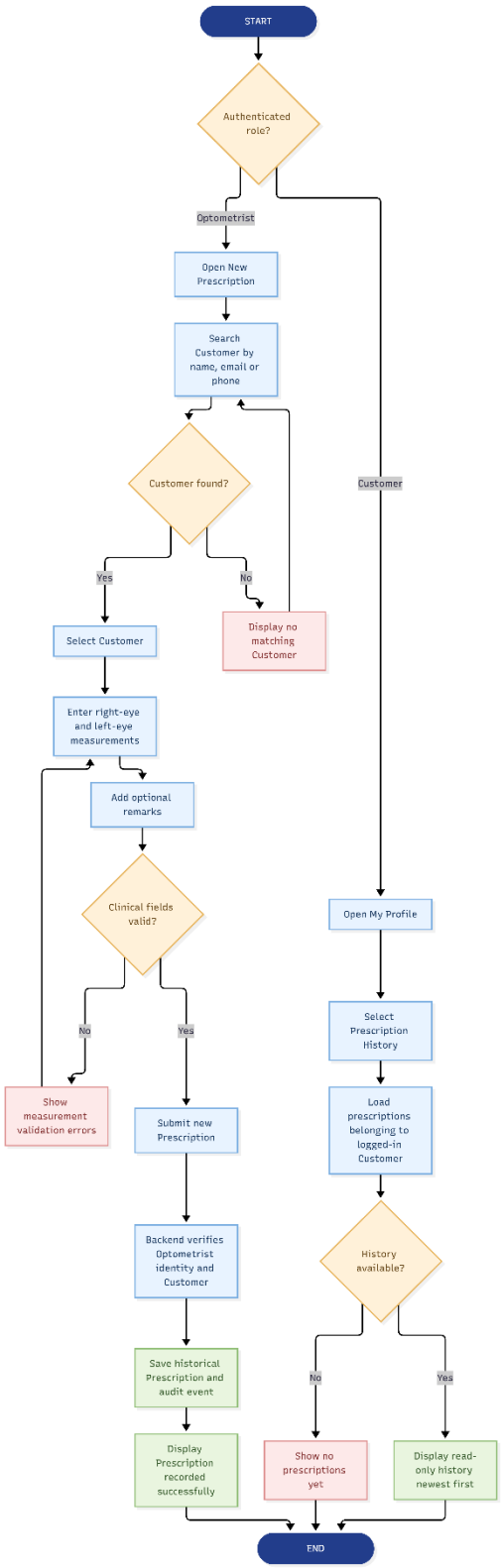


Figure 48: Clinical Prescription Management Flow

Illustrates Optometrist recording of clinical prescriptions and authenticated Customer access to read-only prescription history.

5.2.13 Admin Staff and Doctor Management Flow

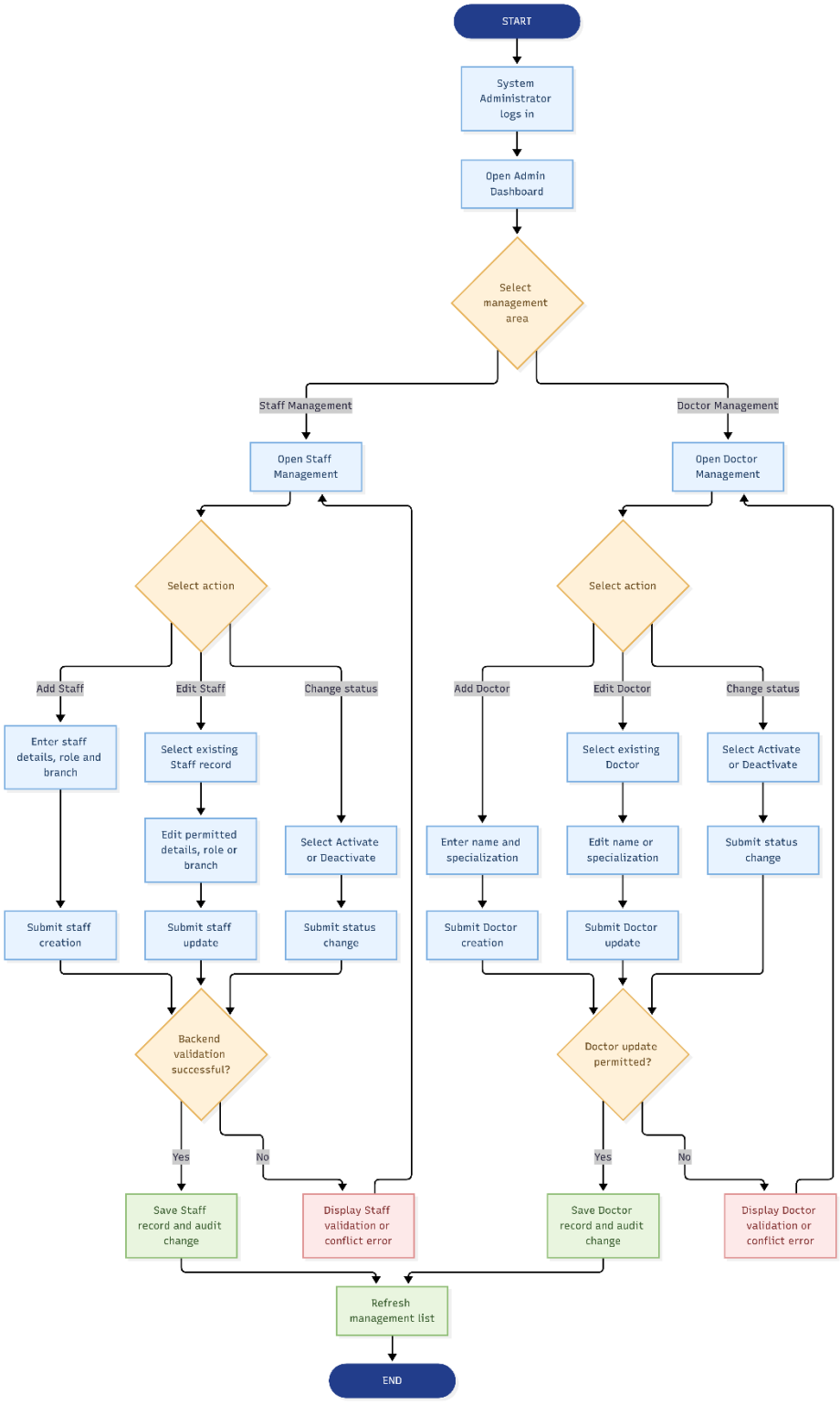


Figure 49: Admin Staff and Doctor Management Flow

Illustrates System Administrator activities for creating, updating, activating, and deactivating Staff and Doctor records under the approved access-control rules.

5.2.14 Sales and Inventory Reporting Flow

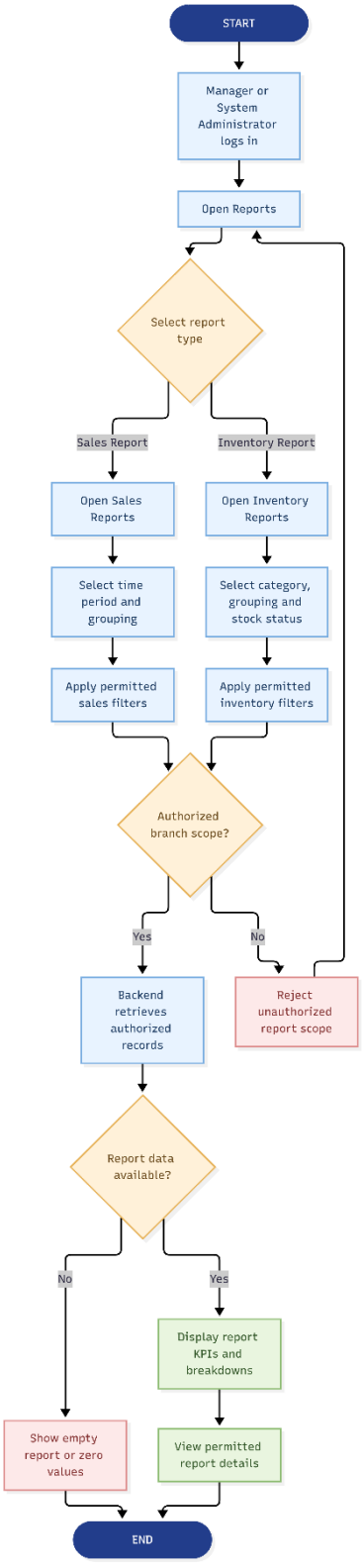


Figure 50: Sales and Inventory Reporting Flow

Illustrates authorized access to Sales and Inventory Reports, filtering, branch-based data restrictions, and report result presentation.

5.2.15 Loyalty Points and History Flow

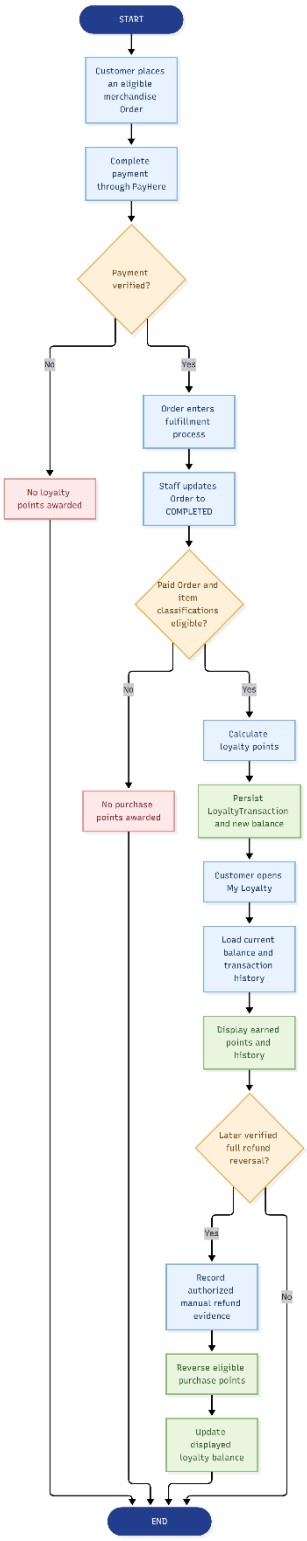


Figure 51: Loyalty Points and History Flow

Illustrates loyalty point eligibility, earning points after verified payment and completed fulfillment, customer balance and transaction-history viewing, and permitted reversal handling.

User Flow Design Considerations

The updated diagrams use the following conventions:

- **Activity Nodes:** Represent navigation actions, system operations, and processing steps.
- **Decision Nodes:** Represent validations, permissions, user choices, and alternative paths.
- **Success Nodes:** Represent successful operations or confirmed outcomes.
- **Error Nodes:** Represent validation failures, access restrictions, or unsuccessful operations.
- **Start and End Nodes:** Indicate the boundaries of the illustrated user flow.

The diagrams represent the implemented application behavior and do not imply support for features excluded from the final system, including product wishlists, direct card-entry payments, automated appointment reminders, automatic supplier purchase orders, or customer loyalty redemption.

5.3 Accessibility Considerations

The Optical Shop Management System (OSMS) incorporates accessibility considerations into its React-based user interface to support users with different abilities, technical skills, and levels of familiarity with web applications.

The system targets the Web Content Accessibility Guidelines (WCAG) 2.1 Level AA. Accessibility features are incorporated into major customer-facing and staff interfaces, including navigation, authentication, product discovery, shopping cart management, checkout, appointment booking, prescription management, and the AI chatbot.

The following considerations are organized according to the four WCAG principles: Perceivable, Operable, Understandable, and Robust.

1. Perceivable

Alternative Text for Images

- Product images use descriptive alternative text derived from the available product information, including the product name.
- Images that are unavailable are represented using accessible fallback labels where implemented.
- Decorative images and icons, including selected navigation icons, category illustrations, and background elements, use empty alternative text or aria-hidden attributes where appropriate.
- This approach helps assistive technologies distinguish meaningful content from decorative elements.

Use of Colour and Text

- Important information is presented using readable text labels rather than relying solely on colour.
- Order status indicators display descriptive status names such as PROCESSING, READY_FOR_PICKUP, PICKED_UP, and COMPLETED.
- Appointment booking progress indicators use numbered steps and descriptive labels to indicate the current stage.
- Form validation messages provide textual explanations of errors.
- Success and error messages communicate their meaning through text rather than colour alone.

Colour Contrast

- The interface uses the approved Darshana Opticals branding, including navy blue, white, and neutral background colours.
- The accessibility design targets a minimum contrast ratio of 4.5:1 for normal text and 3:1 for large text, in accordance with applicable WCAG 2.1 Level AA criteria.
- Interactive controls and focus indicators are designed to remain visually distinguishable from surrounding elements.

- Full colour-contrast conformance across all pages and component states requires dedicated accessibility testing before it can be confirmed.

2. Operable

Keyboard Navigation

- The application uses native interactive HTML elements, including buttons, links, text inputs, checkboxes, and form controls, to support keyboard interaction.
- Major operations such as navigation, searching, registration, login, cart management, and appointment selection use standard keyboard-operable controls.
- Product catalogue filters use selectable inputs for categories and brands, with numeric controls for minimum and maximum prices.
- Appointment booking uses selectable Doctor cards, available-session controls, and navigation buttons for the four-step booking process.
- Keyboard accessibility across all pages and interactive states remains subject to complete manual testing.

Visible Keyboard Focus

- Visible :focus-visible styles are defined for several major interface components, including navigation links, homepage controls, authentication buttons, appointment controls, and chatbot controls.
- Focus indicators help keyboard users identify the currently selected interactive element.
- Form inputs and controls use their native focus behaviour where appropriate.

Chatbot Keyboard and Focus Management

- The chatbot is available to authenticated Customers through a floating launcher button.
- The launcher includes an accessible label indicating whether it opens or closes the Darshana Assistant.
- When the chatbot opens, keyboard focus is moved to the message input field.
- The chatbot is presented as a modal dialog using appropriate dialog-related ARIA attributes.

- Background content is made inert while the chatbot dialog is open.
- Tab navigation is constrained to the chatbot's focusable controls while the dialog remains open.
- Pressing Escape closes the dialog, and focus is returned to the chatbot launcher.
- The floating launcher also supports documented keyboard-based positioning controls.

Responsive Navigation

- The Navbar includes a responsive account-menu control with an accessible label and aria-expanded state.
- Navigation links are provided according to the authenticated user's role.
- The interface adapts to smaller screen sizes through responsive layouts and CSS media queries.
- Complete focus-management behaviour for the mobile navigation menu requires additional verification.

3. Understandable

Clear Labels and Form Instructions

- Customer registration, login, profile management, payment initiation, and appointment booking forms use descriptive field labels.
- The reusable FormField component associates labels with their corresponding inputs.
- Required form inputs use native HTML validation-related attributes and visible required-field indicators where applicable.
- Input fields use relevant autocomplete attributes to assist users when entering personal and account information.

Validation and Error Feedback

- The reusable FormField component uses aria-invalid to identify inputs containing validation errors.
- Error messages are associated with affected inputs using aria-describedby and matching error identifiers.

- Validation messages use clear language to explain issues such as invalid email addresses, missing information, and password confirmation mismatches.
- The system displays appropriate messages for failed authentication, unavailable products, invalid cart operations, checkout failures, and appointment booking conflicts.
- Loading, success, and error indicators are used to communicate relevant application states.

Predictable Navigation and Workflows

- Major application sections use consistent navigation patterns and role-appropriate destinations.
- Customer-facing navigation separates product browsing, shopping cart management, order history, appointment booking, prescription history, and loyalty information.
- Protected functionality redirects unauthenticated users to the login page where applicable.
- The checkout process separates Order Review, Order Creation, PayHere payment initiation, and payment confirmation.
- Appointment booking follows four clearly identified steps:
    1. Your Details
    2. Choose Doctor
    3. Date & Time
    4. Confirm
- The active appointment step is identified using numbered progress indicators and aria-current="step".

Understandable Chatbot Interaction

- The Darshana Assistant presents a welcome message and five suggested questions: Frame Recommendations, Track My Order, Opening Hours, Our Services, and Location & Contact.
- Suggested questions populate the message input field and allow the Customer to review or edit the text before submitting.
- The chatbot uses internal Knowledge Area information and Gemini AI rather than an exclusively scripted response engine.
- Unsupported questions, usage restrictions, and temporary service failures produce appropriate explanatory responses.
- The chatbot does not provide automated clinical diagnoses or process payment-card credentials.

4. Robust

Semantic HTML Structure

- The frontend is implemented using React.js and standard HTML elements.
- The application uses semantic elements such as <nav>, <main>, <header>, <section>, <form>, <button>, and <label> where applicable.
- Relevant page headings and section labels provide structural information to assistive technologies.
- The product search interface uses a form with role="search" and an associated text label.

ARIA Attributes and Dynamic Feedback

- ARIA attributes are used in selected components to improve accessibility.
- Navigation controls use aria-label, aria-expanded, and aria-current where relevant.
- Form inputs use aria-invalid and aria-describedby to communicate validation states.
- Appointment Doctor and date-selection controls use aria-pressed to indicate selected states.
- The shopping cart quantity display includes an aria-live="polite" region.

- Checkout confirmation messages use status-related accessibility attributes.
- The chatbot conversation uses role="log" and aria-live="polite" to communicate newly added messages.
- Chatbot errors, processing indicators, and service notices use appropriate alert and status roles.

Role-Based Interface Accessibility

- Customer, Branch Assistant, Branch Manager, Optometrist, and System Administrator interfaces provide navigation according to their authorized responsibilities.
- Protected routes prevent users from accessing pages outside their assigned roles.
- Backend authorization independently enforces permission boundaries.
- Role restrictions are part of access control and do not replace the accessibility requirements of individual interfaces.

Additional Considerations

Responsive Design and Text Resizing

- The application includes responsive layout rules for desktop, tablet, and mobile screen sizes.
- Product grids, forms, navigation components, appointment booking interfaces, and chatbot layouts use responsive CSS techniques where implemented.
- WCAG 2.1 Success Criterion 1.4.4 targets text resizing up to 200% without loss of content or functionality.
- Full verification of 200% browser zoom, text resizing, content reflow, and horizontal scrolling behaviour has not yet been established across every implemented page.

Reduced Motion Support

- Selected components, including the homepage and chatbot, provide CSS rules using the prefers-reduced-motion: reduce media query.
- These rules reduce or disable relevant animations for users who request reduced motion through their operating system settings.
- Consistent reduced-motion behaviour across every animated component requires further verification.

Appointment Booking Accessibility

- The appointment booking interface uses labelled steps, visible status information, and native interactive controls.
- Customer details are populated from the authenticated profile and displayed as read-only information.
- Available Doctor Sessions are presented using selectable controls, with relevant session dates, times, and capacity information.
- The booking confirmation screen displays the selected Doctor, appointment time, approved Gampaha Main Branch location, booking reference, and confirmation status.
- Automated appointment confirmation emails and SMS reminders are not part of the current implementation.

External Payment Interface

- OSMS uses a PayHere-hosted payment interface.
- Cardholder credentials are not collected directly by the OSMS frontend.
- Accessibility conformance of PayHere's external payment interface must be evaluated separately from the OSMS application's own interface.

Accessibility Testing and Verification

The implementation contains accessibility-supporting design and coding practices. However, complete WCAG 2.1 Level AA conformance has not been independently established by the available verification evidence.

The following checks remain necessary before claiming full conformance:

- Automated accessibility analysis of representative application pages.
- Colour contrast verification for normal text, large text, and interactive components.
- Keyboard-only navigation and focus-order testing across all supported user roles.
- Screen-reader testing with appropriate assistive technologies.
- Browser zoom, 200% text resizing, and responsive reflow testing.
- Review of error announcements, dialog focus behaviour, and dynamic content updates.
- Verification of accessible interaction with the external payment provider.

Accessibility issues identified during these checks should be documented and addressed according to their severity and project acceptance requirements.

The system therefore targets WCAG 2.1 Level AA through its interface design and implementation practices, while full compliance remains subject to formal accessibility verification.

Chapter 6: Security Design

6.1 Authentication Mechanism

The Optical Shop Management System (OSMS) employs a token-based authentication mechanism using JSON Web Tokens (JWT) to authenticate users and protect backend API resources.

The React.js frontend communicates with the Node.js/Express.js backend through RESTful APIs. JWT authentication is primarily stateless, while additional database verification is performed for Staff accounts to enforce current role, branch assignment, and account status.

The system maintains separate Customer, Staff, and System Administrator identity models and applies Role-Based Access Control (RBAC) to protected operations.

A. Core Technology & Standards

1. **JWT (JSON Web Token):** Used to generate signed access tokens for authenticated users. Tokens contain the user's identifier (userId), authorized role (role), and optional branch identifier (branchId). The backend uses the HS256 signing algorithm.
2. **Bcrypt Password Hashing:** Passwords are hashed using the bcrypt library with 12 salt rounds before being stored in MongoDB. Plaintext passwords are not stored in user identity records.
3. **Authorization Bearer Header:** The frontend stores authentication tokens in browser localStorage and transmits them using the Authorization: Bearer <token> HTTP header for protected API requests. HTTP-only authentication cookies are not implemented in the current system.
4. **HTTPS/TLS Communication:** HTTPS is used in the deployed application to protect authentication credentials and tokens while they are transmitted between the frontend and backend. JWT signing provides token integrity and authenticity but does not encrypt the token contents.
5. **Environment-Based Secret Management:** The JWT signing secret is configured using the server-side JWT_SECRET environment variable. The backend requires this configuration in non-test environments and does not expose the secret to the frontend.

B. Authentication Workflow

1. **User Login:** A Customer, Staff member, or System Administrator enters their email address and password through the React login interface.
2. **Credential Submission:** The frontend submits the credentials to the backend through POST /api/auth/login.
3. **Identity Lookup:** The backend searches the Customer, Staff, and Admin collections in MongoDB for the matching account.
4. **Password Verification:** The authentication service compares the submitted password with the stored Bcrypt hash. It also checks the account's active status and applicable temporary lockout conditions.
5. **Token Generation:** After successful authentication, the backend generates a signed JWT containing the verified user identifier, role, and optional branch identifier.
6. **Token Transmission and Storage:** The backend returns the JWT and a safe user identity object. The frontend stores them in browser localStorage and updates the AuthProvider authentication context.
7. **Subsequent Requests:** The frontend includes the stored JWT in the Authorization header when accessing protected backend endpoints.
8. **Token Validation:** The backend authentication middleware validates the token's signature, expiry, supported signing algorithm, user identifier, and canonical role.
9. **Staff Account Revalidation:** For Staff identities, the backend additionally retrieves the current Staff record and verifies its role, branch assignment, and active status. This prevents reliance solely on outdated Staff role or branch information contained in a previously issued token.
10. **Authorization:** Role-specific middleware verifies whether the authenticated user has permission to access the requested API resource. Unauthorized and forbidden requests are rejected.

C. Session Management & Security

- **Token Expiry:** JWT access tokens have a default validity period of 24 hours. The duration is configurable through the JWT_EXPIRES_IN environment variable.
- **Logout Mechanism:** When a user logs out, the frontend removes the stored JWT and user information from localStorage and clears the AuthProvider session state. The current implementation does not maintain a server-side JWT revocation list, so logout does not independently invalidate an already issued token before its expiry.
- **Account Lockout:** The system temporarily locks an account after five consecutive unsuccessful login attempts, by default. The lockout duration is 15 minutes. The configured limits can be adjusted through environment variables.
- **Authentication Rate Limiting:** Authentication endpoints use an additional IP-based rate limiter to reduce automated login attempts. The default configuration permits ten authentication requests per 15-minute window, with successful requests excluded from the authentication limiter's count.
- **Password Recovery:** Eligible Customer and Staff accounts can request password reset links through the Forgot Password workflow. Reset tokens are cryptographically generated, stored as SHA-256 hashes, expire after 15 minutes, and are invalidated after successful use. Public password recovery is not available for System Administrator accounts.
- **Password Reset Validation:** New passwords must contain at least eight characters, including an uppercase letter, a lowercase letter, and a number. Successfully reset passwords are stored using Bcrypt hashing.
- **Authentication Error Handling:** Invalid credentials and account-lockout conditions use controlled error responses to reduce disclosure of sensitive account information. Authentication events are recorded using privacy-conscious operational logging.
- **Client-Side Token Security:** Because the frontend stores JWTs in localStorage, tokens may be exposed if an attacker successfully executes malicious JavaScript within the application's origin. The implementation therefore relies on complementary security controls, including input sanitization, HTTP security headers, restricted CORS configuration, HTTPS, and backend authorization checks.

- **Session Limitations:** A separate 30-minute inactivity-based automatic logout mechanism and server-side access-token revocation are not implemented in the current authentication architecture.

6.2 Authorization Levels

The Optical Shop Management System (OSMS) implements Role-Based Access Control (RBAC) to restrict access to protected resources according to the responsibilities and permissions of each authenticated user.

The implemented system defines five canonical roles: SYSTEM_ADMIN, BRANCH_MANAGER, BRANCH_ASSISTANT, OPTOMETRIST, and CUSTOMER.

Authorization is enforced through backend middleware and domain-specific access checks. Protected operations require a valid JWT, an approved user role, and additional ownership or branch-level authorization where applicable.

The authorization model follows the approved architectural decisions, including ADR-002 for role-based access control and ADR-006 for inventory reorder permissions.

A. Defined User Roles and Permissions

1. System Administrator (SYSTEM_ADMIN)

Primary Responsibility: Organization-level administration, staff and Doctor record management, and authorized system oversight.

Permissions:

- Create Staff accounts with approved roles.
- View and update Staff information.
- Reassign Staff roles and branch assignments.
- Activate and deactivate Staff accounts.
- Create, view, update, activate, and deactivate Doctor directory records.
- Access organization-wide Sales and Inventory Reports.
- View inventory records across branches.
- Perform price-only overrides on inventory records.

- View and update the organization-wide reorder threshold.
- View permitted inventory reorder alerts.

Restrictions:

- Cannot directly modify inventory stock quantities through the Branch Manager stock-management API.
- Does not have general Customer cart, checkout, appointment booking, or prescription-recording permissions.
- Does not have unrestricted access to clinical prescription data.
- Does not automatically inherit Branch Manager payment-monitoring permissions.

2. Branch Manager (BRANCH_MANAGER)

Primary Responsibility: Branch inventory control, order fulfillment, operational monitoring, and authorized financial oversight.

Permissions:

- Create and manage inventory records belonging to the assigned branch.
- Update stock quantities for the assigned branch.
- Upload and manage authorized inventory images.
- View low-stock alerts for the assigned branch.
- View and update the organization-wide reorder threshold.
- Access Sales and Inventory Reports restricted to the assigned branch.
- View and manage permitted branch Orders and fulfillment statuses.
- View authorized Doctor Sessions and Appointments.
- Monitor loyalty transactions through the approved read-only management interface.
- Access approved payment monitoring, payment event, reconciliation-evidence, and external full-refund evidence workflows.

Restrictions:

- Cannot modify inventory records belonging to another branch.
- Cannot create or modify Doctor Sessions.
- Cannot create clinical prescription records.
- Cannot arbitrarily award, redeem, or adjust Customer loyalty points.
- Financial reporting permissions are subject to approved metric restrictions and data masking.
- Payment-monitoring and refund-evidence functions do not grant permission to fabricate payment confirmations or execute automatic gateway refunds.

3. Branch Assistant (BRANCH_ASSISTANT)

Primary Responsibility: Customer order fulfillment and branch appointment administration.

Permissions:

- View authorized Staff Orders and Order details.
- Claim eligible Orders for the assigned branch.
- Update permitted Order fulfillment statuses.
- Create Doctor Sessions for the approved Gampaha branch.
- Update authorized Doctor Sessions.
- View permitted Appointments and Doctor Session information.
- Cancel or reassign eligible Customer Appointments.
- Access relevant Doctor directory information needed for appointment operations.
- Communicate appointment information to Customers manually.

Restrictions:

- Cannot perform Branch Manager inventory stock-management operations.
- Cannot access Branch Manager financial reports or payment-monitoring APIs.
- Cannot create or modify clinical prescription records.
- Cannot manage Staff accounts or the Doctor directory.
- Manual appointment communication does not include automated SMS or email reminders.

4. Optometrist (OPTOMETRIST)

Primary Responsibility: Clinical prescription recording and authorized access to Customer prescription history.

Permissions:

- Search Customer records through the approved clinical lookup interface.
- Create clinical prescription records.
- View authorized Customer prescription history.
- Access permitted Doctor directory reference information.

Restrictions:

- Cannot modify or delete historical prescriptions through the current clinical API.
- Cannot access arbitrary Customer clinical records outside the permitted clinical workflow.
- Cannot manage Doctor Sessions or Customer appointment scheduling.
- Cannot manage inventory quantities, payment records, or administrative accounts.
- Does not have general Branch Assistant or Branch Manager permissions.

5. Customer (CUSTOMER)

Primary Responsibility: Personal account management, eyewear purchasing, appointment booking, and access to authorized customer services.

Permissions:

- Register a new Customer account and authenticate through the login interface.
- View and update their own Customer profile.
- Browse and search the public eyewear catalogue.
- Manage their own shopping cart.
- Place Orders and initiate secure PayHere payments.
- View their own Order history and fulfillment status.
- Book available Doctor Sessions for eye examinations.
- View and cancel their own eligible Appointments.
- View their own clinical prescription history.
- View their own loyalty balance and transaction history.
- Access the authenticated AI chatbot.

Restrictions:

- Cannot create Staff or administrative accounts.
- Cannot access another Customer's private records.
- Cannot modify inventory stock quantities or system prices.
- Cannot create, modify, or delete clinical prescription records.
- Cannot manage Doctor Sessions or staff-controlled fulfillment statuses.
- Cannot redeem loyalty points through the current implementation.

Non-Authenticated Business Stakeholders

Business Owners, Management representatives, and Doctor directory records are relevant stakeholders but are not separate authenticated roles within the implemented RBAC model.

A business owner who requires authorized system access must use an appropriately provisioned account with one of the supported canonical roles. Doctor records do not independently provide login access.

B. Updated Access Control Matrix

Table 12: Updated Access Control Matrix Table

Requirement ID	Functional Requirement	System Admin	Branch Assistant	Branch Manager	Optometrist	Customer
FR-001	User registration and authentication	M/U	U	U	U	R/U
FR-002	Profile management and prescription history	—	—	—	V*	M/V*
FR-003	Digital product catalogue and filters	U	U	U	U	U
FR-004	Shopping cart and checkout	—	—	—	—	M/U

FR-005	Branch inventory and stock synchronization	V/P	—	M*	—	—
FR-006	Automated reorder alerts and threshold configuration	V/T	—	V*/T	—	—
FR-007	Doctor Session appointment booking and management	—	M*	V*	—	U/V*
FR-008	Appointment communication (manual; automated reminders excluded)	—	Manual*	—	—	Receive*
FR-009	Secure payment gateway processing	—	—	V/L	—	U

FR-010	Loyalty point calculation and updates	—	—	V/L	—	V*
FR-011	AI-driven chatbot assistance	—	—	—	—	U
FR-012	Sales and inventory reports	V	—	V*	—	—
FR-013	Clinical prescription recording and history	—	—	—	M/V*	V*
FR-014	Order fulfillment status tracking	—	M*	M*	—	V*

Legend for Access Control Matrix

- **M — Manage:** Permission to create, update, or perform authorized management operations within the specified scope.
- **V — View:** Read-only access to permitted records.
- **U — Use:** Permission to access or execute an implemented feature.
- **R — Register:** Permission to create a new public Customer account.
- **P — Price Override:** Restricted permission to update inventory prices without general stock-management authority.
- **T — Threshold Configuration:** Permission to view and update the organization-wide inventory reorder threshold.

- **L — Limited Operation:** Restricted operation permitted by a specific business rule or backend API.
- **— — No Access:** No permission to perform the specified protected function.
- *** — Scope Restriction:** The permission applies only to authorized Customer-owned, branch-specific, clinical, or otherwise restricted records.
- **Manual / Receive:** Represents human-managed appointment communication rather than an automated system permission.

Access Control Matrix Clarifications

FR-001 — Registration and Authentication

Public registration is restricted to creation of Customer accounts. System Administrators manage Staff identities through separate administrative operations. All five canonical roles can authenticate using the approved login workflow.

FR-002 — Customer Profile and Prescription History

Customers may manage their own profile and view their own clinical history. Optometrists may view authorized Customer prescription history through clinical endpoints. Optometrists do not receive general permission to manage Customer profiles.

FR-003 — Public Catalogue

Product browsing, searching, and supported catalogue filtering are publicly accessible. The matrix indicates that authenticated users of any role can also access these public features. Product catalogue visibility does not imply inventory modification permission.

FR-005 — Inventory Management

Branch Managers can manage inventory belonging to their assigned branch. System Administrators can view inventory records and perform price-only overrides. Dedicated stock-management operations remain restricted to authorized Branch Managers.

FR-006 — Reorder Alerts

Branch Managers view active reorder alerts within their assigned branch. System Administrators can view permitted reorder alerts across branches. Both roles can view and update the organization-wide reorder threshold through the approved backend endpoints.

FR-007 — Appointment Scheduling

Customers create and manage their own eligible bookings. Branch Assistants manage Doctor Sessions and perform permitted appointment administration. Branch Managers have restricted appointment and session viewing access.

FR-008 — Appointment Communication

Automated appointment reminders are excluded under ADR-009. Authorized Branch Assistants are responsible for manual Customer communication. This process is not represented as an automated reminder API permission.

FR-009 — Payment Processing

Customers initiate payments for their own Orders through PayHere. Branch Managers have restricted payment-monitoring and financial-evidence permissions. Successful payment completion depends on verified provider notifications and is not performed by an authenticated staff role through a general payment-status update.

FR-010 — Loyalty Management

Eligible loyalty points are awarded by backend business logic after verified payment and successful Order completion. Customers can view their own loyalty records. Branch Managers can monitor permitted loyalty transaction information and perform narrowly defined external full-refund evidence operations that may result in compensating point reversals. Manual arbitrary point adjustments and loyalty redemption are not implemented.

FR-012 — Reporting

System Administrators can access organization-wide reporting. Branch Managers are restricted to their assigned branch, with sensitive financial metrics excluded from their permitted report responses where required.

FR-013 — Prescription Management

Optometrists create prescription records and view authorized clinical history. Customers have read-only access to their own prescriptions. Other roles do not gain general clinical record access through the role hierarchy.

FR-014 — Order Tracking

Customers can view their own Order status and history. Authorized Branch Assistants and Branch Managers can update eligible fulfillment statuses for permitted Orders. This function concerns Order lifecycle tracking and does not include third-party courier integration.

Authorization Enforcement

The backend implements authorization through several complementary mechanisms:

1. **JWT Authentication:** Protected requests require a valid signed JWT.
2. **Canonical Role Validation:** The backend recognizes only the five approved roles.
3. **Role Authorization Middleware:** The authorizeRoles() middleware restricts access to designated API operations.
4. **Branch-Level Restrictions:** Inventory, Order, Appointment, and Reporting operations apply additional branch-related authorization checks where required.
5. **Customer Ownership Validation:** Customer-facing APIs restrict access to records associated with the authenticated Customer.
6. **Clinical Access Restrictions:** Prescription creation and clinical-history access follow separate permissions for Optometrists and Customers.
7. **Field-Level Restrictions:** Certain operations restrict the fields that an otherwise authorized role may modify, including System Administrator price-only inventory overrides.
8. **Backend Business Validation:** Payment, Order, loyalty, and appointment workflows enforce their own eligibility and lifecycle rules in addition to RBAC.
9. **Staff Account Revalidation:** Current Staff role, branch assignment, and active status are verified against the database for protected requests.

The implemented authorization design therefore follows both role-based and resource-specific restrictions, ensuring that access is granted according to approved operational responsibilities rather than authentication status alone.

6.3 Data Protection Approach

The Optical Shop Management System (OSMS) applies data protection measures to maintain the Confidentiality, Integrity, and Availability (CIA) of customer information, clinical prescriptions, inventory records, payment information, and other operational data.

The data protection design follows the principle of least privilege, role-based authorization, secure communication, data minimization, audit logging, and database integrity controls.

The system is designed with consideration for Sri Lanka's Personal Data Protection Act (PDPA) and relevant security requirements. However, complete legal and regulatory compliance cannot be claimed solely from the implemented technical controls and remains subject to operational and compliance verification.

A. Core Protection Strategies

1. Principle of Least Privilege (PoLP)

- Access to sensitive information is restricted according to the five implemented roles: SYSTEM_ADMIN, BRANCH_MANAGER, BRANCH_ASSISTANT, OPTOMETRIST, and CUSTOMER.
- Backend authentication and authorization middleware enforce role-specific access permissions.
- Customers can access their own profiles, Orders, Appointments, loyalty information, and prescription histories.
- Optometrists can record prescriptions and view Customer clinical history through authorized clinical workflows.
- Branch Managers are restricted to their assigned branch for inventory management and applicable operational reporting.
- System Administrators can perform approved administrative operations without automatically receiving unrestricted access to clinical records.
- Customer ownership, branch assignments, and field-level restrictions provide additional authorization beyond the authenticated role.

2. Data Minimization, Masking, and Restricted Disclosure

- Application APIs return information relevant to the requesting user's approved operation.
- Customer-facing APIs enforce ownership restrictions to prevent access to other Customers' private records.
- Sales and Inventory Reports apply role-specific access restrictions and branch scoping.
- Selected financial metrics are excluded from Branch Manager report responses according to ADR-014.
- Operational and audit logging mechanisms apply sanitization and redaction controls to reduce unnecessary exposure of Personally Identifiable Information (PII).
- Passwords, authentication tokens, payment credentials, and raw clinical measurements must not be copied into audit metadata.
- Data minimization and selected masking controls are implemented; comprehensive anonymization of all stored or exported data has not been independently verified.

3. Secure Audit Trails

- OSMS implements a centralized audit trail for approved critical inventory, financial, clinical, and administrative operations.
- The AuditEvent Mongoose schema includes application-level immutability controls that prevent normal update and delete operations on existing audit records.
- SHA-256 hash chaining provides tamper-evidence by linking consecutive audit events.
- Audit events maintain relevant metadata, including the actor, action, target resource, timestamp, and integrity information.
- Audit metadata is sanitized to avoid unnecessary storage of sensitive information.
- Critical business operations use transactional audit persistence where required by their approved consistency boundaries.
- Retrieval of centralized audit records through GET /api/audit-events is restricted to the SYSTEM_ADMIN role.

- ADR-012 establishes a minimum seven-year retention policy for critical audit records.
- Complete seven-year archival, production database-role provisioning, and long-term retention enforcement require separate operational verification.

4. Secure Communication and External Services

- The deployed frontend and backend communicate through HTTPS to protect information transmitted over public networks.
- The backend uses JWT authentication and Role-Based Access Control to protect sensitive REST API operations.
- HTTP security headers, restricted Cross-Origin Resource Sharing (CORS), request validation, input sanitization, and rate limiting provide additional application protection.
- Secure payment processing is integrated through PayHere. OSMS does not collect or store Customer card numbers or CVV values.
- Payment completion depends on verified payment-provider notifications rather than browser redirects alone.
- Inventory images are managed through Cloudinary using configured server-side credentials and approved asset-management operations.
- Third-party integrations remain subject to their own provider security configurations and applicable data-protection responsibilities.

5. Backup and Recovery

- MongoDB Atlas is the centralized database platform used for application data.
- Database backup, restoration, and disaster recovery are important operational requirements for preventing data loss under RSK-001.
- The architecture allows the use of managed database backup and recovery capabilities, subject to the configured MongoDB Atlas service and backup settings.
- The current project evidence does not independently establish a completed daily backup schedule, separate-region backup storage, or a successfully tested production database restoration procedure.

- Consequently, automatic daily backups and geographically separated disaster recovery must be treated as operational controls requiring verification rather than confirmed implemented functionality.
- Recovery testing should verify data consistency, restoration success, and the preservation of critical financial, inventory, and clinical records.

B. Database Security

1. Database Access Control

- MongoDB Atlas is accessed by the Node.js/Express.js backend using a configured database connection string.
- The database connection information is supplied through the server-side MONGODB_URI environment variable.
- Database connection credentials and other backend secrets must not be exposed in frontend source code, API responses, or application logs.
- The React frontend communicates with MongoDB indirectly through authorized backend REST APIs rather than connecting to the database.
- Database-level permissions should follow the principle of least privilege.

2. Network Security

- The operational system uses Vercel for frontend hosting, Render for backend API hosting, and MongoDB Atlas for database persistence.
- HTTPS is used for public frontend and backend communication.
- MongoDB Atlas network access restrictions should be configured to permit only approved connections appropriate to the deployment architecture.
- Dedicated AWS Virtual Private Cloud (VPC) deployment, private database peering, and AWS IAM-based database access are not established as part of the current operational architecture.
- The exact production MongoDB Atlas network access list and private connectivity configuration require infrastructure-level verification.

3. Sensitive Data Protection

- Authentication passwords are stored using Bcrypt hashes rather than plaintext.
- Password-reset tokens are stored as hashes and subject to expiry and single-use controls.
- Clinical prescription records are stored in MongoDB and accessed through role-restricted and ownership-aware backend operations.
- The Prescription collection maintains Customer references, the recording Optometrist's identity, clinical measurements, and historical recording information.
- Financial records maintain approved payment references and status information without storing raw payment-card credentials.
- HTTPS protects data in transit. Encryption-at-rest guarantees depend on the configured managed database and storage-provider controls and must be verified against the operational environment.
- The current application does not establish a separate application-level encryption scheme for every stored prescription field.

4. Database Integrity and Audit Protection

- Mongoose schemas apply field validation, required attributes, permitted values, and database indexes.
- MongoDB transactions maintain consistency for selected multi-document operations, including checkout, payment-related processing, appointment scheduling, and applicable critical audit records.
- Customer and branch ownership restrictions are enforced before sensitive records are retrieved or modified.
- AuditEvent records use application-level append-only safeguards and cryptographic hash-chain integrity information.
- The repository includes database-role provisioning instructions for additional audit collection protection; successful application of those privileges in the operational database requires verification.

- Audit records are not configured with automatic TTL-based deletion because their approved retention period must be preserved.

C. Data Retention and Disposal

1. Clinical Data Retention

- Clinical prescriptions are maintained as historical records associated with the relevant Customer and recording Optometrist.
- The Prescription schema includes an isArchived field to represent the archival state.
- The current implementation does not automatically invalidate prescription records after 24 months.
- The historical 24-month prescription-validity rule must therefore not be represented as an enforced system control.
- Applicable clinical retention and disposal periods should be confirmed against approved business policies and relevant legal requirements.
- The existence of historical prescription records does not establish that automated seven-year clinical archiving has been implemented.

2. Audit Record Retention

- ADR-012 establishes a minimum seven-year retention policy for critical audit records.
- Audit records use an append-only data model and must not be automatically deleted through a TTL index.
- The approved architecture describes a long-term retention and archival strategy for preserving audit evidence.
- Automated cold-storage archival, retention-lock configuration, long-term retrieval, and archival recovery require separate operational evidence before they can be claimed as fully implemented.
- Database backups are complementary to audit trails and do not independently guarantee audit-record immutability.

3. Temporary Data Retention

- Certain short-lived operational records use separate expiration mechanisms.
- ChatbotUsage records define a TTL index for expiring completed usage windows.
- Password-reset tokens have a limited validity period and are invalidated after successful use.
- The configured production TTL index and cleanup behaviour must be verified in the deployed database environment.
- These temporary-data rules do not apply to the retention of clinical prescriptions, payment evidence, or critical audit records.

4. Secure Data Disposal

- Data disposal should follow approved retention policies and applicable legal requirements.
- Secure disposal must consider active MongoDB records, stored images, backup copies, exported datasets, and archived records where applicable.
- Deleting an active database document does not by itself guarantee immediate removal from database backups or external storage.
- Cloudinary inventory-image deletion and replacement operations are implemented through controlled asset-management workflows.
- No complete automated retention-expiry and secure-erasure process has been verified for all customer, financial, clinical, and backup data.
- Any future disposal process should maintain appropriate authorization, audit evidence, retention checks, and recovery safeguards.

5. Operational Verification Requirements

Before claiming full implementation of the data protection and retention objectives, the following evidence should be confirmed:

- MongoDB Atlas production network access restrictions and database user privileges.
- Database encryption-at-rest configuration and relevant provider controls.

- Automated backup schedule, retention settings, and successful restoration evidence.
- Production audit-collection access restrictions and audit-chain verification.
- Long-term audit archival configuration and retrieval testing.
- Applicable clinical record-retention requirements.
- Secure disposal procedures covering operational records, backup copies, and third-party storage.
- PDPA compliance assessment and any required organizational or procedural controls.

The implemented OSMS therefore provides application-level access restrictions, secure communication, data validation, audit logging, and transaction-integrity mechanisms. Long-term retention, disaster recovery, comprehensive secure disposal, and full regulatory compliance remain dependent on documented operational verification.

6.4 Input Validation and Sanitization Strategy

The Optical Shop Management System (OSMS) implements a multi-layered input validation and sanitization strategy to protect application data, reduce injection risks, and maintain data integrity.

Validation is performed at both the React.js frontend and Node.js/Express.js backend. Frontend validation provides user-friendly feedback, while backend validation acts as the authoritative enforcement mechanism before business operations and database updates.

The implementation combines reusable validation utilities, route-specific request validation, Mongoose schema constraints, input sanitization, and business-rule enforcement.

A. Client-Side Validation (Frontend)

- **Form Validation:** The React.js frontend uses JavaScript validation functions and native HTML input attributes to validate user input. Depending on the form, validation occurs during user interaction or when the form is submitted.
- **Reusable Validation Utilities:** Common validation functions are implemented in src/utils/validators.js, including required-field checks, email format validation, phone number validation, minimum password length, and password confirmation matching.

- **Required Field Validation:** Forms check that mandatory information is provided. For example, Customer registration requires a name, email address, phone number, registered address, and password.
- **Data Type Validation:** Form controls and JavaScript checks validate relevant input formats. Numeric input fields are used for applicable quantities and prices, while email fields are checked against the expected email format.
- **Constraint Enforcement:** Customer registration enforces a minimum password length of eight characters. Other form-specific constraints are applied according to the requirements of each module.
- **Password Confirmation:** The registration interface checks that the password and confirmation fields match before submitting the registration request.
- **Error Feedback:** The reusable FormField component displays validation messages and uses aria-invalid, aria-describedby, and accessible error labels to identify invalid inputs.
- **Business-Related Validation:** The frontend validates selected information for operations such as Cart quantity changes, Checkout navigation, and Appointment booking. Final acceptance remains subject to backend validation.

Client-side validation improves usability but is not treated as a security boundary because browser-side checks can be bypassed.

B. Server-Side Validation (Backend)

- **Custom Validation Middleware:** The Express.js backend implements reusable request validation through src/middleware/validate.js. The middleware supports validation rules for required fields, data types, minimum and maximum lengths, email formats, regular-expression patterns, and custom validation functions.
- **Route-Specific Validation:** Individual API routes define additional validation rules according to the requirements of their operations. These checks reject missing, malformed, unsupported, and unauthorized request fields.
- **Mongoose Schema Validation:** Mongoose models enforce additional data constraints, including required attributes, permitted enumeration values, numeric ranges, and valid document references where applicable.

- **Identifier Validation:** API endpoints validate MongoDB ObjectId values before performing operations involving Customers, Inventory items, Orders, Doctor Sessions, Appointments, and Prescriptions.
- **Business Rule Validation:** The backend enforces domain-specific rules, including:
    - Appointment bookings must satisfy the minimum 24-hour lead time.
    - Doctor Sessions must satisfy capacity and scheduling restrictions.
    - Cart quantities must be positive integers.
    - Inventory quantities and prices must satisfy applicable non-negative constraints.
    - Checkout must validate current stock, prices, and item classifications.
    - Order status changes must follow the permitted lifecycle.
- **Authorization Checks:** Protected endpoints use JWT authentication and Role-Based Access Control (RBAC). Additional ownership, branch, and field-level restrictions are applied before protected operations are executed.
- **Request Body Restrictions:** The backend applies a configurable JSON request-body size limit, with a default of 10 KB. Selected operations apply additional request-specific restrictions.
- **File Upload Validation:** Inventory image uploads are restricted to supported JPEG, PNG, and WebP formats, with a maximum file size of 5 MB. The upload middleware validates both the declared MIME type and the uploaded file's binary signature.
- **Centralized Error Handling:** Validation errors are processed through a centralized error-handling middleware. Invalid requests receive appropriate HTTP error responses, while unexpected internal errors are handled without exposing production stack traces.

The backend uses custom Express validation functions and Mongoose schema validation rather than Joi or Validator.js.

C. Data Sanitization (Protection Against Injection)

- **Centralized Input Sanitization:** The backend implements reusable sanitization logic in src/utils/sanitizer.js. The sanitizer processes incoming request bodies, query parameters, and route parameters.
- **NoSQL Injection Risk Reduction:** The sanitizer recursively processes objects and removes property names that begin with $ or contain a period (.). This reduces the risk of untrusted input being interpreted as MongoDB query operators or nested update paths.
- **Strict Request Field Validation:** Sensitive operations apply explicit field allowlists and reject unsupported input structures. For example, public Customer registration rejects privileged role-related fields, and Checkout does not accept client-supplied prices, totals, stock quantities, or ownership information.
- **XSS Risk Reduction:** The sanitizer removes matching HTML <script> blocks from incoming string values. React's default text rendering and configured HTTP security headers provide additional protection when handling untrusted content. These mechanisms reduce common XSS risks but do not constitute a guarantee against every possible XSS technique.
- **Whitespace Normalization:** The common sanitization function removes leading and trailing whitespace from string values where it is applied.
- **Case Normalization:** Selected fields, including email addresses used during registration and authentication, are normalized to lowercase according to their route-specific validation rules.
- **Safe Database Operations:** Backend services construct database operations using validated identifiers, approved request fields, authenticated ownership information, and applicable Mongoose model constraints rather than trusting arbitrary client-supplied query objects.
- **Sensitive Data Protection in Logs:** Operational log sanitization reduces the exposure of passwords, authentication tokens, database connection strings, and other sensitive values. Audit metadata is subject to additional sanitization and data-minimization rules.

- **Payment Notification Integrity:** PayHere payment callbacks are handled separately from the general JSON sanitization flow to preserve the exact provider-supplied values required for signature verification. The backend validates the permitted callback fields, merchant identity, signature, amount, currency, and related payment state before applying financial changes.

Validation and Sanitization Limitations

The implemented controls provide multiple layers of input protection, but no individual sanitization function can guarantee the prevention of all injection or XSS attacks.

Security also depends on correct output handling, backend authorization, request validation, dependency security, provider configuration, and ongoing testing.

The validation and sanitization strategy should therefore be evaluated through automated security tests, malicious-input test cases, and the project's OWASP security verification process.

6.5 Encryption Standards for Data at Rest and in Transit

The Optical Shop Management System (OSMS) applies encryption-related security controls to protect sensitive information, including customer identities, clinical prescriptions (FR-013), payment records (FR-009), and personal information (FR-001 and FR-002).

The implementation uses HTTPS for communication security, managed cloud services for data storage, cryptographic password hashing, and environment-based secret configuration.

The following measures distinguish verified application controls from encryption capabilities and configurations managed by external hosting providers.

A. Encryption for Data in Transit (Symmetric & Asymmetric)

Data in transit refers to information transmitted between the React.js frontend, Node.js/Express.js backend, database infrastructure, and approved external services.

- **Communication Protocol:** The deployed OSMS frontend is hosted on Vercel, while the backend API is hosted on Render. HTTPS is used for public communication between these components.
- **TLS Encryption:** HTTPS uses Transport Layer Security (TLS) to protect information exchanged over network connections. TLS employs cryptographic mechanisms for secure connection establishment and encrypted communication.
- **Verified Deployment Security:** The project's DDP-085 security verification confirmed that the deployed frontend and backend were accessible through HTTPS. HTTP Strict Transport Security (HSTS) was also verified in the deployed environment.

- **TLS Version Requirements:** The security design targets TLS 1.2 or higher. However, the exact negotiated TLS versions and cipher suites across all provider connections have not been independently documented in the available verification evidence.
- **REST API Protection:** Customer profiles, shopping cart operations, appointment bookings, clinical data requests, and other RESTful API operations are transmitted through HTTPS connections. JSON is the primary API data format.
- **Database Communication:** The backend connects to MongoDB Atlas using Mongoose and a configured MongoDB connection URI. Database transport encryption depends on the configured managed database connection and provider settings, which must be verified operationally.
- **Payment Gateway Communication:** OSMS integrates with PayHere through a hosted checkout process. Customers enter payment credentials directly through PayHere rather than through OSMS.
- **Payment Notification Integrity:** PayHere callbacks are validated using the provider's prescribed signature mechanism and additional merchant, amount, currency, and transaction checks. Signature verification provides authenticity and integrity controls and is separate from TLS encryption.

HTTPS and TLS reduce the risk of unauthorized interception or modification of transmitted data. However, secure transport does not independently eliminate all application-level security risks.

B. Encryption for Data at Rest (Symmetric)

Data at rest refers to information stored in MongoDB Atlas, external media storage, and other persistent storage systems used by OSMS.

- **Database Storage:** MongoDB Atlas is used as the centralized document database for Customer, Staff, Inventory, Order, Payment, Appointment, Prescription, and AuditEvent records.
- **Managed Database Encryption:** MongoDB Atlas provides managed encryption-at-rest capabilities. The exact encryption algorithm, active key-management configuration, and provider-specific storage settings for the deployed database require operational verification before being claimed as independently confirmed.
- **Clinical Data Storage:** Customer prescription records are stored in the MongoDB Prescription collection and protected through application authentication, role-based authorization, ownership restrictions, and database access controls.
- **Application-Level Encryption:** The current implementation does not define a dedicated field-level encryption mechanism for every stored prescription or Customer attribute. The SDS therefore does not claim that all clinical fields are separately encrypted by application code.
- **Object and Media Storage:** OSMS uses Cloudinary for inventory product images rather than Amazon S3. Image uploads and asset-management operations use backend-configured Cloudinary credentials. Encryption-at-rest settings for Cloudinary-hosted assets are provider-managed and require confirmation through the applicable provider configuration.

- **Prescription Scans:** The current application does not implement a separate cloud-storage workflow for uploaded prescription scan files. Therefore, Amazon S3 storage and encryption of prescription scans are not included in the implemented architecture.
- **Password Hashing:** Customer, Staff, and Administrator passwords are stored as Bcrypt hashes using 12 salt rounds. Bcrypt is a one-way password-hashing mechanism rather than reversible encryption.
- **Password Recovery Tokens:** Password-reset tokens are stored as SHA-256 hashes, with restricted validity and single-use handling.
- **Audit Data Protection:** Critical audit records use append-only application controls and SHA-256 hash chaining to provide tamper-evidence. Hash chaining protects record integrity but does not encrypt the contents of the audit records.

C. Key Management

- **Environment-Based Secret Configuration:** Sensitive backend configuration values are loaded from environment variables instead of being exposed through public frontend configuration.
- **JWT Signing Secret:** The backend uses the JWT_SECRET environment variable to configure JWT signing and verification. This secret is required in non-test environments.
- **Database Credentials:** The MongoDB connection URI is supplied through the backend MONGODB_URI configuration variable. Database credentials must remain restricted to authorized server-side environments.
- **External Service Credentials:** Cloudinary API credentials, PayHere merchant credentials, and Gemini AI API credentials are supplied through backend configuration rather than public React application variables.
- **Frontend Configuration:** Public Vite environment variables, such as VITE_API_BASE_URL, are used only for non-secret client-side configuration. Database credentials, JWT signing secrets, and payment-provider secrets must not be placed in frontend environment variables.
- **Cloud Provider Key Management:** The current deployed architecture does not establish OSMS-managed AWS Key Management Service (KMS) integration. Encryption-key management for MongoDB Atlas, Cloudinary, Vercel, and Render is subject to the applicable provider configuration.
- **Key Access and Rotation:** Cryptographic secrets should be restricted to authorized administrators and rotated according to approved operational security procedures. Automatic rotation of all application and storage encryption keys has not been independently verified.
- **Credential Protection:** Backend error handling and operational logging apply controls to reduce accidental disclosure of sensitive secrets, authentication tokens, and database connection information.

Implementation and Verification Considerations

The implemented OSMS provides verified HTTPS communication, Bcrypt password hashing, backend-managed application secrets, and payment signature verification.

However, the following controls require independent infrastructure or provider-level verification before full implementation claims can be made:

- Exact TLS protocol versions and cipher suites.
- MongoDB Atlas encryption-at-rest configuration and active key-management settings.
- Cloudinary storage encryption configuration.
- Database and service credential-rotation procedures.
- Provider-managed key access restrictions and recovery procedures.
- Any future application-level encryption requirements for sensitive clinical fields.

Accordingly, the encryption design documents the implemented security mechanisms without asserting unsupported AWS KMS integration, Amazon S3 encryption, or unverified cryptographic configurations.

Chapter 7: Deployment Design

The Optical Shop Management System (OSMS) follows a cloud-hosted deployment architecture using Vercel, Render.com, MongoDB Atlas, and Cloudinary.

The deployment separates the React.js frontend, Node.js/Express.js backend, database, and external integration services to support maintainability, security, and operational flexibility.

The backend is containerized using Docker, preserving consistency between local development and cloud deployment environments.

The implemented architecture reflects ADR-015 and the approved deployment approach under DDP-083, which replaced the original AWS App Runner and Amazon S3 deployment assumptions.

The system retains the SRS targets of 99.9% availability and support for a 50% increase in concurrent transactions. These are non-functional performance and availability objectives rather than independently verified guarantees of the deployed infrastructure.

7.1 Hosting Plan

The OSMS hosting strategy uses managed cloud platforms to reduce infrastructure administration and support the operational requirements of Darshana Opticals (Pvt) Ltd.

The implemented deployment consists of a Vercel-hosted React frontend, a Docker-containerized Express backend running on Render.com, a managed MongoDB Atlas database, and Cloudinary-based inventory image storage.

1. Frontend Hosting (Web Application Delivery)

- **Platform:** Vercel.
- **Technology:** React.js with Vite and React Router.
- **Implementation:** The frontend application is built using Vite and deployed through Vercel. The approved deployment configuration tracks the develop branch in the Vercel Production environment.
- **Configuration:** The frontend uses VITE_API_BASE_URL to communicate with the approved Render backend through HTTPS.

- **Routing:** The deployed vercel.json configuration provides a Single-Page Application (SPA) rewrite to support direct navigation and page refreshes on React Router routes.
- **Security:** The frontend deployment includes configured HTTP security headers and HTTPS communication.
- **Justification:** Vercel provides managed frontend hosting, static asset delivery, deployment integration, and simplified release management without requiring the development team to maintain a dedicated web server.

Implementation Considerations:

The exact Vercel subscription tier and any paid performance features must be confirmed through the hosting account. Sub-second page loads, a particular uptime SLA, and specific WAF entitlements are not asserted without verification.

2. Backend API Hosting (Docker Container Deployment)

- **Platform:** Render.com.
- **Technology:** Node.js 20, Express.js, and Docker.
- **Implementation:** The backend REST API is deployed as a Docker-based Render web service.
- **Container Configuration:** The backend uses an approved Dockerfile based on node:20-alpine, with npm start as the production application entry point.
- **API Communication:** The frontend communicates with the backend over HTTPS.
- **Health Monitoring:** The backend exposes /api/health for application health verification.
- **Environment Configuration:** Database credentials, JWT secrets, payment gateway credentials, and external API keys are supplied through server-side environment variables.
- **Justification:** Render supports deployment of the existing Dockerized Express application without requiring major changes to the backend architecture or business logic.

Architectural Decision:

ADR-015 approved Render.com as the operational hosting fallback after AWS App Runner provisioning restrictions affected the original deployment plan.

The current deployment preserves Docker compatibility, allowing the application to remain portable to other compatible container-hosting environments.

Implementation Considerations:

Automatic horizontal scaling, multi-instance redundancy, and specific production resource allocations depend on the selected Render service plan and configuration. These capabilities are not assumed to be enabled.

3. Database Hosting (Managed Cloud Database)

- **Platform:** MongoDB Atlas.
- **Technology:** MongoDB with Mongoose Object Data Modeling (ODM).
- **Implementation:** The backend connects to MongoDB Atlas using the configured MONGODB_URI.
- **Data Storage:** MongoDB stores Customers, Staff, Inventory, Orders, Payments, Appointments, Doctor Sessions, Prescriptions, loyalty transactions, and audit records.
- **Transaction Support:** MongoDB transactions are used for applicable operations requiring atomic updates across multiple documents, including checkout and appointment-related workflows.
- **Access Control:** Database access is performed through the backend application rather than directly from the browser.
- **Security:** Application authentication, RBAC, database credentials, data validation, and audit controls restrict access to sensitive information.
- **Justification:** MongoDB Atlas provides a managed document database that supports the application's Mongoose models and transactional requirements while reducing direct database-server administration.

Implementation Considerations:

The deployed Atlas cluster tier, region configuration, private networking, backup schedule, point-in-time recovery, and database availability guarantees require verification through the operational Atlas account.

The architecture does not assume that an M10+ cluster, multi-region replication, or VPC peering has been provisioned.

4. Media and Object Storage

- **Platform:** Cloudinary.
- **Implementation:** Cloudinary stores product and inventory images uploaded through authorized inventory-management operations.
- **Supported Formats:** JPEG, PNG, and WebP.
- **Upload Validation:** The backend validates file type, binary content, and a maximum upload size of 5 MB.
- **Credential Management:** Cloudinary API credentials are configured through backend environment variables.
- **Asset Management:** Authorized operations manage the association of hosted images with inventory records and support relevant image replacement and deletion procedures.
- **Justification:** Cloudinary provides managed image hosting and delivery while avoiding the need to maintain a separate Amazon S3 storage workflow.

Implementation Considerations:

The current system does not use Amazon S3 for inventory images or implement scanned clinical prescription uploads.

Automatic S3 Intelligent-Tiering, seven-year prescription image archiving, and AWS-managed object storage encryption are not part of the implemented hosting plan.

5. Scaling and Cost Management

- **Hosting Model:** OSMS uses managed cloud services, with costs and available resources determined by the configured provider plans.
- **Frontend Scaling:** Vercel handles frontend asset hosting and delivery according to the capabilities of the selected account plan.
- **Backend Scaling:** Render provides managed container hosting. Available processing capacity, instance scaling, and service availability depend on the configured plan and allocated resources.
- **Database Scaling:** MongoDB Atlas provides cluster management and scaling options according to the selected tier and configuration.
- **Media Management:** Cloudinary provides external image storage, reducing the need to serve large image files directly from the backend application.
- **Cost Management:** Hosting expenses should be monitored through Vercel, Render, MongoDB Atlas, and Cloudinary usage and billing dashboards.
- **Performance Monitoring:** Application health checks, performance testing results, and deployment logs should be used to evaluate service capacity and identify bottlenecks.
- **Justification:** The chosen architecture reduces the need to provision and maintain dedicated physical or virtual servers while supporting future infrastructure improvements.

Implementation Considerations:

The SRS requirement to support a 50% increase in concurrent transactions remains a scalability target. Compliance with this target must be demonstrated through performance testing and, where required, appropriate infrastructure capacity upgrades.

Similarly, the 99.9% availability objective depends on provider service levels, application stability, operational monitoring, and recovery procedures. It is not automatically guaranteed by the use of managed cloud platforms.

External Service Integrations

The cloud-hosted application also communicates with external services:

- **PayHere:** Hosted payment processing and verified server-to-server payment notifications.
- **Google Gemini AI:** AI-powered chatbot responses supported by the internal Knowledge Area.
- **SMTP Email Service:** Email-based application notifications, including supported order-related communication and password recovery.
- **Cloudinary:** Inventory image storage and delivery.

These services are integrated through backend-controlled configuration and credentials. Their availability and provider-specific security characteristics form part of the application's operational dependencies.

Deployment and Release Considerations

The project maintains separate frontend, backend, infrastructure, and documentation repositories.

GitHub Actions is used for automated build, testing, and quality checks, while the approved hosting platforms support cloud deployment workflows.

The active deployment approach uses the frontend develop branch for the Vercel Production environment and the existing Render backend. Historical AWS deployment files are retained for architectural traceability but do not represent the active operational hosting environment.

Deployment readiness must be supported by appropriate CI results, successful builds, application health verification, API connectivity tests, and relevant security checks. A successful cloud deployment alone does not establish that all availability, scalability, or security requirements have been satisfied.

7.2 Hardware Requirements

The Optical Shop Management System (OSMS) is a cloud-hosted, browser-based application that does not require dedicated on-premise server hardware at Darshana Opticals branches.

The frontend is hosted on Vercel, the backend on Render.com, and application data is stored in MongoDB Atlas. Branch employees and Customers access the system through supported web browsers and internet-connected devices.

The following hardware specifications are recommended for reliable day-to-day operation. They represent practical deployment guidelines rather than independently verified minimum hardware requirements.

1. Staff and Administrative Terminals

- **Device Type:** Desktop computers and laptops are recommended for Branch Managers, Branch Assistants, Optometrists, and System Administrators. Tablets may also be used where the interface has been verified for the relevant screen size.
- **Processor:** A modern dual-core or quad-core processor is recommended for routine browser-based operations. A quad-core processor running at approximately 2.0 GHz or higher is a suitable workstation recommendation.
- **Memory (RAM):** 8 GB RAM is recommended for comfortable multitasking between the OSMS application, browser tabs, and other office software.
- **Display:** A Full HD (1920 × 1080) monitor is recommended for viewing inventory tables, sales reports, customer orders, and clinical forms. Smaller displays may be used subject to responsive layout and usability testing.
- **Input Devices:** Standard keyboards, mice, and touchpads are sufficient for the implemented Staff and Administrative workflows.
- **Web Browser:** A modern, updated web browser with JavaScript enabled is required to access the React-based application.

These workstation recommendations do not guarantee the SRS response-time target. Application response time also depends on backend processing, database performance, cloud availability, and network conditions.

2. Clinical Operations (Optometrist Room)

- **Clinical Workstation:** A desktop computer or laptop with internet access is recommended for recording and reviewing customer prescriptions.
- **Prescription Entry:** The implemented system provides a digital prescription form for entering right-eye and left-eye measurements, including SPH, CYL, Axis, visual acuity, reading measurements, and optional remarks.
- **Input Devices:** A standard keyboard and pointing device are sufficient for prescription recording.
- **Tablets:** Portable tablets may be used as an alternative where the browser, screen size, and touch interaction are compatible with the implemented interface.
- **Prescription Scanning:** Dedicated prescription scanners and camera-based clinical document uploads are not required by the current implementation because prescriptions are recorded as structured digital data rather than uploaded scanned documents.

Any future scanner or camera integration would require separate functional development and hardware compatibility testing.

3. Retail and Inventory Peripherals

- **Inventory Management:** Branch Managers manage stock through the web-based Inventory Management interface. Stock quantities can be entered and updated manually using standard computer input devices.
- **Barcode Scanners:** Barcode-scanner integration is not implemented in the current system. USB or Bluetooth barcode scanners may be considered for future enhancements, but they are not required to operate the existing inventory workflows.
- **Receipt Printers:** Dedicated thermal receipt-printer integration and automated point-of-sale receipt printing are not implemented. Therefore, specialized receipt printers are not mandatory hardware requirements.
- **Standard Printers:** A conventional office printer may be used for general business documentation where needed, independently of any specialized OSMS printing integration.

- **Product Images:** Authorized inventory image management supports selecting and uploading existing JPEG, PNG, or WebP image files through the browser. Dedicated image-capture hardware is not required.

4. Network Infrastructure

- **Router/Gateway:** A reliable broadband router or gateway is recommended for each branch. Dual-band Wi-Fi capability is suitable for connecting supported office devices.
- **Internet Connectivity:** A stable broadband connection is necessary because the frontend, backend, database, and external services operate through cloud infrastructure.
- **Recommended Bandwidth:** A connection providing approximately 10 Mbps or higher may be used as an initial planning recommendation for small-branch operations. This value has not been independently validated as the minimum bandwidth required by OSMS.
- **Connection Security:** Branch networks should use appropriately secured Wi-Fi configurations, updated router firmware, and restricted administrative access.
- **Reliability:** A backup internet connection or mobile hotspot may be considered for branches where temporary broadband interruptions would disrupt operational activities.
- **Cloud Dependency:** Internet connectivity is necessary for accessing live inventory information, Orders, Appointments, and other backend-dependent functions. The implemented system does not provide a complete offline operational mode.

Hardware and Performance Considerations

The OSMS uses cloud infrastructure to reduce the need for physical server maintenance at individual branches.

The SRS response-time objective of approximately two seconds applies to application performance rather than a specific client processor speed or RAM capacity.

Local synthetic backend performance tests have evaluated the response-time objective; however, these results do not establish universal performance guarantees for every client device, browser, network connection, or production workload.

Hardware recommendations should therefore be validated during deployment and User Acceptance Testing (UAT) using representative devices and branch network conditions.

7.3 Network Diagram

The network architecture of the Optical Shop Management System (OSMS) follows a cloud-hosted, multi-tier deployment model using Vercel, Render.com, MongoDB Atlas, and approved external service providers.

The architecture separates frontend delivery, backend application processing, database persistence, and third-party integrations. This separation supports maintainability, security, and controlled access to sensitive operational and clinical information.

Unlike the original AWS-centered design, the implemented network architecture does not rely on dedicated AWS Virtual Private Cloud (VPC) subnets, AWS Application Load Balancers, or Amazon S3 storage.

A. Network Components and Flow

1. Public Web Tier (Vercel)

- **Platform:** Vercel.
- **Application:** React.js and Vite Single-Page Application (SPA).
- **Users:** Customers, Branch Assistants, Branch Managers, Optometrists, and System Administrators access the application using supported web browsers.
- **Communication:** The frontend is delivered to users through HTTPS.
- **Routing:** React Router provides client-side navigation, supported by the Vercel SPA rewrite configuration.
- **API Configuration:** The frontend uses the configured VITE_API_BASE_URL to send requests directly from the browser to the Render backend API.
- **Security:** The frontend deployment uses HTTPS and configured HTTP security headers. Browser authentication requests and protected API operations use the existing JWT-based authentication workflow.

2. Application Tier (Render.com)

- **Platform:** Render.com.
- **Technology:** Docker-containerized Node.js 20 and Express.js backend.
- **Implementation:** The backend runs as a managed web service and exposes RESTful API endpoints over HTTPS.
- **Authentication:** Protected endpoints validate signed JWT access tokens.
- **Authorization:** Role-Based Access Control (RBAC), Customer ownership checks, and branch-specific access restrictions protect sensitive operations.
- **Application Security:** The backend applies CORS restrictions, input validation, request sanitization, HTTP security headers, rate limiting, and centralized error handling.
- **Business Processing:** Backend services handle Customer registration, inventory management, Order processing, payments, appointments, prescriptions, loyalty operations, reporting, and chatbot interactions.
- **Health Endpoint:** /api/health provides application health information for deployment verification and operational monitoring.

The Render platform manages the public HTTPS service endpoint. A dedicated AWS Application Load Balancer or separately provisioned AWS private subnet is not part of the current architecture.

3. Data Tier (MongoDB Atlas)

- **Platform:** MongoDB Atlas.
- **Database Access:** The Node.js backend accesses MongoDB through Mongoose using the configured MONGODB_URI.
- **Data Storage:** MongoDB stores Customer and Staff information, Inventory records, Orders, Payments, Doctor Sessions, Appointments, Prescriptions, loyalty records, and AuditEvents.
- **Data Integrity:** Transactional operations are used where required to maintain consistency across related records.

- **Access Restriction:** The React frontend does not directly connect to MongoDB. Database operations are performed through authorized backend services.
- **Network Protection:** MongoDB Atlas network access restrictions and connection security must be configured appropriately for the deployed Render service.

The exact Atlas network allowlist, private connectivity configuration, cluster tier, and provider-managed encryption settings require operational verification.

4. External Integration Services

The backend communicates with approved external systems:

- **PayHere Payment Gateway:** Provides hosted checkout services. Payment completion is determined by verified server-to-server Instant Payment Notifications (IPNs). Browser return and cancellation redirects do not independently establish payment success.
- **Cloudinary:** Provides inventory product-image hosting. The backend uses the Cloudinary SDK for authorized image uploads and deletion operations. Hosted images are delivered through approved HTTPS image URLs.
- **Google Gemini AI:** Processes eligible chatbot requests using relevant Knowledge Area information supplied by the backend.
- **SMTP Email Provider:** Supports approved Order notifications and password-recovery communication through the Nodemailer library and configured SMTP transport.

Nodemailer is an email-sending library rather than a separate cloud hosting platform. Dedicated Twilio SMS integration and automated appointment reminders are not implemented.

B. Connectivity Logic

1. Secure Frontend Communication

Customers and authorized staff load the React frontend from Vercel through HTTPS.

The browser executes the frontend application and sends RESTful API requests directly to the approved Render backend URL.

Protected requests include the JWT access token in the HTTP Authorization header.

2. Backend API Communication

The Render backend receives frontend requests, validates their structure, authenticates users, and enforces role-based and resource-level authorization.

CORS restricts which configured browser origins are permitted to read cross-origin API responses. CORS is not a substitute for backend authentication, authorization, or network isolation.

3. Database Connectivity

Backend services communicate with MongoDB Atlas through the configured database connection.

Users cannot directly query the database through the frontend application.

Database access is controlled through application permissions and the configured MongoDB Atlas credentials and network settings.

4. Payment Connectivity

When a Customer initiates payment, the backend generates the required signed PayHere checkout information.

The Customer's browser navigates to the PayHere-hosted payment interface.

PayHere sends payment notifications to the backend endpoint /api/payments/notify.

The backend validates the notification signature and associated payment details before making authorized financial-state changes.

5. Media Connectivity

Authorized inventory-image uploads pass through the backend and are processed using Cloudinary.

The application stores approved Cloudinary image references in MongoDB, while actual image assets are managed by Cloudinary.

Product images can subsequently be retrieved through HTTPS URLs without storing the files directly in MongoDB.

6. AI and Email Connectivity

The backend communicates with the Google Gemini API for supported chatbot requests.

For supported email operations, the backend uses Nodemailer and the configured SMTP transport to send messages through the approved email service.

External provider credentials remain in server-side environment configuration and are not exposed through public frontend variables.

Network Security Considerations

The network design incorporates several protective controls:

- HTTPS communication for public frontend and backend endpoints.
- JWT authentication for protected API operations.
- Role-Based Access Control and resource ownership validation.
- Restricted frontend origins through backend CORS configuration.
- Server-side management of database and external service credentials.
- Request validation, sanitization, HTTP security headers, and rate limiting.
- PayHere signature verification for payment notifications.
- Database access through the backend rather than direct browser connections.
- Audit logging for approved critical application operations.

The architecture does not claim that AWS private networking, MongoDB Atlas VPC peering, or dedicated load-balancer infrastructure has been provisioned.

Deployment Verification

The project's deployment documentation and security assessment confirm the use of Vercel and Render and provide evidence of HTTPS operation and frontend–backend integration.

However, provider-specific infrastructure characteristics, including MongoDB Atlas network restrictions, private connectivity, selected hosting tiers, and infrastructure redundancy, require verification through the relevant operational accounts.

The diagram therefore represents the implemented application connections without asserting unverified private networking or high-availability guarantees.

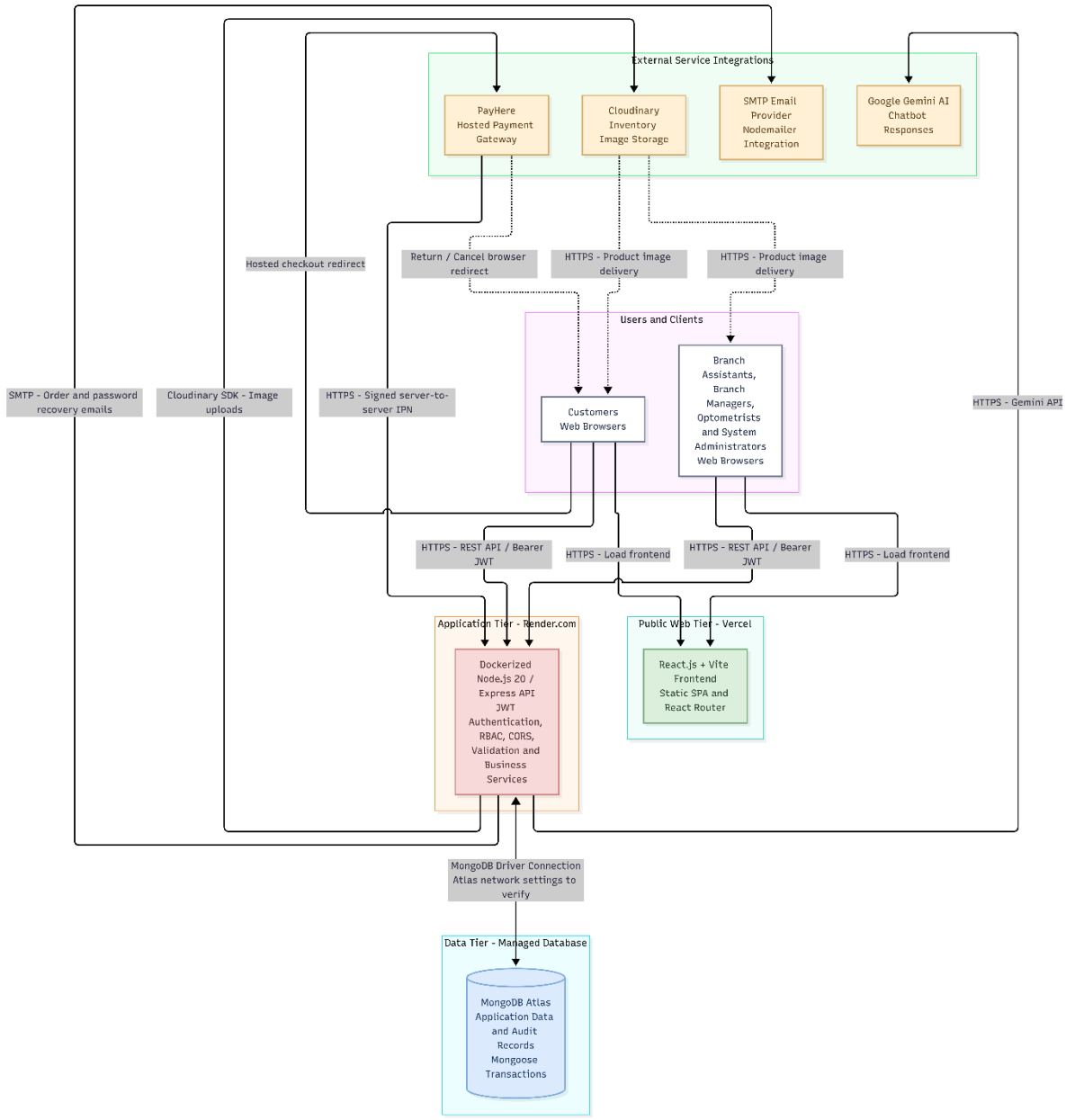


Figure 52: Network Diagram — OSMS Cloud Deployment Architecture

7.4 Environment Specification

The Optical Shop Management System (OSMS) follows an environment configuration strategy that separates local software development from the deployed cloud application.

The original deployment plan defined three independent environments: Development, Staging, and Production. During Sprint 3, the deployment approach was revised to use a single cloud-hosted application environment, consisting of the Vercel frontend, Render.com backend, and configured MongoDB Atlas database.

Under DDP-083, a separate staging frontend, backend, and database stack is not provisioned. Integration testing, User Acceptance Testing (UAT), and academic demonstrations are performed using controlled test configurations and approved test data.

The environment strategy supports consistent software development, secure configuration management, automated quality checks, and controlled release verification.

1. Development Environment (Dev)

Purpose: Used by developers for feature implementation, debugging, unit testing, integration testing, and local verification.

- **Frontend:** The React.js and Vite application runs locally, normally through http://localhost:5173.
- **Backend:** The Node.js/Express.js backend runs locally, normally through http://localhost:5000.
- **Database:** Development uses a local MongoDB database or an appropriately isolated development database.
- **Docker Support:** Docker Compose provides a containerized local environment containing the frontend, backend, and a MongoDB replica set.
- **Configuration:** Environment-specific variables configure database connections, API addresses, JWT secrets, and external integrations.
- **Data:** Developers use synthetic or non-production data to prevent accidental modification or disclosure of real Customer and clinical information.
- **Testing:** Unit and integration tests are performed using the project's automated test suites. Selected backend tests use an isolated in-memory MongoDB environment.

- **Security:** Authentication, RBAC, input validation, and relevant business restrictions remain enabled during normal development.

The Development Environment is independent of the cloud-hosted application and supports testing changes before they are integrated into the shared codebase.

2. User Acceptance Testing Environment (UAT)

Purpose: Used for validating the integrated application against the approved functional requirements, business workflows, and university project evaluation criteria.

A dedicated cloud-based Staging Environment was included in the original deployment design. However, the revised DDP-083 deployment approach does not provision an additional independent staging stack.

Instead, UAT is conducted as a controlled testing activity using the available deployed application and approved test configurations.

- **Frontend:** The deployed Vercel application is used for approved testing and demonstrations.
- **Backend:** The existing Render.com API service supports integrated application testing.
- **Database:** Testing uses the configured MongoDB Atlas database with approved test records and controlled data-handling procedures. A separately provisioned UAT database is not assumed.
- **Payment Testing:** PayHere Sandbox is used when testing payment workflows without processing live financial transactions.
- **Other Integrations:** Cloudinary, Gemini AI, and SMTP-related functionality are tested according to approved provider and notification configurations.
- **Access:** Testing is conducted by authorized project team members, evaluators, and approved test users.
- **Testing Activities:** The team validates Customer registration, authentication, product discovery, cart operations, checkout, payment notifications, appointments, prescriptions, inventory management, reporting, and administrative functions.

Testing must use approved fictional or synthetic identities and avoid uncontrolled real payments or sensitive Customer information.

Because UAT shares the available deployed infrastructure, it does not provide the same infrastructure isolation as a separately provisioned staging environment.

3. Deployed Cloud Environment (Vercel Production Configuration)

Purpose: Provides the publicly accessible cloud-hosted OSMS application for approved system operation, integration verification, and project demonstrations.

- **Frontend Platform:** Vercel, using its Production environment configuration.
- **Frontend Source Branch:** The approved DDP-083 configuration tracks the develop branch.
- **Backend Platform:** Render.com, running the Docker-containerized Node.js/Express.js application.
- **Database:** MongoDB Atlas, accessed through the backend using Mongoose.
- **Media Storage:** Cloudinary for inventory images.
- **External Services:** PayHere, Google Gemini AI, and the configured SMTP email service.
- **Communication:** HTTPS between browsers and the deployed frontend/backend endpoints.
- **Authentication:** JWT authentication and the implemented five-role RBAC system.
- **Configuration:** Backend credentials and provider secrets are maintained through server-side environment configuration.
- **Monitoring:** Application health checks, provider logs, GitHub CI results, and deployment verification procedures support operational monitoring.

The project deployment documentation identifies the following application endpoints:

- **Frontend:** https://osms-frontend-staging.vercel.app
- **Backend:** https://osms-backend-staging.onrender.com
- **Backend Health Check:** https://osms-backend-staging.onrender.com/api/health

Although these endpoints contain the word "staging", their names do not establish the presence of an additional, independent staging infrastructure. The active deployment configuration is determined by the actual hosting services, environment variables, and release procedures.

The Vercel Production environment designation also does not independently establish that the application is processing live Customer transactions or real clinical data. Production-business readiness requires appropriate release approval, operational configuration, and supporting verification evidence.

Environment Comparison Table

Table 13: Environment Comparison Table

Feature	**Development**	**UAT / Integration Testing**	**Deployed Cloud Application**
Purpose	Development, debugging, unit testing	Integrated workflow testing and evaluation	Hosted OSMS application and approved demonstrations
Infrastructure	Local workstation / Docker Compose	Uses existing deployed infrastructure	Vercel + Render + MongoDB Atlas
Frontend	Local React/Vite application	Shared deployed Vercel frontend	Vercel Production configuration
Backend	Local Express server / Docker	Shared Render backend	Render Docker web service
Database	Local or isolated development MongoDB	Configured Atlas database with controlled test data	Configured MongoDB Atlas

Accessibility	Development team	Authorized testers and evaluators	Publicly accessible application with protected features
Data Type	Synthetic / development data	Approved fictional or synthetic test data	Data according to approved operational configuration
Security	JWT, RBAC and application validation	JWT, RBAC, HTTPS and test safeguards	JWT, RBAC, HTTPS, CORS and production security controls
Payment Configuration	Test configuration	PayHere Sandbox	Configured PayHere mode; live mode requires separate approval
CI/CD	GitHub Actions on supported pushes and PRs	CI results and controlled integration verification	Platform deployment from approved source revisions
Isolation	Separate local runtime	No dedicated cloud staging stack	Single approved cloud deployment
Primary Users	Developers	QA team, project supervisors and evaluators	Authorized Customers, Staff and system administrators

Configuration and Release Management

1. Source Code Management

The project uses GitHub repositories for frontend, backend, infrastructure, and documentation.

Developers implement changes through reviewed branches and integrate accepted work into develop.

The main branch is reserved for approved release integration and final version tagging according to the project's release management strategy.

2. Continuous Integration

GitHub Actions executes automated verification processes.

The frontend CI workflow runs ESLint, formatting checks, tests with coverage, and the production build on supported pushes and pull requests targeting develop or main.

The backend CI workflow runs linting, formatting checks, automated tests, and dependency security checks according to its configured triggers.

3. Cloud Deployment

The deployed Vercel frontend is configured to track the approved develop branch.

The backend runs through the existing Render Docker hosting service.

Cloud deployment success must be verified against the intended source revision. A successful build alone does not prove that all CI, security, integration, and business acceptance criteria have passed.

4. Environment Variables and Secrets

Separate configuration values are used for local and deployed environments.

Sensitive backend settings include MongoDB credentials, JWT signing secrets, PayHere merchant credentials, Cloudinary credentials, Gemini API credentials, and SMTP information.

Public frontend configuration uses approved Vite variables such as VITE_API_BASE_URL. Backend secrets must never be embedded in frontend application bundles.

5. Deployment Verification and Rollback

Approved deployments should be verified using:

- Source commit and deployment revision identification.
- Required automated CI checks.
- Frontend and backend health verification.
- API connectivity and configured CORS restrictions.
- Authentication and authorization tests.
- Critical Customer and Staff workflow smoke tests.
- Appropriate external integration verification.

The infrastructure repository maintains procedures for controlled frontend and backend rollback.

Database recovery operations must preserve business data and avoid destructive resets during routine release verification.

Environment Limitations

The revised deployment approach minimizes operational complexity but does not provide complete infrastructure isolation between UAT and the deployed application.

In particular:

- A separate AWS App Runner staging backend is not part of the active deployment.
- A separate Vercel staging frontend is not provisioned under DDP-083.
- A new independent MongoDB Atlas UAT database is not provisioned under the revised approach.
- Vercel's Production environment label does not imply that every external integration operates in live mode.

- The exact database tier, backup configuration, and infrastructure availability guarantees require operational verification.
- UAT activities must be controlled to avoid modifying sensitive operational data or triggering unintended financial transactions and notifications.

The environment strategy therefore reflects the implemented OSMS deployment while distinguishing development, controlled testing, and cloud operation without claiming separate infrastructure that has not been provisioned.

7.5 CI/CD Pipeline Overview

The Optical Shop Management System (OSMS) uses GitHub Actions for Continuous Integration (CI) and cloud platform deployment capabilities for application delivery.

The CI/CD approach supports code quality, automated testing, version control, deployment traceability, and controlled release management across the frontend, backend, infrastructure, and documentation repositories.

The current deployment strategy uses Vercel for the React.js frontend and Render.com for the Docker-containerized Node.js/Express.js backend, in accordance with ADR-015 and DDP-083.

The original AWS ECR, App Runner, and ECS Fargate deployment architecture has been replaced by the operational Vercel and Render hosting approach. Historical AWS automation remains in the infrastructure repository for reference and possible future use.

A. Pipeline Stages

1. Continuous Integration (CI)

Source Control and Branch Management

- OSMS source code is managed using GitHub.
- Development work is performed through feature, fix, and other approved short-lived branches.
- Developers submit Pull Requests for code review before integrating changes into the develop branch.
- The develop branch is the main integration branch used during the project.

- The main branch is reserved for approved release integration and version tagging.
- GitHub Actions executes CI workflows according to the configured push and Pull Request triggers.

Frontend Continuous Integration

The frontend CI workflow is defined in .github/workflows/frontend-ci.yml.

The workflow uses Node.js 24 and performs the following steps:

1. Check out the relevant frontend source-code revision.
2. Install dependencies using npm ci.
3. Execute ESLint through npm run lint.
4. Verify code formatting through npm run format:check.
5. Execute automated tests and generate coverage results using npm run test:coverage.
6. Build the frontend application using npm run build.

The workflow runs on supported branch pushes and Pull Requests targeting develop or main.

A successful workflow confirms that the configured frontend CI checks completed successfully for the tested revision.

Backend Continuous Integration

The backend CI workflow is defined in .github/workflows/ci.yml.

The workflow uses Node.js 20 and performs the following steps:

1. Check out the relevant backend source-code revision.
2. Install dependencies using npm ci.
3. Execute ESLint through npm run lint.
4. Verify formatting using npm run format:check.
5. Execute Jest unit and integration tests with code coverage.
6. Upload the generated backend coverage report as a GitHub Actions artifact.

7. Run npm audit --audit-level=high for applicable Pull Request and protected-branch events.

The backend workflow runs on pushes and Pull Requests targeting develop, according to its configured triggers.

Automated CI results support code review and release approval but do not replace end-to-end testing, security verification, or production deployment validation.

2. Continuous Deployment (CD)

Frontend Deployment — Vercel

- **Platform:** Vercel.
- **Application:** React.js Single-Page Application built using Vite.
- **Source Repository:** darshana-opticals-osms/frontend.
- **Deployment Branch:** develop, tracked by the configured Vercel Production environment.
- **Build Command:** npm run build.
- **Output Directory:** dist.
- **Configuration:** VITE_API_BASE_URL identifies the approved Render backend API.
- **Routing:** The vercel.json configuration supports React Router navigation and direct page refreshes.

Vercel's Git integration supports deployment from the configured source branch.

The development team must verify the relationship between deployment triggers and GitHub CI checks before treating any deployment as fully approved.

A successful Vercel build does not independently prove that every required quality gate has passed.

Backend Deployment — Render.com

- **Platform:** Render.com.
- **Application:** Node.js 20 and Express.js.
- **Containerization:** Docker.
- **Source Repository:** darshana-opticals-osms/backend.
- **Container Configuration:** The approved backend Dockerfile uses node:20-alpine.
- **Application Startup:** npm start, executing the backend server.
- **Configuration:** Runtime settings and service credentials are supplied through server-side environment variables.
- **Health Check:** /api/health.

The backend is deployed as a Docker-based Render web service.

The application preserves the approved Docker configuration for consistency between local and cloud execution.

The active deployment does not require Amazon ECR image uploads or AWS ECS Fargate deployment.

Render deployment history and service logs support deployment verification and troubleshooting.

Historical AWS Deployment Automation

The infrastructure repository retains a GitHub Actions workflow for the original AWS staging deployment design.

This workflow includes:

- Backend CI verification.
- Docker image creation.
- Amazon ECR image management.
- AWS App Runner deployment.
- Application health verification.

However, it is not the active deployment mechanism under the revised Vercel and Render architecture.

The backend's cross-repository AWS deployment dispatch is controlled through the ENABLE_AWS_DEPLOY_DISPATCH configuration flag.

These historical artifacts are preserved for architectural traceability and potential future infrastructure migration.

3. Deployment Verification

Following a cloud deployment, the team should verify:

- The deployed Git commit and expected application revision.
- GitHub Actions CI results.
- Successful frontend and backend builds.
- Availability of the frontend through HTTPS.
- Backend health through /api/health.
- Correct frontend-to-backend API connectivity.
- Configured CORS restrictions.
- Authentication and Role-Based Access Control (RBAC).
- Critical Customer and Staff workflows.
- Relevant PayHere Sandbox and external integration behaviour.

The project's release quality-gate process additionally considers end-to-end testing, OWASP security verification, performance evaluation, and final documentation approval.

Deployment acceptance requires appropriate verification evidence rather than relying only on a successful build or hosting-provider status.

B. Key Benefits for OSMS

1. Faster Bug Fixes and Maintenance

- Automated CI reduces the manual effort required to validate source-code changes.
- Developers receive early feedback on linting errors, formatting issues, test failures, and build problems.
- Consistent verification supports faster correction and integration of software defects.
- The SRS objective of implementing and deploying 90% of minor bug fixes within 24 hours is treated as a maintainability target.
- Achievement of this target must be demonstrated using defect tracking and deployment records rather than assumed from the existence of the CI/CD pipeline.

2. Version Control and Auditability

- GitHub maintains the history of source-code changes, Pull Requests, and reviews.
- GitHub Actions records the outcomes of automated verification workflows.
- Vercel and Render deployment histories provide additional information about deployed application revisions.
- The final release-management process uses Semantic Versioning, with v1.0.0 as the target initial release.
- Approved releases should maintain traceability between Git commits, CI results, deployment records, and release documentation.

This traceability supports troubleshooting, academic evaluation, and controlled software maintenance.

3. Reduced Deployment Risk

- Automated tests and quality checks help identify defects before approved release integration.
- Pull Request reviews reduce the likelihood of unreviewed changes entering the shared codebase.
- Deployment smoke tests help detect configuration, connectivity, and application-startup problems.
- Backend health checks support operational verification.
- Controlled deployment rollback procedures are documented for both Vercel and Render.

The current infrastructure documentation describes rollback to a previously verified deployment through the appropriate hosting platform.

Fully automatic rollback triggered by failed health checks has not been independently verified and is not claimed as an implemented feature.

4. Deployment Consistency and Portability

- Docker provides a consistent execution environment for the backend.
- The same core Express application can run locally and on compatible cloud-container platforms.
- Environment variables separate deployment-specific configuration from application source code.
- The deployment architecture can be modified in the future without requiring fundamental changes to the business logic.

5. Security and Release Governance

- Backend CI includes dependency vulnerability checking for applicable events.
- Frontend and backend verification pipelines support consistent software quality.
- Sensitive deployment credentials are maintained through approved server-side configuration.

- Release acceptance follows the project's documented quality-gate and sign-off procedures.
- Critical end-to-end, security, performance, and documentation evidence must be completed before a final release is declared fully approved.

CI/CD Implementation Limitations

The implemented CI/CD solution provides automated testing and managed cloud deployment capabilities. However, the following distinctions are maintained:

- The active backend hosting platform is Render.com, not AWS ECS Fargate.
- The active frontend hosting platform is Vercel, tracking the approved develop branch.
- The historical AWS staging pipeline is retained but is not the active deployment path.
- A separate cloud staging environment is not provisioned under DDP-083.
- Automatic promotion from main to AWS production infrastructure is not implemented.
- Guaranteed zero-downtime rolling updates are not independently verified.
- Automatic rollback on health-check failure is not independently verified.
- Complete CI-gated deployment behaviour requires verification against the actual hosting-provider and branch-protection settings.

The CI/CD design therefore reflects the implemented project workflows while distinguishing automated quality assurance, cloud deployment functionality, and release-management controls that require operational verification.

Chapter 8: References

[1] J. Nielsen, "10 Usability Heuristics for User Interface Design," Nielsen Norman Group, 1994, reviewed Jan. 2024. [Online]. Available: https://www.nngroup.com/articles/ten-usability-heuristics/ [Accessed: Oct. 25, 2026].

[2] Interaction Design Foundation, "User Experience Design," [Online]. Available: https://ixdf.org/literature/topics/ux-design [Accessed: Oct. 25, 2026].

[3] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[4] Vercel, "Deploying to Vercel," Vercel Documentation. [Online]. Available: https://vercel.com/docs/deployments [Accessed: Oct. 25, 2026].

[5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed. New York, NY, USA: McGraw-Hill, 2010.

[6] Object Management Group, "Unified Modeling Language (UML), Version 2.5.1," 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1 [Accessed: Oct. 25, 2026].

[7] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, updated May 2023. [Online]. Available: https://csrc.nist.gov/pubs/fips/197/final [Accessed: Oct. 25, 2026].

[8] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446.html [Accessed: Oct. 25, 2026].

[9] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/WCAG21/ [Accessed: Oct. 25, 2026].

[10] PayHere, "Checkout API," PayHere Knowledge Base. [Online]. Available: https://support.payhere.lk/api-%26-mobile-sdk/checkout-api.I [Accessed: Oct. 25, 2026].

[11] React Team, "React Documentation: Quick Start." [Online]. Available: https://react.dev/learn [Accessed: Oct. 25, 2026].

[12] Vite, "Getting Started," Vite Documentation. [Online]. Available: https://vite.dev/guide/ [Accessed: Oct. 25, 2026].

[13] Express.js, "Routing," Express Documentation. [Online]. Available: https://expressjs.com/en/guide/routing/ [Accessed: Oct. 25, 2026].

[14] MongoDB, Inc., "MongoDB Atlas Documentation." [Online]. Available: https://www.mongodb.com/docs/atlas/ [Accessed: Oct. 25, 2026].

[15] Mongoose, "Mongoose Guides and Documentation." [Online]. Available: https://mongoosejs.com/docs/guides.html [Accessed: Oct. 25, 2026].

[16] Render, "Docker on Render," Render Documentation. [Online]. Available: https://render.com/docs/docker [Accessed: Oct. 25, 2026].

[17] Cloudinary, "Node.js Image and Video Upload," Cloudinary Documentation. [Online]. Available: https://cloudinary.com/documentation/node_image_and_video_upload [Accessed: Oct. 25, 2026].

[18] Google AI for Developers, “Gemini API: Generating Content,” Google API Documentation. [Online]. Available: https://ai.google.dev/api/generate-content [Accessed: Oct. 25, 2026].
[19] Nodemailer, “SMTP Transport,” Nodemailer Documentation. [Online]. Available: https://nodemailer.com/smtp [Accessed: Oct. 25, 2026].
[20] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519.html [Accessed: Oct. 25, 2026].
[21] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: https://top10.owasp.org/2021/ [Accessed: Oct. 25, 2026].
[22] OWASP Foundation, “Application Security Verification Standard (ASVS),” OWASP Project Documentation. [Online]. Available: https://asvs.owasp.org/ [Accessed: Oct. 25, 2026].
[23] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection Act, No. 9 of 2022*. Sri Lanka, 2022. [Online]. Available: https://www.dpa.gov.lk/guidelines.php [Accessed: Oct. 25, 2026].
[24] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection (Amendment) Act, No. 22 of 2025*. Sri Lanka, 2025. [Online]. Available: https://www.dpa.gov.lk/guidelines.php [Accessed: Oct. 25, 2026].
[25] GitHub, “GitHub Actions Documentation,” GitHub Docs. [Online]. Available: https://docs.github.com/en/actions [Accessed: Oct. 25, 2026].
[26] Docker, Inc., “Dockerfile Reference,” Docker Documentation. [Online]. Available: https://docs.docker.com/reference/dockerfile [Accessed: Oct. 25, 2026].
[27] Darshana Opticals OSMS Development Team, “Darshana Opticals User Interface Design Prototype,” Figma, 2026. [Online]. Available: https://www.figma.com/proto/OecN23wEqdSIWsY4xjWwUQ/Darshana-Opticals
[28] Darshana Opticals OSMS Development Team, “ADR-015: Staging Backend Container Hosting Fallback,” OSMS Architectural Decision Records, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-015-staging-backend-container-hosting-fallback.md
[29] Darshana Opticals OSMS Development Team, “ADR-012: Immutable Audit Trail Architecture,” OSMS Architectural Decision Records, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-012-immutable-audit-trail-architecture.md
[30] Darshana Opticals OSMS Development Team, “OSMS Frontend Deployment — Vercel and Render (DDP-083),” OSMS Infrastructure Documentation, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra/blob/develop/docs/frontend-vercel-deployment.md